Faster Short Pairing-Based NIZK Proofs for Ring LWE Ciphertexts

Olivier Bernard, Sarah Elkazdadi, Benoît Libert,
Arthur Meyre, Jean-Baptiste Orfila, and Nicolas Sarlin

Zama, France

Abstract. Several works explored the use of discrete-logarithm-based zero-knowledge proof systems in order to prove the validity of Ring LWE ciphertexts and/or FHE ciphertexts. A technique suggested by del Pino *et al.* (PKC'19) notably enables proofs of 1KB for the task of proving the validity of NewHope ciphertexts using a variant of BulletProofs. A recent work of Libert (PKC'24) described a pairing-based adaptation of del Pino *et al.*'s approach with proofs of 3 or 6 group elements. While space-efficient, the latter solution is rather expensive in terms of proving time. In this work, we provide new NIZK arguments for the Ring-LWE-based public-key scheme proposed by Joye (CT-RSA'24), which is used in a variant of TFHE. The new schemes feature slightly longer proofs than in earlier pairing-based constructions with short proofs, but the prover is much faster. The number of exponentiations is reduced by a factor ≈ 7 and the common reference string is compressed by a factor ≈ 9 (and reduced to 1.5MB for practically relevant parameters). We provide implementation results that confirm these estimations.

Keywords. NIZK proofs, Ring LWE and FHE ciphertexts.

1 Introduction

Advanced FHE-based protocols often require an encryptor to provide evidence that a ciphertext was properly encrypted, which incurs to demonstrate knowledge of the plaintext and the corresponding encryption randomness. For this purpose, several recent applications of FHE to private smart contracts [12,40,11] consider the use of succinct non-interactive arguments, where the proof size grows sub-linearly with the length of the statement and the witness. The use of discrete-logarithm-based argument systems was sometimes preferred [14,40] as they tend to be more economical in terms of communication. For example, [40] used a proof of plaintext knowledge based on a technique suggested by del Pino *et al.* [14], which was itself built upon BulletProofs [8].

In [27], Libert described short NIZK argument showing the validity of TFHE ciphertexts [10] (in the variant considered by Joye [24]) using vector commitments [29,9]. While the latter construction enables very short proofs (either 3 or 6 group elements in pairing-friendly groups depending on the preferred tradeoff), its prover is rather slow. For practically relevant parameters, generating a proof takes several seconds on a laptop, and can even exceed 10s if the prover is executed in the browser. The main reason is that the NIZK argument of [27] builds

on an adaptation of del Pino *et al.*'s technique [14], which requires to encode the statement in relatively large vectors that have to be committed to using Pedersen commitments [36]. For a Ring LWE dimension $d = 2048$, a ciphertext modulus $q \approx 2^{64}$, a plaintext modulus $t = 2^5$, and a standard deviation $\sigma = 2^{12}$, the dimension of committed vectors has to be as large as $n = 65000$ when $k = 320$ message components are packed in a given ciphertext. As a result, computing a proof costs over 400000 exponentiations in a pairing-friendly group. In addition, the scheme uses a structured common reference string (CRS) containing $O(n)$ group elements and whose size is roughly 15MB.

1.1 Our Contribution

We present a variant of the scheme in [27] allowing to reduce the dimension of committed vectors down to $n \approx 10000$ or even $n \approx 7000$ (depending on the length of encrypted messages) for standard parameters. As a result, the CRS size is shrunk by a factor > 9 and drops to 1.5 MB. The number of exponentiations done by the prover is reduced by a factor ≈ 7. At the same time, we retain the useful properties of the scheme. For example, we can still prove other statements (like equalities/inequalities between distinct plaintexts or Hamming weight bounds on these plaintexts) about encrypted data without any change in the CRS. Like [27], the resulting NIZK argument is proven simulation-extractable (i.e., it preserves knowledge-soundness even against malicious provers attempting to maul honest users' proofs) in the combined algebraic group [16] and random oracle model under the q-discrete-logarithm assumption in asymmetric pairings.

We describe two variants of the scheme. In the slow-verifier variant, the verifier has to perform $O(n)$ exponentiations but proofs only cost 7 group elements, or ≈ 0.5KB on a BLS12-446 pairing-friendly curve. We then provide a fast-verifier variant with slightly longer proofs, where the verifier only computes $O(n)$ field operations in $\mathbb{Z}_p$ (where p is the group order) and a much smaller number of exponentiations. In the fast-verifier variant, we actually suggest two tradeoffs between the proof size and the verification time, which only slightly differ in the number of exponentiations at the verifier. In these fast-verifier variants, the proof length increases to 11 or 13 group elements, of which the total length does not exceed 1KB.

For these schemes, we provide implementation results in Rust, run natively and in a browser *via* a WebAssembly (WASM) binary and show that they significantly outperform [27] as far as the proving time is concerned. In WASM, the proving time ranges between 1.7 and 2.0s, instead of 8s previously. We also report very competitive verification timings. Note that WASM timings are interesting since they are representative of a concrete end-user experience.

One minor caveat is that, when it comes to proving the smallness of the noise in valid ciphertexts, our construction is able to avoid a soundness slack as long as the proven bound for the noise is on its Euclidean norm (similarly to arguments based on modular Johnson-Lindenstrauss projections [20]). If we aim at proving a bound on its infinity norm, we end up with a moderate slack due to the standard norm equivalence relation $\|\mathbf{x}\|_\infty \leq \|\mathbf{x}\|_2 \leq \sqrt{d} \cdot \|\mathbf{x}\|_\infty$, where $\mathbf{x} \in \mathbb{Z}^d$.

Hence, if the prover's witness contains a noise $\mathbf{e} \in \mathbb{Z}^d$ such that $\|\mathbf{e}\|_\infty \leq B_\infty$, it can only convince the verifier that $\|\mathbf{e}\|_\infty \leq \sqrt{d} \cdot B_\infty$. In FHE applications that have to guarantee a worst-case bound on the variance of the decryption error, a bound on the infinity norm is usually necessary and parameters must be chosen while taking the slack into account. On the other hand, for applications that only require to prove that a fresh ciphertext decrypts correctly, proving an upper bound on the Euclidean norm is sufficient. In this case, we can prove a bound $\|\boldsymbol{e}\|_2 \leq B$ on the Euclidean norm without any slack.

1.2 Technical Overview

The NIZK construction of [27] relies on the vector commitment of [29] and its extension [28]. In asymmetric pairings $e : \mathbb{G} \times \hat{\mathbb{G}} \to \mathbb{G}_T$, the scheme uses a CRS of the form $\left(\{g^{\alpha^i}\}_{i=1}^n, \{\hat{g}^{\alpha^i}\}_{i \in [2n] \setminus \{n+1\}}\right)$ for a secret $\alpha \in \mathbb{Z}_p$. A commitment to $\boldsymbol{x} = (x_1, \ldots, x_n) \in \mathbb{Z}_p^n$ consists of $\hat{C} = \hat{g}^{\gamma + \sum_{i=1}^n x_i \cdot \alpha^i}$ for a random $\gamma \in \mathbb{Z}_p$. As shown in [28], the committer can convince a verifier that $\langle \boldsymbol{x}, \boldsymbol{y} \rangle = z$, for some public $(z, \boldsymbol{y}) \in \mathbb{Z}_p \times \mathbb{Z}_p^n$, by using the fact that $\langle \boldsymbol{x}, \boldsymbol{y} \rangle$ is the coefficient of α^{n+1} in the polynomial product $(\gamma + \sum_{i=1}^n x_i \cdot \alpha^i) \cdot (\sum_{i=1}^n y_i \cdot \alpha^{n+1-i})$. From the CRS, the committer can then generate a short proof $\pi_z \in \mathbb{G}$ such that $e(\prod_{i=1}^n g^{y_i \cdot (\alpha^{n+1-i})}, \hat{C}) = e(g^\alpha, \hat{g}^{(\alpha^n)})^z \cdot e(\pi_z, \hat{g})$. The latter verification convinces the verifier that $\langle \boldsymbol{x}, \boldsymbol{y} \rangle = z$ since, otherwise, computing π_z would be infeasible. Indeed, without knowing $g^{(\alpha^{n+1})}$, the monomial α^{n+1} can be obtained in the exponent in $\mathbb{G}_T$, but not in the source groups $(\mathbb{G}, \hat{\mathbb{G}})$.

Using the above commitment, Libert [27] adapted the technique of del Pino *et al.* [14] to prove the validity of RLWE ciphertexts using shorter proofs. The main bottleneck in the proving time of [27] was the large number of exponentiations induced by the large dimension of committed vectors. This large dimension was a consequence of the approach of breaking the noise terms into bits.

In order to provide a more accurate intuition, we need to first recall that approach of del Pino *et al.* [14] and its adaptation in [27]. We will then explain the way we can make the prover faster without sacrificing the short proof length.

DEL PINO *et al.*'S APPROACH. Let the polynomial rings $R = \mathbb{Z}[X]/(X^d + 1)$ and $R_q = R/(qR)$, where d is a power of two. Proving the validity of an (R)LWE ciphertext can be reduced to proving the existence of small-norm ring elements $\boldsymbol{s} = (s_1, \ldots, s_M) \in R^M$ such that

$$\sum_{i=1}^{M} \boldsymbol{a}_i \cdot s_i = \boldsymbol{c} \bmod (q, X^d + 1) \tag{1}$$

where $\boldsymbol{c} \in R_q^N$ is the ciphertext and $\boldsymbol{a}_1, \ldots, \boldsymbol{a}_M \in R_q^N$ form the public key. The relation is defined as the set of pairs

$$(\mathbf{x}, \mathbf{w}) = \Big((\boldsymbol{c}, \boldsymbol{a}_1, \ldots, \boldsymbol{a}_M) \in (R_q^N)^{M+1}, (\boldsymbol{s}_1, \ldots, \boldsymbol{s}_M) \in R^M \Big)$$

satisfying (1). To prove this relation, del Pino *et al.* [14] re-write (1) as the following equality over $\mathbb{Z}[X]/(X^d+1)$ (without any reduction mod$q$)

$$\sum_{i=1}^{M} \boldsymbol{a}_i \cdot s_i = \boldsymbol{c} + \boldsymbol{r} \cdot q \bmod (X^d+1), \tag{2}$$

where $\boldsymbol{r} = (r_1, \ldots, r_N)^\top \in R^N$ is a vector of polynomials of degree $\leq d-1$ and the components of $\{\boldsymbol{a}_i\}_{i=1}^M$ and $\boldsymbol{c}$ are interpreted as polynomials with coefficients in $\{-\lfloor q/2 \rfloor, \ldots, \lfloor q/2 \rfloor\}$. If $\|s_i\|_\infty \leq B_i$ for each $i \in [M]$, $\boldsymbol{r}$ contains polynomials with coefficients bounded by $\|\boldsymbol{r}\|_\infty \leq B_r \triangleq dM \cdot \max_{i\in[M]}(B_i)/2$.

Let $\boldsymbol{a}_i = (a_{i,1}, \ldots, a_{i,N})^\top \in R_q^N$. Let the coefficient embedding $\phi : R \to \mathbb{Z}^d$ that maps s_i to its coefficient vector $\phi(s_i) \in \mathbb{Z}^d$. Let $\mathsf{rot}(a_{i,j}) \in \mathbb{Z}^{d\times d}$ the anti-circulant matrix such that $\phi\big(a_{i,j} \cdot s_i \bmod (X^d+1)\big) = \mathsf{rot}(a_{i,j}) \cdot \phi(s_i) \in \mathbb{Z}^d$. Then, (2) can be written as the following linear relation over $\mathbb{Z}$

$$[\mathbf{A}_1 \mid \ldots \mid \mathbf{A}_M \mid \ -q \cdot \mathbf{I}_{Nd}\ ] \cdot \underbrace{\big[\phi(s_1) \mid \ldots \mid \phi(s_M) \mid \phi(\boldsymbol{r})\big]^\top}_{\triangleq\ \bar{\boldsymbol{w}}} = \phi(\boldsymbol{c}), \tag{3}$$

where $\mathbf{A}_i = [\mathsf{rot}(a_{i,1})^\top \mid \ldots \mid \mathsf{rot}(a_{i,N})^\top]^\top \in \mathbb{Z}_q^{Nd\times d}$ for all $i \in [M]$, $\phi(\boldsymbol{c}) \in \mathbb{Z}_q^{Nd}$, and $\phi(\boldsymbol{r}) \in \mathbb{Z}^{Nd}$.

In order to prove (3), the idea of [14] is to commit to $\bar{\boldsymbol{w}} \in \mathbb{Z}^{Md+Nd}$ and succinctly prove that its components have small infinity norm. Then, the prover shows that (3) holds over $\mathbb{Z}_p$,[1] where p is the discrete-logarithm group order. If p is sufficiently larger than $\max_i(B_i)$ and B_r, this ensures that (3) also holds over the integers.

Instead of explicitly using a range proof, the proof can be made shorter by directly committing to the bits of $\bar{\boldsymbol{w}}$. For any integer $z \in \mathbb{Z}$, define $\boldsymbol{g}_z = (1, 2, 4, \ldots, 2^{z-2}, -2^{z-1})^\top \in \mathbb{Z}^{1\times z}$ and $\boldsymbol{G}_z = \mathbf{I}_d \otimes \boldsymbol{g}_z^\top \in \mathbb{Z}^{d\times dz}$. We also define $\mathbf{G}_z^{-1}(\boldsymbol{v})$ as the decomposition function that inputs an integer vector $\boldsymbol{v} \in [-2^{z-1}, 2^{z-1}-1]^d$ and outputs a decomposition $\boldsymbol{G}_z^{-1}(\boldsymbol{v}) \in \{0,1\}^{d\cdot z}$ such that $\boldsymbol{G}_z \cdot \boldsymbol{G}_z^{-1}(\boldsymbol{v}) = \boldsymbol{v}$. Then, if we define

$$\tilde{\mathbf{A}}_i \triangleq \big[\mathbf{G}_{1+\log B_i}^\top \cdot \mathsf{rot}(a_{i,1})^\top \mid \ldots \mid \mathbf{G}_{1+\log B_i}^\top \cdot \mathsf{rot}(a_{i,N})^\top\big]^\top \in \mathbb{Z}_q^{Nd\times d(1+\log B_i)}$$

for each $i \in [M]$, one can prove that holds (3) for a small $\bar{\boldsymbol{w}}$ by proving that

$$\underbrace{\big[\ \tilde{\mathbf{A}}_1 \ \ \ldots \ \ \tilde{\mathbf{A}}_M \ \mid \ -q \cdot \big(\mathbf{I}_N \otimes \boldsymbol{G}_{1+\log B_r}\big)\ \big]}_{\triangleq\ \tilde{\mathbf{A}}} \cdot \underbrace{\big[\boldsymbol{s}_1 \mid \ldots \mid \boldsymbol{s}_M \,\big\|\, \boldsymbol{r}_1 \mid \ldots \mid \boldsymbol{r}_N\big]^\top}_{\triangleq\ \tilde{\boldsymbol{w}}} = \phi(\boldsymbol{c}), \tag{4}$$

where $\{\boldsymbol{s}_i = \boldsymbol{G}_{1+\log B_i}^{-1}(\phi(s_i))\}_{i\in[M]}$ and $\{\boldsymbol{r}_i = \boldsymbol{G}_{1+\log B_r}^{-1}(\phi(r_i))\}_{i\in[N]}$ are binary decompositions of the components of $\bar{\boldsymbol{w}}$.

The pairing-based variant. In Libert's construction of [27], the prover commits to the binary decomposition $\tilde{\boldsymbol{w}}$ used in (4) and proves that $\tilde{\boldsymbol{w}}$ is indeed

[1] Here, $x \bmod p$ is defined as the value $y \in (-p/2, p/2)$ such that $y \equiv x \pmod p$

binary. In order to prove that relation (4) holds modulo p (and thus also over $\mathbb{Z}$ since both members have infinity norm smaller than $p/2$), the prover uses a random $\boldsymbol{\theta} \in \mathbb{Z}_p^{Nd}$ (derived from a random oracle) and proves that the committed $\tilde{\boldsymbol{w}} \in \{0,1\}^D$ satisfies $\boldsymbol{\theta}^\top \cdot \tilde{\mathbf{A}} \cdot \tilde{\boldsymbol{w}} = \boldsymbol{\theta}^\top \cdot \phi(\boldsymbol{c}) \bmod p$. If $\tilde{\mathbf{A}} \cdot \tilde{\boldsymbol{w}} \neq \phi(\boldsymbol{c}) \bmod p$, then we have $\boldsymbol{\theta}^\top \cdot (\tilde{\mathbf{A}} \cdot \tilde{\boldsymbol{w}} - \phi(\boldsymbol{c})) = 0 \bmod p$ with probability $1/p$. Proving $\boldsymbol{\theta}^\top \cdot \tilde{\mathbf{A}} \cdot \tilde{\boldsymbol{w}} = \boldsymbol{\theta}^\top \cdot \phi(\boldsymbol{c})$ is possible using one element of $\mathbb{G}$ when $\tilde{\boldsymbol{w}}$ is committed using the commitment scheme of [29] and its inner-product functionality [28].

In order to prove the validity of a ciphertext in the public-key TFHE scheme of [24], the NIZK argument of [27] adapts (3) into a linear relation of the form

$$\underbrace{\begin{bmatrix} \mathsf{rot}(a) & \mathbf{I}_d & -q \cdot \mathbf{I}_d \\ \phi(b)^\top & \Delta \quad 1 & -q \end{bmatrix}}_{\triangleq\, \bar{\mathbf{A}}} \cdot \underbrace{\begin{bmatrix} \phi(r) \\ m \\ \phi(e_1) \\ e_2 \\ \phi(r_1) \\ r_2 \end{bmatrix}}_{\triangleq\, \bar{\boldsymbol{w}}} = \underbrace{\begin{bmatrix} \phi(c_1) \\ c_2 \end{bmatrix}}_{\triangleq\, \bar{\boldsymbol{c}}}. \tag{5}$$

for a public matrix $\bar{\mathbf{A}}$ and a secret vector $\bar{\boldsymbol{w}}$. In the witness, $m \in R/(tR)$ is the plaintext (where t is the plaintext modulus), $r \in R/(2R)$ is the RLWE secret involved in the ciphertext, $e_1 \in R$, $e_2 \in \mathbb{Z}$ are noise terms, and $r_1 \in R$, $r_2 \in \mathbb{Z}$ are quotients obtained by rewriting the relations over the integers. To prove (5), the prover of [27] decomposes the witness into bits, so that relation (3) becomes

$$\underbrace{\begin{bmatrix} \mathsf{rot}(a) & \boldsymbol{G}_{1+\log B} & -q \cdot \boldsymbol{G}_{\log d} \\ \phi(b)^\top & \Delta \cdot \boldsymbol{g}_{\log t}^\top \quad \boldsymbol{g}_{1+\log B}^\top & -q \cdot \boldsymbol{g}_{\log d}^\top \end{bmatrix}}_{\triangleq\, \tilde{\mathbf{A}} \,\in\, \mathbb{Z}^{(d+1)\times D}} \cdot \underbrace{\begin{bmatrix} \phi(r) \\ \boldsymbol{m} \\ \boldsymbol{e}_1 \\ \boldsymbol{e}_2 \\ \boldsymbol{r}_1 \\ \boldsymbol{r}_2 \end{bmatrix}}_{\triangleq\, \tilde{\boldsymbol{w}}} = \underbrace{\begin{bmatrix} \phi(c_1) \\ c_2 \end{bmatrix}}_{\triangleq\, \tilde{\boldsymbol{c}}}, \tag{6}$$

where[2] $\boldsymbol{m} = \boldsymbol{g}_{\log t}^{-1}(m) \in \{0,1\}^{\log t}$ contains the bits of the plaintext $m \in R/(tR)$; $\boldsymbol{e}_1 = \boldsymbol{G}_{1+\log B}^{-1}(\phi(e_1)) \in \{0,1\}^{d(1+\log B)}$ and $\boldsymbol{e}_2 = \boldsymbol{g}_{1+\log B}^{-1}(e_2) \in \{0,1\}^{1+\log B}$ contain the binary decomposition of the noise; and $\boldsymbol{r}_1 = \boldsymbol{G}_{\log d}^{-1}(r_2) \in \{0,1\}^{d \log d}$, and $\boldsymbol{r}_2 = \boldsymbol{g}_{\log d}^{-1}(r_2) \in \{0,1\}^{\log d}$ are the binary decompositions of the quotient terms in (5). The prover then uses the vector commitment scheme of [29] to

[2] The message $m \in \mathbb{Z}_t$ is interpreted as an integer in the interval $[-t/2, t/2)$, so that the gadget vector $\boldsymbol{g}_{\log t} = (1, 2, 4, \ldots, 2^{\log t - 2}, -2^{\log t - 1}) \in \mathbb{Z}^{1 \times \log t}$ can be used to express its decomposition.

commit to the witness $\tilde{\boldsymbol{w}} \in \{0,1\}^D$, where $D = d + \log t + (d+1)(1 + \log B + \log d)$.

The main overhead of [27] stems from the relatively large D. For typical parameters such as $d = 2048$, $t = 32$ and $B = 2^{16}$, we have $D = 59425$. In the faster-verifier variant of [27], the prover has to compute about $6D \approx 356000$ exponentiations in $\mathbb{G}$, which can be quite slow (in particular if the prover is implemented in a browser) even with some amount of parallelism.

The main source of inefficiency is the approach of directly decomposing noise terms into bits. Concretely, to prove that a noise vector $\phi(e_1) \in \mathbb{Z}^d$ of dimension $d = 2048$ has infinity norm $\leq B \approx 2^{16}$, one has to commit to a sub-vector of dimension $2048 \times 16 = 32768$, which amounts to more than 50% of D.[3] This decomposition step has to be repeated for $\phi(r_1) \in \mathbb{Z}^d$ whose entries are contained in $[-(d+1)/2, (d+1)/2]$, which costs another sub-vector of dimension $2048 \times 11 \approx 22500$. At the end of the day, the direct decomposition method of [27] is doomed to commit to vectors of dimension > 50000 and compute several multi-exponentiations of the same dimension.

Our Approach. We adapt an approach previously considered in lattice-based proof systems [4,31,20,5,34]. We do not directly commit to the binary decompositions of short vectors. Instead, we can first project them to smaller dimensions and then prove that projected vectors are small. Gentry *et al.* [20] used a similar technique to build a lattice-based verifiable encryption using a discrete-logarithm-based instantiation of BulletProofs [8]. A difference is that they used an (M)LWE modulus that is large as the discrete-logarithm-hard group order p. Here, we follow [14] in that we assume that the RLWE modulus is significantly smaller than p. At the same time, we use modular projections as in [20] and we rely on pairing-based vector-commitment techniques to concisely prove that committed vectors are projections of one another.

In [4,31], it was proven that, for a flat matrix $\mathbf{R} \in \{-1,0,1\}^{\lambda \times \ell}$ sampled from a suitable distribution centered in 0, the infinity norm of $\mathbf{R} \cdot \mathbf{e} \bmod p$ is unlikely to be smaller than half of $\|\mathbf{e}\|_\infty$ when the probability is taken over the choice of $\mathbf{R}$. To prove the smallness of $\phi(e_1) \in \mathbb{Z}^d$ and $\phi(r_1) \in \mathbb{Z}^d$ in (5), we can thus project them to $\mathbf{w}_R = \mathbf{R} \cdot (\phi(e_1)^\top \mid \phi(r_1)^\top)^\top$ and only prove the smallness of $\mathbf{w}_R \in \mathbb{Z}^\lambda$ via binary decomposition, which is much cheaper since its entries are not too large and its dimension is only $\lambda = 128$ (instead of 2048 or more).

The bounds of [4,31] alone induce a slack since the infinity norm of $\boldsymbol{e}$ is only guaranteed to be no more than twice as large as the infinity norm of the product $\mathbf{R} \cdot \boldsymbol{e} \bmod p$, which is itself (heuristically) at most 10 times as large as the ℓ_2-norm of the initial $\boldsymbol{e}$. As a result, the upper bound for $\|\mathbf{R} \cdot \boldsymbol{e} \bmod p\|_\infty$ that can be shown by the prover only convinces the verifier that $\|\boldsymbol{e}\|_\infty$ is at most 20 times as large as the actual $\|\boldsymbol{e}\|_2$. However, this is not a problem for us since, like [20], we only use this trick to prove that no implicit reduction modulo p occurs when we prove the smallness of $\langle \boldsymbol{e}, \boldsymbol{e} \rangle \bmod p$ for a committed $\boldsymbol{e}$. The slacky bound then guarantees that $\langle \boldsymbol{e}, \boldsymbol{e} \rangle \bmod p$ is also $\langle \boldsymbol{e}, \boldsymbol{e} \rangle = \|\boldsymbol{e}\|_2^2$.

[3] Reducing the ring dimension to $d = 1024$ does not help since it requires the noise distribution to have larger standard deviation, which leads to $B > 2^{40}$.

In more details, we only partially decompose the witness into bits. We consider a packed variant of the scheme with k message-carrying ciphertext components, each of which has its own noise term $e_{2,i}$. We do not directly decompose the noise terms $(e_1, \{e_{2,i}\}_{i=1}^k)$ nor the quotients $(r_1, \{r_{2,i}\}_{i=1}^k)$ into bits. Instead, we keep them encoded as integer vectors, which are committed separately. In order to hide the actual ℓ_2-norm[4] of $\boldsymbol{e} = (e_1 \mid (e_{2,1}, \ldots, e_{e,k}))$ and only reveal an upper bound $\|\boldsymbol{e}\| \leq B$, we apply a trick used in [20] and append a tuple $(v_1, \ldots, v_4) \in \mathbb{Z}^4$ such that $\bar{\boldsymbol{e}} = (\boldsymbol{e}^\top \mid (v_1, \ldots, v_4)^\top)^\top \in \mathbb{Z}^{d+k+4}$ has norm exactly B. By Lagrange's four-square theorem, such integers $\{v_i \in [0, B)\}_{i=1}^4$ always exist and can be computed using the Rabin-Shallit algorithm [38].[5]

Then, we commit to $\bar{\boldsymbol{e}} \in \mathbb{Z}^{d+k+4}$ and prove that $\langle \bar{\boldsymbol{e}}, \bar{\boldsymbol{e}} \rangle = B^2 \bmod p$ by evaluating the inner product in the exponent using the pairing and generating a short proof for the inner product relation as in [28]. More precisely, our prover creates vector commitments $C_e \in \mathbb{G}$ and $\hat{C}_e \in \hat{\mathbb{G}}$ to $\bar{\boldsymbol{e}}$ in both source groups. It generates a short proof that C_e commits to the same vector as $\hat{C}_e$ (but in the reverse order) using aggregation techniques used in [21,27]. Then, it can generate a short $\pi_\ell \in \mathbb{G}$ such that $e(C, \hat{C}_e) = e(g, \hat{g})^{\alpha^{n+1} \cdot B^2} \cdot e(\pi_\ell, \hat{g})$.

In order to make sure that $\langle \bar{\boldsymbol{e}}, \bar{\boldsymbol{e}} \rangle$ does not wrap around modulo p (i.e., that $\langle \bar{\boldsymbol{e}}, \bar{\boldsymbol{e}} \rangle \bmod p$ is also $\langle \bar{\boldsymbol{e}}, \bar{\boldsymbol{e}} \rangle$ over $\mathbb{Z}$), we use the approximate range-proof technique of [4,31]. Namely, the prover computes a flat matrix $\mathbf{R} \in \{-1, 0, 1\}$ such that 0 occurs with probability $1/2$ and ± 1 with probability $1/4$ and convinces the verifier that $\mathbf{R} \cdot \bar{\boldsymbol{e}} \bmod p$ is small. This only convinces the verifier about a loose upper bound for $\|\bar{\boldsymbol{e}}\|_\infty$, but it suffices to guarantee that $\langle \bar{\boldsymbol{e}}, \bar{\boldsymbol{e}} \rangle \ll p$. In fact, we apply this technique to a vector $(\bar{\boldsymbol{e}} \mid \tilde{\boldsymbol{r}})$ consisting of the concatenation of $\bar{\boldsymbol{e}}$ and the quotients $\tilde{\boldsymbol{r}} = (\phi(r_1) \mid (r_{2,1}, \ldots, r_{2,k})) \in \mathbb{Z}^{d+k}$ of the Euclidean division in (6). The prover then demonstrates that $\|\mathbf{R} \cdot (\bar{\boldsymbol{e}}^\top \mid \tilde{\boldsymbol{r}}^\top)^\top \bmod p\|_\infty$ is small. Although this only proves a slacky bound on $\|(\bar{\boldsymbol{e}} \mid \tilde{\boldsymbol{r}})\|_\infty$, it will be sufficient to simultaneously convince the verifier that $\langle \bar{\boldsymbol{e}}, \bar{\boldsymbol{e}} \rangle \ll p$ *and* that (6) holds over $\mathbb{Z}$ (and not only over $\mathbb{Z}_p$) since both members have infinity norm smaller than $p/2$.

To prove that $\|\mathbf{R} \cdot (\bar{\boldsymbol{e}} \mid \tilde{\boldsymbol{r}})^\top \bmod p\|_\infty$ is smaller than some bound, the prover will create a separate commitment to the product $\mathbf{w}_R \triangleq \mathbf{R} \cdot (\bar{\boldsymbol{e}}^\top \mid \tilde{\boldsymbol{r}}^\top)^\top \bmod p$ and prove that it really commits to the product $\mathbf{R} \cdot (\bar{\boldsymbol{e}}^\top \mid \tilde{\boldsymbol{r}}^\top)^\top$. This part can be done using the inner-product functional commitment of [28]. The smallness of $\mathbf{R} \cdot (\bar{\boldsymbol{e}} \mid \tilde{\boldsymbol{r}})^\top \bmod p$ is then proven by decomposing its components into bits and using the short proof of binarity from [27]. Here, it will be much cheaper than in [27] because the dimension of $\mathbf{R} \cdot (\bar{\boldsymbol{e}} \mid \tilde{\boldsymbol{r}})^\top \bmod p$ is only $\lambda = 128$.

1.3 Related Work

The work of del Pino *et al.* [14] was the first one to consider the use of discrete-logarithm-based space-succinct arguments to prove the validity of RLWE ciphertexts without using a SNARK for general NP statements. By expressing

[4] We note that revealing the Euclidean norm of the noise would not break the security of the encryption scheme since the adversary can guess it with non-negligible probability. However, the proof would not be zero-knowledge if it was leaking it.

[5] This algorithm runs in time $O(\log^2 B)$ under the ERH. It was optimized in [30].

the statement as a linear relation over the integers, they were able to prove the validity of NewHope ciphertexts [2] for a much smaller modulus than the group order. While they obtained a 1.25-kilobyte proof size without relying on a trusted setup, their estimated prover/verifier costs are rather large (around 700000 and 200000 exponentiations at the prover and the verifier, respectively). Here, we can exploit the shape of our structured CRS to shift the most expensive exponentiations from the verifier to the prover, as previously done in [27].

The technique of del Pino *et al.* [14] was used in smartFHE [40] to prove the validity of a BFV ciphertext [15]. Their prover and verifier seemingly require a similarly large number of exponentiations.

Gentry *et al.* [20] also used BulletProofs to prove ring/module-LWE ciphertexts. In contrast with [14], they assume a large (M)LWE modulus that coincides with the discrete-log-group order. At the same time, they reduced the dimension of witness vectors by using modular projections and a modular version of the Johnson-Lindenstrauss lemma [23]. The use of modular projections in lattice-based zero-knowledge proofs was previously considered in [4,31]. A difference between this paper and [20] is that we use both modular projections and the trick of [14] to handle small RLWE moduli. To do this, we apply the slacky range proof technique used in [20, Section 3.4] to the noise $\boldsymbol{e}$ *and* the quotient polynomials $\tilde{\boldsymbol{r}}$ resulting from writing the proven linear relation (6) over the integers. In order the prove the smallness of $\langle \boldsymbol{e}, \boldsymbol{e} \rangle \bmod p$, we use a similar technique to [20] with the difference that the BulletProofs inner-product argument [8] is replaced by an inner-product-functional-commitment argument [28]. Gentry *et al.* [20] gave implementation results to demonstrate the feasibility of their approach in the context of large-scale publicly verifiable secret sharing.

Recently, Bottazzi [7, Section 7] provided a Rust implementation of zero-knowledge proofs of ciphertext validity for the BFV FHE [15]. The results are not directly comparable as they are obtained for different schemes (thus making the proven statements different) and on different machines. Our construction is nonetheless designed to have a faster prover in Rust even at the cost of a somewhat slower verifier. We note that, in applications like [11,40], one may prefer optimizing the prover rather than the verifier since proofs are usually verified by transaction validators that have more computational resources.

2 Background and Definitions

2.1 Hardness Assumptions

Let groups $(\mathbb{G}, \hat{\mathbb{G}}, \mathbb{G}_T)$ of prime order p with a bilinear map $e : \mathbb{G} \times \hat{\mathbb{G}} \to \mathbb{G}_T$, where $\mathbb{G} \neq \hat{\mathbb{G}}$ and no isomorphism from $\hat{\mathbb{G}}$ to $\mathbb{G}$ is efficiently computable.

Definition 1 ([16]). *Let $(\mathbb{G}, \hat{\mathbb{G}}, \mathbb{G}_T)$ be asymmetric bilinear groups of prime order p. For integers m, n, the (m,n)-**Discrete Logarithm** ((m,n)-DLOG) problem is, given $(g, g^{\alpha}, g^{(\alpha^2)}, \ldots, g^{(\alpha^m)}, \hat{g}, \hat{g}^{\alpha}, \ldots, \hat{g}^{(\alpha^n)})$, where $\alpha \xleftarrow{R} \mathbb{Z}_p$, $g \xleftarrow{R} \mathbb{G}$, $\hat{g} \xleftarrow{R} \hat{\mathbb{G}}$, to compute $\alpha \in \mathbb{Z}_p$.*

2.2 Non-interactive Arguments

Let $\{\mathcal{R}_\lambda\}_\lambda$ a family of NP relations. A NIZK argument for $\{\mathcal{R}_\lambda\}_\lambda$ consists of algorithms $\Pi = (\mathsf{CRS\text{-}Gen}, \mathsf{Prove}, \mathsf{Verify})$ with the following specifications. On input of a security parameter $\lambda \in \mathbb{N}$ and, optionally, language-dependent parameters $\mathsf{lpp} \in \mathcal{P}$ for some parameter space $\mathcal{P}$, algorithm $\mathsf{CRS\text{-}Gen}$ generates a common reference string pp and a simulation trapdoor τ. We allow pp to parameterize the proven relation (when it specifies the public parameters of a commitment scheme), which then becomes $\mathcal{R}_{\mathsf{pp}} \in \{\mathcal{R}_\lambda\}_\lambda$. Algorithm Prove takes as input the common reference string pp, a statement x and a witness w and outputs a proof π when $(x, w) \in \mathcal{R}_{\mathsf{pp}}$. Verify takes in pp, a statement x and a proof π and returns 0 or 1. Correctness requires that, for any $\mathcal{R} \in \{\mathcal{R}_\lambda\}_\lambda$ and any $(x, w) \in \mathcal{R}_{\mathsf{pp}}$, honestly generated proofs are always (or at least with overwhelming probability) accepted by the verifier.

NIZK arguments should satisfy two security properties. The *zero-knowledge* property requires that proofs leak no information about the witness. *Knowledge-soundness* property requires that there exists an extractor that can compute a witness whenever the adversary generates a valid proof. The extractor has access to the adversary's internal state, including its random coins. Let the universal relation $\mathcal{R}^*$ for $\{\mathcal{R}_\lambda\}_\lambda$ that inputs $(\mathcal{R}_{\mathsf{pp}}, x, w)$ and outputs 1 iff $\mathcal{R}_{\mathsf{pp}} \in \{\mathcal{R}_\lambda\}_\lambda$ and $(x, w) \in \mathcal{R}_{\mathsf{pp}}$. We say that $\Pi = (\mathsf{CRS\text{-}Gen}, \mathsf{Prove}, \mathsf{Verify})$ is a NIZK argument for $\mathcal{R}^*$ if it satisfies the properties defined as follows.

Completeness: For any $\lambda \in \mathbb{N}$ and $\mathsf{lpp} \in \mathcal{P}$, any (not necessarily efficient) adversary $\mathcal{A}$, there is a negligible function $\mathsf{negl} : \mathbb{N} \to \mathbb{N}$ such that

$$\Pr\big[\mathsf{Verify}_{\mathsf{pp}}(x, \pi) = 1 \ \vee \ (x, w) \notin \mathcal{R}_{\mathsf{pp}} \ \mid \ (\mathsf{pp}, \tau) \leftarrow \mathsf{CRS\text{-}Gen}(\lambda, \mathsf{lpp}), \\ (x, w) \leftarrow \mathcal{A}(\mathsf{pp}) \ , \ \pi \leftarrow \mathsf{Prove}_{\mathsf{pp}}(x, w)\big] = 1 - \mathsf{negl}(\lambda).$$

Knowledge-soundness: For any $\lambda \in \mathbb{N}$ and $\mathsf{lpp} \in \mathcal{P}$, any PPT adversary $\mathcal{A}$, there is a PPT extractor $\mathcal{E}_{\mathcal{A}}$ that has access to $\mathcal{A}$'s internal state and random coins ρ such that

$$\Pr\big[\mathsf{Verify}_{\mathsf{pp}}(x, \pi) = 1 \ \wedge \ (x, w) \notin \mathcal{R}_{\mathsf{pp}} \ \mid \ (\mathsf{pp}, \tau) \leftarrow \mathsf{CRS\text{-}Gen}(\lambda, \mathsf{lpp}), \\ (x, \pi) \leftarrow \mathcal{A}(\mathsf{pp}; \rho), \ w \leftarrow \mathcal{E}_{\mathcal{A}}(\mathsf{pp}, (x, \pi), \rho)\big] = \mathsf{negl}(\lambda).$$

(Statistical) Zero-knowledge: for any $\lambda \in \mathbb{N}$, $\mathsf{lpp} \in \mathcal{P}$, there is a PPT simulator Sim such that, for any (possibly inefficient) adversary $\mathcal{A}$,

$$\Pr\big[b \leftarrow \mathcal{A}^{\mathcal{O}_b}(\mathsf{pp}) \mid (\mathsf{pp}, \tau) \leftarrow \mathsf{CRS\text{-}Gen}(\lambda, \mathsf{lpp}), \ b \xleftarrow{R} \{0, 1\}\big] = 1/2 + \mathsf{negl}(\lambda).$$

where $\mathcal{O}_1$ is an oracle that inputs (x, w) and returns $\pi \leftarrow \mathsf{Prove}_{\mathsf{pp}}(x, w)$ if $(x, w) \in \mathcal{R}_{\mathsf{pp}}$ and $\perp$ otherwise; $\mathcal{O}_0$ is oracle that inputs (x, w) and returns $\pi \leftarrow \mathsf{Sim}(\mathsf{pp}, \tau, x)$ if $(x, w) \in \mathcal{R}_{\mathsf{pp}}$ and $\perp$ otherwise.

For applications in multi-user environments, knowledge-soundness against stand-alone provers appears insufficient as a security notion. Simulation-extractability

is arguably a more appropriate notion (notably considered in [12]) as it prevents malicious users from hijacking proofs from one another. It considers an adversary that can observe simulated proofs (for possibly false statements) and exploit some malleability of these proofs to generate a fake proof of its own.

Simulation-Extractability: For any $\lambda \in \mathbb{N}$, $\mathsf{lpp} \in \mathcal{P}$, any PPT $\mathcal{A}$, there is a PPT extractor $\mathcal{E}_{\mathcal{A}}$ with access to $\mathcal{A}$'s internal state/randomness ρ such that

$$\Pr\left[\mathsf{Verify}_{\mathsf{pp}}(x,\pi)=1 \ \wedge \ (x,w)\notin\mathcal{R}_{\mathsf{pp}} \ \wedge \ (x,\pi)\notin Q \mid (\mathsf{pp},\tau)\leftarrow \mathsf{CRS\text{-}Gen}(\lambda,\mathsf{lpp}),\right.$$
$$\left.(x,\pi)\leftarrow\mathcal{A}^{\mathsf{SimProve}}(\mathsf{pp};\rho),\ w\leftarrow\mathcal{E}_{\mathcal{A}}(\mathsf{pp},(x,\pi),\rho,Q)\right]=\mathsf{negl}(\lambda),$$

where $\mathsf{SimProve}(\mathsf{pp},\tau,\cdot)$ is an oracle that returns a simulated proof $\pi \leftarrow \mathsf{Sim}(\mathsf{pp},\tau,x)$ for a given statement x and $Q=\{(x_i,\pi_i)\}_i$ denotes the set of queried statements and the simulated proofs returned by $\mathsf{SimProve}$.

2.3 Algebraic Group Model

The algebraic group model (AGM) [16] is an idealized model, where all algorithms are assumed algebraic. Algebraic algorithms [6,35] generalize the notion of generic ones [39] in the sense that, whenever they compute a group element, they do it by taking linear combinations of available group elements so far. Therefore, whenever they output a group element $X \in \mathbb{G}$, they also output a representation $\{\alpha_i\}_{i=1}^N$ of $X=\prod_{i=1}^N g_i^{\alpha_i}$ as a function of all previously observed elements $(g_1,\ldots,g_N)\in\mathbb{G}^N$ in the same group.

Unlike generic algorithms, algebraic algorithms can take advantage of the group structure and obtain more information than they would in the GGM. The AGM provides a powerful framework to analyze the security of succinct protocols. It has been widely used in the context of SNARKs [16,33,17,19,18].

2.4 Useful Lemmas

We rely on the following lemmas.

Lemma 1 ([4, Lemma 2.3],[31, Lemma 2.5]). *Fix integers $\ell, p \in \mathbb{Z}$ and any two vectors $\mathbf{u}\in[\pm p/2]^{128}$ and $\mathbf{w}\in[\pm p/2]^{\ell}$. Let a distribution $\mathcal{D}$ with support $\{-1,0,1\}$ such that $\mathcal{D}[0]=1/2$ and $\mathcal{D}[\pm 1]=1/4$. Then, we have*

$$\Pr_{\mathbf{R}}\left[\left\|\mathbf{u}+\mathbf{R}\cdot\mathbf{w} \bmod p\right\|_\infty < \frac{1}{2}\cdot\left\|\mathbf{w}\right\|_\infty\right] < 2^{-128}$$

over the random choice of $\mathbf{R}\hookleftarrow\mathcal{D}^{128\times\ell}$.

Our proof of knowledge-soundness (and simulation-extractability) relies on Lemma 1 with $\mathbf{u}=0$, in which case a proof was already given in [4, Lemma 2.3].

For the correctness and zero-knowledge properties of the scheme, we need the product $\mathbf{R}\cdot\mathbf{w}$ to be small. When $\mathbf{R}$ is sampled from a continuous normal distribution, a tight bound can be proven on its infinity norm (see [20] and references therein). When $\mathbf{R}$ is sampled from a discrete distribution over $\{-1,0,1\}$, [20, Appendix D] gives a justification as to why one can heuristically assume that a similarly tight bound also holds.

Lemma 2 (heuristic, [20, Corollary 3.2]). *Let a distribution $\mathcal{D}$ with support $\{-1, 0, 1\}$ such that $\mathcal{D}[0] = 1/2$ and $\mathcal{D}[\pm 1] = 1/4$. Under the heuristic of using $\mathcal{D}$ instead of a continuous normal distribution $\frac{1}{\sqrt{2}}\mathcal{N}$,[6] for any vector $\mathbf{w} \in \mathbb{Z}^\ell$, we have $\Pr_{\mathbf{r} \hookleftarrow \mathcal{D}^\ell}[|\langle \mathbf{w}, \mathbf{r} \rangle| > 9.75 \cdot \|\mathbf{w}\|] < 2^{-141}$, where $\langle \mathbf{w}, \mathbf{r} \rangle$ is computed over $\mathbb{Z}$.*

The heuristic of Lemma 2 can be avoided using the bound $|\langle \mathbf{w}, \mathbf{r} \rangle| \leq \sqrt{\ell} \cdot \|\mathbf{w}\|$ given by the Cauchy-Schwarz inequality, which holds with probability 1. However, this bound is pessimistic and does not exploit the distribution of $\mathbf{r}$. One can obtain a proven bound using Hoeffding's inequality which says that, if $(X_i)_{i \in [\ell]}$ are random independent real variables satisfying $\Pr[a_i \leq X_i \leq b_i] = 1$ for real $(a_i)_{i \in [\ell]}$, $(b_i)_{i \in [\ell]}$ such that $a_i < b_i$ for all $i \in [\ell]$, then $S_\ell \triangleq \sum_{i=1}^{\ell} X_i$ satisfies

$$\Pr\left[|S_\ell - \mathbb{E}[S_\ell]| \geq t\right] \leq 2 \cdot \exp\left(-\frac{2t^2}{\sum_{i=1}^{\ell}(a_i - b_i)^2}\right)$$

By the same argument as in [1, Lemma 16], we can view each inner product $\langle \mathbf{r}, \mathbf{w} \rangle$ as a sum of random variables $X_i \in \{-w_i, 0, w_i\}$ such that $\mathbb{E}[X_i] = 0$. By applying Hoeffding with $t = \|\mathbf{w}\| \sqrt{2\lambda / \log(\exp(1))}$, we obtain

$$\Pr_{\mathbf{r} \hookleftarrow \mathcal{D}^\ell}\left[\, |\langle \mathbf{w}, \mathbf{r} \rangle| > \|\mathbf{w}\| \cdot \sqrt{2\lambda / \log(\exp(1))} \,\right] \leq 2^{-\lambda+1}. \tag{7}$$

In the heuristic inequality, we can thus replace the 9.75 factor by 13.32 and obtain a provable bound. By a union bound over the independent rows of $\mathbf{R}$, the probability that $\|\mathbf{R} \cdot \mathbf{w}\|_\infty > 13.32 \cdot \|\mathbf{w}\|$ is at most 2^{-120} for $\lambda = 128$.

2.5 Short Proofs of Binarity

For convenience, this section extends the syntax of Section 2.2 with an algorithm Com that inputs a vector $\boldsymbol{x}$ over a domain D and outputs a commitment C. This commitment is incorporated in the relation $\mathcal{R}_{\mathsf{pp}}$ defined by $\mathsf{CRS\text{-}Gen}$.

In [27], Libert described a short NIZK argument of knowledge showing that a committed vector is binary using two group elements. The proven relation is

$$\mathcal{R}_{\mathsf{pp}} = \left\{ (\mathbf{x}, \mathbf{w}) = \left(\hat{C} = \hat{g}^\gamma \cdot \prod_{i=1}^{n} \hat{g}_i^{x_i} \in \hat{\mathbb{G}}, \quad (\gamma, (x_1, \ldots, x_n)) \in \mathbb{Z}_p \times \{0,1\}^n\right) \right\}$$

where pp denotes the CRS containing the commitment key $(\hat{g}, \{\hat{g}_i\}_{i=1}^n)$ and the description of groups $(\mathbb{G}, \hat{\mathbb{G}}, \mathbb{G}_T)$.

The construction of [27] is recalled hereunder and builds on the functional commitment of [28]. A non-interactive argument showing that a committed vector is binary is then obtained by showing that $\sum_{i=1}^{n} y_i \cdot x_i \cdot (x_i - 1) = 0$, for a random vector $\boldsymbol{y} \in \mathbb{Z}_p^n$ obtained by hashing $\hat{C}$. To do this, the committer generates an auxiliary commitment $C_y = g^{\gamma_y + \sum_{i=1}^{n} y_i \cdot x_i}$ to the Hadamard product $\boldsymbol{y} \circ \boldsymbol{x}$ and generates a short proof π_{eq} that satisfies (8) and shows that C_y is really

[6] $\mathcal{N}$ denotes the continuous normal distribution with mean 0 and variance 1.

a commitment to $\boldsymbol{y} \circ \boldsymbol{x}$. In a second step, the prover shows that $\langle \boldsymbol{y} \circ \boldsymbol{x} - \boldsymbol{y}, \boldsymbol{x} \rangle = 0$. This is done using the inner-product-argument functionality of the commitment [28] and generating a short π_y satisfying (10). Then, π_{eq} and π_y can be aggregated into one group element π. In [27], it was shown that π is computable using only $O(n)$ exponentiation and $O(n \cdot \log n)$ field operations.

CRS-Gen(λ, n)**:** On input of a security parameter λ and the maximal dimension $n \in \mathsf{poly}(\lambda)$ of committed vectors, do the following:

1. Choose asymmetric bilinear groups $(\mathbb{G}, \hat{\mathbb{G}}, \mathbb{G}_T)$ of prime order $p > 2^{l(\lambda)}$, for some function $l : \mathbb{N} \to \mathbb{N}$, and $g \xleftarrow{R} \mathbb{G}$, $\hat{g} \xleftarrow{R} \hat{\mathbb{G}}$.
2. Pick $\alpha \xleftarrow{R} \mathbb{Z}_p$. Compute $g_1, \ldots, g_n, g_{n+2}, \ldots, g_{2n} \in \mathbb{G}$ and $\hat{g}_1, \ldots, \hat{g}_n \in \hat{\mathbb{G}}$, where $g_i = g^{(\alpha^i)}$ for each $i \in [2n] \setminus \{n+1\}$ and $\hat{g}_i = \hat{g}^{(\alpha^i)}$ for each $i \in [n]$.
3. Choose hash functions $H, H_t : \{0,1\}^* \to \mathbb{Z}_p^n$ and $H_{\text{agg}} : \{0,1\}^* \to \mathbb{Z}_p^2$.

The public parameters are

$$\mathsf{pp} = \Big((\mathbb{G}, \hat{\mathbb{G}}, \mathbb{G}_T), g, \hat{g}, \{g_i\}_{i \in [2n] \setminus \{n+1\}}, \{\hat{g}_i\}_{i \in [n]}, \mathbf{H} = \{H, H_t, H_{\text{agg}}\} \Big) .$$

Com$_{\mathsf{pp}}(\boldsymbol{x})$**:** To commit to $\boldsymbol{x} = (x_1, \ldots, x_n) \in \mathbb{Z}_p^n$, choose $\gamma \xleftarrow{R} \mathbb{Z}_p$ and compute $\hat{C} = \hat{g}^{\gamma} \cdot \prod_{j=1}^{n} \hat{g}_j^{x_j}$. Return $\hat{C} \in \hat{\mathbb{G}}$ and the opening information $\mathsf{aux} = \gamma \in \mathbb{Z}_p$.

Prove$_{\mathsf{pp}}\big(\hat{C}, (\boldsymbol{x}, \mathsf{aux})\big)$**:** given a commitment $\hat{C}$ and witnesses $\big(\boldsymbol{x}; \mathsf{aux}\big)$ consisting of a vector $\boldsymbol{x} = (x_1, \ldots, x_n) \in \mathbb{Z}_p^n$ and randomness $\mathsf{aux} = \gamma \in \mathbb{Z}_p$, return $\perp$ if $(x_1, \ldots, x_n) \notin \{0,1\}^n$. Otherwise, do the following:

1. Compute $\boldsymbol{y} = (y_1, \ldots, y_n) = H(\hat{C}) \in \mathbb{Z}_p^n$. Choose $\gamma_y \xleftarrow{R} \mathbb{Z}_p$ and compute $C_y = g^{\gamma_y} \cdot \prod_{j=1}^{n} g_{n+1-j}^{y_j \cdot x_j}$. Compute $\boldsymbol{t} = (t_1, \ldots, t_n) = H_t(\boldsymbol{y}, \hat{C}, C_y) \in \mathbb{Z}_p^n$.
2. Generate a proof

$$\pi_{eq} = \frac{\prod_{i=1}^{n} \left(g_{n+1-i}^{\gamma} \cdot \prod_{j \in [n] \setminus \{i\}} g_{n+1-i+j}^{x_j} \right)^{t_i \cdot y_i}}{\prod_{i=1}^{n} \left(g_i^{\gamma_y} \cdot \prod_{j \in [n] \setminus \{i\}} g_{n+1-j+i}^{y_j \cdot x_j} \right)^{-t_i}}$$

which satisfies

$$\frac{e(\prod_{i=1}^{n} g_{n+1-i}^{t_i \cdot y_i}, \hat{C})}{e(C_y, \prod_{i=1}^{n} \hat{g}_i^{t_i})} = e(\pi_{eq}, \hat{g}) , \tag{8}$$

and argues that C_y commits to $(y_n \cdot x_n, \ldots, y_1 \cdot x_1) \in \mathbb{Z}_p^n$.
3. Compute a proof

$$\pi_y = g^{\gamma \cdot \gamma_y} \cdot \prod_{j=1}^{n} g_{n+1-j}^{\gamma \cdot y_j \cdot (x_j - 1)} \cdot \prod_{i=1}^{n} \big(g_i^{\gamma_y} \cdot \prod_{j \in [n] \setminus \{i\}} g_{n+1-j+i}^{y_j \cdot (x_j - 1)} \big)^{x_i} \tag{9}$$

showing that $\sum_{i=1}^{n} y_i \cdot x_i \cdot (x_i - 1) = 0$ and satisfying

$$e\big(C_y \cdot \prod_{j=1}^{n} g_{n+1-j}^{-y_j}, \hat{C}\big) = e(\pi_y, \hat{g}) \tag{10}$$

4. Compute $(\delta_{eq}, \delta_y) = H_{\text{agg}}(\hat{C}, C_y) \in \mathbb{Z}_p^2$ and then $\pi = \pi_{eq}^{\delta_{eq}} \cdot \pi_y^{\delta_y}$.

Output the final proof $\boldsymbol{\pi} := \big(C_y, \pi\big) \in \mathbb{G}^2$.

$\mathsf{Verify}_{\mathsf{pp}}\big(\hat{C}, \boldsymbol{\pi}\big)$: Given $\hat{C} \in \hat{\mathbb{G}}$ and a purported proof $\boldsymbol{\pi} = \big(C_y, \pi\big) \in \mathbb{G}^2$,

1. Compute $\boldsymbol{y} = H(\hat{C}) \in \mathbb{Z}_p^n$, $(\delta_{eq}, \delta_y) = H_{\text{agg}}(\hat{C}, C_y) \in \mathbb{Z}_p^2$ and $\boldsymbol{t} = H_t(\boldsymbol{y}, \hat{C}, C_y) \in \mathbb{Z}_p^n$.
2. Return 1 if the following equations is satisfied and 0 otherwise:

$$\frac{e\big(C_y^{\delta_y} \cdot \prod_{i=1}^n g_{n+1-i}^{(\delta_{eq} \cdot t_i - \delta_y) \cdot y_i}, \hat{C}\big)}{e\big(C_y, \prod_{i=1}^n \hat{g}_i^{\delta_{eq} \cdot t_i}\big)} = e(\pi, \hat{g}). \tag{11}$$

Correctness follows from the observation that equation (11) is obtained by aggregating (8)-(10). A detailed proof of correctness is given in [27].

Das *et al.* [13] proposed a different constant-size proof of binarity which additionally provides constant verification time. We use the above construction because, as in [27], it makes it easier to aggregate π with other proof components π showing that other committed vectors satisfy other inner product relations.

3 New Constructions: The Slow-Verifier Variant

We consider proofs of validity in an encryption scheme proposed by Joye [24], which was designed to be used as a component of the TFHE [10] homomorphic encryption scheme. The scheme of [24] can be seen as a variant of the LPR cryptosystem [32] where the second ciphertext component computes an inner product over $\mathbb{Z}_q$ instead of a multiplication over R_q. For polynomials $r, s \in R$, we denote by $\bar{r}, \bar{s}$ the polynomials obtained by reversing the coefficients of r and s respectively. The public key consists of $a \in R_q$ and $b = a \cdot \bar{s} + e$, for a binary secret key $s \in R/(2R)$ and where $e \in R$ is a noise.

For a plaintext $m \in \mathbb{Z}_t$, a random $r \in R/(2R)$, and noise terms $e_1 \in R$ and $e_2 \in \mathbb{Z}$, ciphertexts are of the form

$$(c_1, c_2) = \big(a \cdot \bar{r} + e_1, \langle \phi(b),\ \phi(r) \rangle + e_2 + \Delta \cdot m\big) \in R_q \times \mathbb{Z}_q \tag{12}$$

where $\Delta = \lceil q/t \rfloor$ (for a plaintext modulus t which is usually a power of 2) and $\phi(b) = (b_0, \ldots, b_{d-1}) \in \mathbb{Z}_q^d$ contains the coefficients of $b(x) = b_0 + b_1 X + \cdots + b_{n-1} X^{d-1} \in R_q$. Note that $c_2 \in \mathbb{Z}_q$ can be seen as the extraction of the last slot from the second component $b \cdot \bar{r} + \Delta \cdot m + \mathsf{noise}$ of an LPR ciphertext since the degree-$(d-1)$ coefficient of the polynomial product $b \cdot \bar{r} \in R_q$ is $\langle \phi(b), \phi(r) \rangle$.

In order to encrypt k message components at once in a ciphertext using the same r, the scheme exploits the property that the coefficient vector of the ring element $b \cdot \bar{r} \in R_q$ can be obtained as a matrix-vector product $\mathsf{rot}(b)\phi(\bar{r}) \in \mathbb{Z}_q^d$ for an anti-circulant matrix. If we denote by $\phi_i(b)$ the i-th row of the matrix $\mathsf{rot}(b) \in \mathbb{Z}_q^{d \times d}$, the degree-$i$ coefficient of $b \cdot \bar{r} \in R_q$ is equal to $\langle \phi_{i+1}(b), \phi(\bar{r}) \rangle$. Therefore, a packed variant proceeds by encrypting $(m_1, \ldots, m_k) \in \mathbb{Z}_t^k$ as

$$c_1 = a \cdot \bar{r} + e_1 \in R_q$$

$$c_{2,i} = \langle \phi_{d+1-i}(b),\ \phi(\bar{r}) \rangle + e_{2,i} + \Delta \cdot m_i \in \mathbb{Z}_q \qquad \forall i \in [k] \tag{13}$$

for independent noise terms $\{e_{2,i}\}_{i=1}^{k}$. A detailed description of the scheme is given in Supplementary Material A. The statement showing the validity of a ciphertext becomes of the form

$$\underbrace{\left[\begin{array}{cccccc} \mathsf{rot}(a) & & -q \cdot \mathbf{I}_d & & \mathbf{I}_d & \\ \phi_d(b)^\top & \Delta & & -q & & 1 \\ & \ddots & & \ddots & & \ddots \\ \phi_{d-k+1}(b)^\top & & \Delta & & -q & 1 \end{array}\right]}_{\triangleq\ \bar{\mathbf{A}}} \cdot \underbrace{\left[\begin{array}{c} \phi(\bar{r}) \\ m_1 \\ \vdots \\ m_k \\ \phi(r_1) \\ r_{2,1} \\ \vdots \\ r_{2,k} \\ \phi(e_1) \\ e_{2,1} \\ \vdots \\ e_{2,k} \end{array}\right]}_{\triangleq\ \bar{\boldsymbol{w}}} = \left[\begin{array}{c} \phi(c_1) \\ c_{2,1} \\ \vdots \\ c_{2,k} \end{array}\right]. \tag{14}$$

When it comes to proving the validity of a packed ciphertext, the NIZK construction of [27] decomposes the entire witness vector into bits, which leads to a binary witness of length $D = d + k \cdot \log t + (d+k)(1 + \log B + \log d)$. When the ring LWE dimension is $d = 2048$ and a plaintext modulus $t = 2^5$, a ciphertext modulus $q \approx 2^{64}$, and a noise of magnitude $B = 2^{16}$, we have $D > 60000$, which results in a fairly large CRS and a large number of exponentiations to compute. As explained in the introduction, the most expensive part of it is to prove the smallness of noise terms since decomposing $d+k$ integers of absolute value $\leq B$ introduces $(d+k)(1+\log B) \approx 35360$ components in the vectors we need to commit to (which is roughly 50% of their dimension n).

To reduce this overhead, we do not directly decompose the entire witness $\bar{\boldsymbol{w}}$ of (14) into bits. Specifically, we refrain from decomposing $\phi(r_1), \{r_{2,i}\}_{i=1}^{k}$ and $\phi(e_1), \{e_{2,i}\}_{i=1}^{k}$ but rather prove their smallness by first projecting them to a smaller dimension vector using a trick introduced in [4]. Then, we prove the smallness of the projection by breaking it into bits, which is much cheaper.

This leads us to write the statement as a linear relation of the form (15), where only the plaintext is immediately decomposed into bits

$$\underbrace{\left[\begin{array}{cccccc} \mathsf{rot}(a) & & -q \cdot \mathbf{I}_d & & \boldsymbol{I}_d & \\ \phi_d(b)^\top & \Delta \cdot \boldsymbol{g}_{\log t}^\top & & -q & & 1 \\ & \ddots & & & \ddots & & \ddots \\ \phi_{d-k+1}(b)^\top & & \Delta \cdot \boldsymbol{g}_{\log t}^\top & & -q & 1 \end{array}\right]}_{\triangleq\ \tilde{\mathbf{A}} = [\tilde{\mathbf{A}}_0 | -q \cdot \mathbf{I}_{d+k} | \mathbf{I}_{d+k}]}$$

$$\cdot \begin{bmatrix} \phi(\bar{r}) \\ \boldsymbol{m}_1 \\ \vdots \\ \boldsymbol{m}_k \\ \phi(r_1) \\ r_{2,1} \\ \vdots \\ r_{2,k} \\ \phi(e_1) \\ e_{2,1}) \\ \vdots \\ e_{2,k} \end{bmatrix} = \underbrace{\begin{bmatrix} \phi(c_1) \\ c_{2,1} \\ \vdots \\ c_{2,k} \end{bmatrix}}_{\triangleq\ \tilde{\mathbf{c}}}, \quad (15)$$

and where $\tilde{\mathbf{A}}_0 \in \mathbb{Z}^{(d+k)\times(d+k\cdot\log t)}$. For each $i \in [k]$, $\boldsymbol{m}_i = \boldsymbol{g}_{\log t}^{-1}(m_i) \in \{0,1\}^{\log t}$.

We still need a binary decomposition of the plaintext because we want to be able to prove statements about its individual bits. In the context of TFHE, it is useful to prove that the most significant bit of each plaintext slot (which is used as a padding bit) is zero. To do this, we can simply replace the gadget vector $\boldsymbol{g}_{\log t} = (1, 2, \ldots, 2^{\log t - 2}, -2^{\log t - 1})^\top$ in the left-hand-side member of (15) by its prefix $\bar{\boldsymbol{g}}_{\log t} = (1, 2, 4, \ldots, 2^{\log t - 2})^\top \in \mathbb{Z}^{1\times(\log t - 1)}$ obtained by discarding the last component. We thus modify (15) and prove the relation

$$\underbrace{\begin{bmatrix} \mathsf{rot}(a) & & -q\cdot\mathbf{I}_d & & \boldsymbol{I}_d & \\ \phi_d(b)^\top & \Delta\cdot\bar{\boldsymbol{g}}_{\log t}^\top & & -q & 1 & \\ & \ddots & & \ddots & & \ddots \\ \phi_{d-k+1}(b)^\top & & \Delta\cdot\bar{\boldsymbol{g}}_{\log t}^\top & & -q & 1 \end{bmatrix}}_{\triangleq\ \tilde{\mathbf{A}}=[\tilde{\mathbf{A}}_0|-q\cdot\mathbf{I}_{d+k}|\mathbf{I}_{d+k}]}$$

$$\cdot \begin{bmatrix} \phi(\bar{r}) \\ \boldsymbol{m}_1 \\ \vdots \\ \boldsymbol{m}_k \\ \phi(r_1) \\ r_{2,1} \\ \vdots \\ r_{2,k} \\ \phi(e_1) \\ e_{2,1}) \\ \vdots \\ e_{2,k} \end{bmatrix} = \underbrace{\begin{bmatrix} \phi(c_1) \\ c_{2,1} \\ \vdots \\ c_{2,k} \end{bmatrix}}_{\triangleq\ \tilde{\mathbf{c}}}, \quad (16)$$

where $\tilde{\mathbf{A}}_0 \in \mathbb{Z}^{(d+k)\times(d+k\cdot(\log t-1))}$ and, for each $i \in [k]$, $\boldsymbol{m}_i = \bar{\boldsymbol{g}}_{\log t}^{-1}(m_i) \in \{0,1\}^{\log t - 1}$ stands for the $(\log t - 1)$-bit representation of m_i.

If we prove the smallness of the noise using the infinity norm, the honest prover generates a membership argument of the language

$$\mathcal{L}_{\mathsf{zk}} = \Big\{ \big(\mathsf{lpp} = (q, d, k, B_\infty, t), (a, b), (c_1, \{c_{2,i}\}_{i=1}^k)\big) \in \mathcal{P} \times (R_q^*)^2 \times (R_q \times \mathbb{Z}_q^k) \mid \\ c_1 = a \cdot \bar{r} + e_1, \quad \forall i \in [k] : c_{2,i} = \langle \phi_{d+1-i}(b),\ \phi(\bar{r}) \rangle + e_{2,i} + \Delta \cdot m_i, \\ (m_1, \ldots, m_k) \in \mathbb{Z}_t^k,\ \phi(\bar{r}) \in \{0,1\}^d,\ \|(\phi(e_1) \mid (e_{2,1}, \ldots, e_{2,k}))\|_\infty \leq B_\infty \Big\}$$

for which zero-knowledge is guaranteed. In terms of soundness, the verifier will only be convinced that that ciphertext belongs to the language $\mathcal{L}_{\mathsf{snd}} \supset \mathcal{L}_{\mathsf{zk}}$ defined as

$$\mathcal{L}_{\mathsf{snd}} = \Big\{ \big(\mathsf{lpp} = (q, d, k, B_\infty, t), (a, b), (c_1, \{c_{2,i}\}_{i=1}^k)\big) \in \mathcal{P} \times (R_q^*)^2 \times (R_q \times \mathbb{Z}_q^k) \mid \\ c_1 = a \cdot \bar{r} + e_1, \quad \forall i \in [k] : c_{2,i} = \langle \phi_{d+1-i}(b),\ \phi(\bar{r}) \rangle + e_{2,i} + \Delta \cdot m_i, \\ (m_1, \ldots, m_k) \in \mathbb{Z}_t^k,\ \phi(\bar{r}) \in \{0,1\}^d, \\ \|(\phi(e_1) \mid (e_{2,1}, \ldots, e_{2,k}))\|_\infty \leq \gamma_{\mathsf{slack}} \cdot B_\infty \Big\} \tag{17}$$

for a slack factor[7] $\gamma_{\mathsf{slack}} = \sqrt{d+k}$ induced by proving a bound B on the Euclidean norm of $\boldsymbol{e} \triangleq (\phi(e_1) \mid (e_{2,1}, \ldots, e_{2,k}))$. The factor $\gamma_{\mathsf{slack}} = \sqrt{d+k}$ comes from the fact that, if $\|\boldsymbol{e}\|_\infty \leq B_\infty$, the smallest Euclidean norm bound that can be proven without affecting zero-knowledge is $B = B_\infty\sqrt{d+k}$. Since we always have $\|\boldsymbol{e}\|_\infty \leq \|\boldsymbol{e}\|_2$, the verifier is convinced that $\|\boldsymbol{e}\|_\infty \leq B_\infty\sqrt{d+k}$ if the prover begins with a noise $\boldsymbol{e}$ such that $\|\boldsymbol{e}\|_\infty \leq B_\infty$.

If the infinity norm is replaced by the ℓ_2-norm (namely, if we want to prove $\|\boldsymbol{e}\|_2 \leq B$), the slack disappears and we have $\gamma_{\mathsf{slack}} = 1$.

3.1 Description

In the description below, we assume that the CRS generation algorithm $\mathsf{CRS\text{-}Gen}$ takes as input upper bounds $\bar{q}$, $\bar{t}$ for the ciphertext/plaintext moduli, upper bounds $\bar{d}$ and $\bar{k}$ for the dimension and the number of message slots, and an upper bound $\bar{B}_\infty$ for the infinity norm of the noise in the encryption algorithm.

In the construction, the prover defines a vector $\bar{\boldsymbol{e}} = (\boldsymbol{e} \mid (v_1, \ldots, v_4)) \in \mathbb{Z}^{d+k+4}$, where $\boldsymbol{e}$ contains the noise terms and $(v_1, \ldots, v_4)$ is the 4-square decomposition of $B^2 - \|\boldsymbol{e}\|^2$. At step 3, the prover commits to $\bar{\boldsymbol{e}}$ in both source groups $\mathbb{G}$ and $\hat{\mathbb{G}}$ by computing

$$\hat{C}_e = \hat{g}^{\hat{\gamma}_e + \sum_{i=1}^{d+k+4} \bar{\boldsymbol{e}}[i] \cdot \alpha^i}, \qquad C_e = g^{\gamma_e + \sum_{i=1}^{d+k+4} \bar{\boldsymbol{e}}[i] \cdot \alpha^{n+1-i}}.$$

It also computes a commitment $C_{\tilde{r}}$ to the quotients $(\phi(r_1), \{r_{2,i}\}_{i=1}^k)$ of the Euclidean division by q when the linear relation (16) is written over $\mathbb{Z}$. At steps 4-6, the prover commits to a projection $\mathbf{w}_R = \mathbf{R} \cdot \left(\bar{\boldsymbol{e}}^\top \mid \tilde{\boldsymbol{r}}^\top\right)^\top \bmod p$, where

[7] For parameters $d = 2048$ and $k \leq d$, we have $\gamma_{\mathsf{slack}} \leq 2^6$.

the flat ternary matrix $\mathbf{R}$ is obtained by hashing $(C_e, \hat{C}_e, C_{\bar{r}})$, and proves that the committed $\mathbf{w}_R$ has the correct form. Again, this is done by proving that a linear combination of inner product relations using the inner-product functional commitment of [28] (we are actually using a linear-map vector commitment [26] for this purpose). Then, the smallness of $\mathbf{w}_R$ is proven by considering its binary decomposition $\mathbf{w}_{R,\mathsf{bin}}$ at step 7 and proving that $\mathbf{w}_{R,\mathsf{bin}}$ is really the binary decomposition of $\mathbf{w}_R$ at step 8.

In order to minimize the proof size, step 9 generates a single proof of binarity for a longer binary vector $\mathbf{w}_{\mathsf{bin}}$ obtained as the concatenation of $\mathbf{w}_{R,\mathsf{bin}}$, the plaintext bits $(\boldsymbol{m}_1, \ldots, \boldsymbol{m}_k)$, and the encryption randomness $\phi(\bar{r})$. Steps 10-11 then prove the linear relation (16) by compressing the matrix-vector product into an inner product relation proven as as in [28].

At step 13, the prover provides evidence that the equality $B^2 = \langle \bar{\boldsymbol{e}}, \bar{\boldsymbol{e}} \rangle \bmod p$ holds. To do this, at step 12, the prover first generates a proof π_e that, for each $i \in [d+k+4]$, $\hat{C}_e$ commits to a vector whose i-th component is equal to the $(n+1-i)$-th component of the vector committed in C_e. We also need π_e to prove that the last $n-(d+k+4)$ components of the vector committed in $\hat{C}_e$ are zeroes. In the left-hand-side member of (24), it is the reason why we use n random aggregation coefficients $\{\omega_i\}_{i=1}^n$ in the numerator, but only $d+k+4$ of them in the denominator. If the verifier is convinced that $\hat{C}_e$ is really of the form $\hat{C}_e = \hat{g}^{\hat{\gamma}_e} \cdot \prod_{i=1}^{d+k+4} \hat{g}_i^{e_i}$ and that C_e is of the form $C_e = g^{\gamma_e} \cdot \prod_{i=1}^{n} g_{n+1-i}^{e_i}$, it knows that $e(C_e, \hat{C}_e)$ is of the form

$$e(C_e, \hat{C}_e) = e(g_1, \hat{g}_n)^{\sum_{i=1}^{d+k+4} e_i^2} \cdot e(g^{p_\ell(\alpha)}, \hat{g}) \tag{18}$$

for some polynomial $p_\ell[X]$ of degree $\leq 2n$ that has no monomial of degree $n+1$. The reason is that the pairing in the left-hand-side member of (18) computes a product of polynomials in the exponent and this product has a coefficient of degree $n+1$ which is

$$\langle (e_1, \ldots, e_{d+k+4}, \mathbf{0}^{n-(d+k+4)}), (e_1, \ldots, e_n) \rangle.$$

Hence, the verifier is convinced if the prover can compute $\pi_\ell = g^{p_\ell(\alpha)}$ such that

$$e(C_e, \hat{C}_e) = e(g_1, \hat{g}_n)^{B^2} \cdot e(\pi_\ell, \hat{g}).$$

The precise description goes as follows.

CRS-Gen$(\lambda, (\bar{q}, \bar{d}, \bar{k}, \bar{B}_\infty, \bar{t}))$**:** Given a security parameter λ, a maximal dimension $\bar{d} \in \mathsf{poly}(\lambda)$, integers $\bar{q}, \bar{t}, \bar{k} \in \mathsf{poly}(\lambda)$, and an infinity norm bound $\bar{B}_\infty$,[8] define $\bar{B} = \bar{B}_\infty \cdot \sqrt{\bar{d}+\bar{k}} \in \mathsf{poly}(\lambda)$, and $\bar{B}_{\mathsf{GHL}} = 9.75 \cdot \sqrt{\bar{B}^2 + \frac{(\bar{d}+2)^2(\bar{d}+\bar{k})}{4}}$, $\bar{m} = \lceil 1 + \log \bar{B}_{\mathsf{GHL}} \rceil$. Then, do the following.

[8] The upper bounds $(\bar{q}, \bar{d}, \bar{k}, \bar{B}_\infty, \bar{t})$ define a set $\mathcal{P}$ of acceptable parameters in the syntax of Section 2.2.

1. Generate asymmetric pairing-friendly groups $(\mathbb{G}, \hat{\mathbb{G}}, \mathbb{G}_T)$ of prime order

$$p > \max\left(2^{l(\lambda)}, \bar{q} \cdot (\bar{d} + \bar{k} \cdot (\log \bar{t} - 1) + 2^{\bar{m}+1}) + 2\bar{B}, (\bar{d} + \bar{k} + 4) \cdot 2^{2\bar{m}}\right),$$

for some polynomial $l : \mathbb{N} \to \mathbb{N}$. Let[9]

$$n \geq \max(\bar{d} + \bar{k} \cdot (\log \bar{t} - 1) + 128 \cdot \bar{m}, 2(\bar{d} + \bar{k}) + 4).$$

2. Pick a random $\alpha \xleftarrow{R} \mathbb{Z}_p$ and compute $g_1, \ldots, g_n, g_{n+2}, \ldots, g_{2n} \in \mathbb{G}$ as well as $\hat{g}_1, \ldots, \hat{g}_n \in \hat{\mathbb{G}}$, where $g_i = g^{(\alpha^i)}$ for each $i \in [2n] \setminus \{n+1\}$ and $\hat{g}_i = \hat{g}^{(\alpha^i)}$ for each $i \in [n]$.
3. Choose a hash function $H_R : \{0,1\}^* \to \mathcal{D}^{128 \times (2(\bar{d}+\bar{k})+4)}$, where $\mathcal{D}$ is the distribution that outputs 0 with probability 1/2 and 1 or -1 with probability 1/4 each.[10]
4. Choose hash functions $H, H_t, H_\omega : \{0,1\}^* \to \mathbb{Z}_p$, $H_{\mathrm{agg}} : \{0,1\}^* \to \mathbb{Z}_p^7$ and $H_{\mathsf{lmap}} : \{0,1\}^* \to \mathbb{Z}_p$, $H_\phi, H_\xi : \{0,1\}^* \to \mathbb{Z}_p$ that will be modeled as random oracles.

Output the CRS

$$\mathsf{pp} = \left((\mathbb{G}, \hat{\mathbb{G}}, \mathbb{G}_T), g, \hat{g}, \{g_i\}_{i \in [2n] \setminus \{n+1\}}, \{\hat{g}_i\}_{i \in [n]}, \mathbf{H}\right),$$

where $\mathbf{H} = \{H, H_R, H_\phi, H_\xi, H_t, H_\omega, H_{\mathrm{agg}}, H_{\mathsf{lmap}}\}$.

$\mathsf{Prove}_{\mathsf{pp}}(\mathbf{x}, \mathbf{w})$: Given a statement $\mathbf{x} = (q, d, k, B_\infty, t, \tilde{\mathbf{A}}, \mathbf{c})$ consisting of dimensions $d \leq \bar{d}$, moduli $q \leq \bar{q}$ and $t \leq \bar{t}$, a ciphertext $\mathbf{c} = (c_1, (c_{2,i})_{i=1}^k) \in R_q \times \mathbb{Z}_q^k$, a norm bound $B_\infty \leq \bar{B}_\infty$ for the vector $\boldsymbol{e} \triangleq (\phi(e_1), e_{2,1}, \ldots, e_{2,k}) \in \mathbb{Z}^{d+k}$, a matrix $\tilde{\mathbf{A}}$ of the form (16),[11] as well as a witness $\mathbf{w} = (\bar{r}, e_1, (m_i, e_{2,i})_{i=1}^k) \in R^2 \times (\mathbb{Z}_t \times \mathbb{Z})^k$ such that (13) holds with $\bar{r} \in \{0,1\}^d$, return $\perp$ if $\|\boldsymbol{e}\|_\infty > B_\infty$ or if there exists $i \in [k]$ such that $\mathsf{msb}(m_i) \neq 0$. Otherwise, define the ℓ_2-norm bound $B = B_\infty \sqrt{d+k}$ and do the following.

1. Define $B_{\mathsf{GHL}} = 9.75 \cdot \sqrt{B^2 + \frac{(d+2)^2(d+k)}{4}}$ and $m = \lceil 1 + \log B_{\mathsf{GHL}} \rceil \leq \bar{m}$.
2. Compute the quotients $r_1 \in R$ and $(r_{2,1}, \ldots, r_{2,k}) \in \mathbb{Z}^k$ such that $\|r_1\|_\infty, \|r_{2,i}\|_\infty \leq \frac{d+1}{2}$ for each $i \in [k]$ and satisfying (14). Encode the noise terms $(e_1, \{e_{2,i}\}_{i=1}^k)$ as $\boldsymbol{e} = \left(\phi(e_1)^\top \mid (e_{2,i}^\top)_{i=1}^k\right)^\top \in \mathbb{Z}^{d+k}$ and the quotients $(r_1, \{r_{2,i}\}_{i=1}^k)$ as $\tilde{\boldsymbol{r}} = \left(\phi(r_1)^\top \mid (r_{2,i}^\top)_{i=1}^k\right)^\top \in \mathbb{Z}^{d+k}$. Then, encode $(r, m_1, \ldots, m_k)$ as a vector

$$\tilde{\boldsymbol{w}} = \left[\ \phi(\bar{r}) \mid \boldsymbol{m}_1 \mid \ldots \mid \boldsymbol{m}_k\ \right]^\top \in \{0,1\}^D,$$

[9] For $\bar{d} = 2048$, $\bar{k} = 320$ with an infinity norm bound $\bar{B}_\infty = 2^{17}$, we have $\bar{B} \leq 2^{22.60}$, $\bar{B}_{\mathsf{GHL}} \leq 2^{25.89}$ and $\bar{m} = 27$. If $\bar{t} = 2^5$, we only need $n \geq 6784$.

[10] Such a hash function modeled as a random oracle can easily be obtained as the difference between two random oracles with output space $\{0,1\}^{128 \times (2(\bar{d}+\bar{k})+4)}$.

[11] We include the matrix $\tilde{\mathbf{A}}$ in the statement since, besides the public key (a, b), it also encodes the padding bit positions which are proven to be zeroes in the plaintext.

of dimension $D = d + k \cdot (\log t - 1)$ using bit decompositions $\boldsymbol{m}_i = \bar{\boldsymbol{g}}_{\log t}^{-1}(m_i) \in \{0,1\}^{\log t - 1}$ for each $i \in [k]$. Note that we have the equality $\tilde{\mathbf{c}} = \tilde{\mathbf{A}}_0 \cdot \tilde{\boldsymbol{w}} - q \cdot \tilde{\boldsymbol{r}} + \boldsymbol{e}$ over $\mathbb{Z}$ in (16).

3. Choose $\hat{\gamma}_e, \gamma_e \xleftarrow{R} \mathbb{Z}_p$ and commit to $\bar{\boldsymbol{e}} = (\boldsymbol{e} \mid (v_1, \ldots, v_4)) \in \mathbb{Z}^{d+k+4}$ by computing

$$\hat{C}_e = \hat{g}^{\hat{\gamma}_e} \cdot \prod_{i=1}^{d+k+4} \hat{g}_i^{\bar{\boldsymbol{e}}[i]}, \qquad C_e = g^{\gamma_e} \cdot \prod_{i=1}^{d+k+4} g_{n+1-i}^{\bar{\boldsymbol{e}}[i]},$$

where $v_1, \ldots, v_4 \in [0, B]$ are integers such that $\sum_{i=1}^{4} v_i^2 = B^2 - \|\boldsymbol{e}\|^2$. Also, choose $\gamma_r \xleftarrow{R} \mathbb{Z}_p$ and compute

$$C_{\tilde{r}} = g^{\gamma_r} \cdot \prod_{i=1}^{d+k} g_i^{\tilde{\boldsymbol{r}}[i]}.$$

4. Compute $\bar{\mathbf{R}} = H_R(\mathbf{x}, \hat{C}_e, C_e, C_{\tilde{r}}) \in \{-1, 0, 1\}^{128 \times (2(\bar{d}+\bar{k})+4)}$ of which the columns are distributed as per $\mathcal{D}^{128}$. Let $\mathbf{R} \in \{-1, 0, 1\}^{128 \times (2(d+k)+4)}$ be the submatrix consisting of the first $d + k + 4$ columns of $\bar{\mathbf{R}}$. Compute

$$\mathbf{w}_R = \mathbf{R} \cdot \begin{bmatrix} \bar{\boldsymbol{e}} \\ \tilde{\boldsymbol{r}} \end{bmatrix} \in \mathbb{Z}^{128}$$

which compresses $\bar{\boldsymbol{e}} \in \mathbb{Z}^{d+k+4}$ and $\tilde{\boldsymbol{r}} \in \mathbb{Z}^{d+k}$.[12] If $\|\mathbf{w}_R\|_\infty > B_{\mathsf{GHL}}$, abort and return $\perp$.

5. Choose $\gamma_R \xleftarrow{R} \mathbb{Z}_p$ and compute

$$C_R = g^{\gamma_R} \cdot \prod_{i=1}^{128} g_i^{\mathbf{w}_R[i]}$$

Then, compute $\phi = H_\phi(\mathbf{x}, \mathbf{R}, \hat{C}_e, C_e, C_R, C_{\tilde{r}}) \in \mathbb{Z}_p$ and define $\boldsymbol{\phi} = (\phi_1, \ldots, \phi_{128}) \in \mathbb{Z}_p^{128}$ where $\phi_i = \phi^{i-1}$ for each $i \in [128]$..

6. Generate a proof π_r satisfying

$$e\Big(\prod_{j=1}^{d+k+4} g_{n+1-j}^{\sum_{i=1}^{128} \phi_i \cdot \mathbf{R}[i,j]}, \hat{C}_e\Big) \cdot e\Big(C_{\tilde{r}}, \prod_{j=1}^{d+k} \hat{g}_{n+1-j}^{\sum_{i=1}^{128} \phi_i \cdot \mathbf{R}[i,d+k+4+j]}\Big) \cdot e(C_R, \prod_{i=1}^{128} \hat{g}_{n+1-i}^{\phi_i})^{-1} = e(\pi_r, \hat{g}) \,, \quad (19)$$

which argues that C_R commits to $\mathbf{R} \cdot \left(\bar{\boldsymbol{e}}^\top \mid \tilde{\boldsymbol{r}}^\top\right)^\top \in \mathbb{Z}_p^{128}$.

[12] Due to the smallness of $\bar{\boldsymbol{e}}$ and $\mathbf{R}$, the product $\mathbf{w}_R = \mathbf{R} \cdot \bar{\boldsymbol{e}} \in \mathbb{Z}^{128}$ is also $\mathbf{R} \cdot \bar{\boldsymbol{e}} \bmod p$ when seen as a vector over $\{-(p-1)/2, \ldots, (p-1)/2\}^{128}$.

7. Let $\mathbf{w}_{R,\mathsf{bin}} \in \{0,1\}^{128\cdot m}$ the binary decomposition of $\mathbf{w}_R$ such that $\mathbf{w}_{R,\mathsf{bin}}[(i-1)\cdot m+1, i\cdot m] = \boldsymbol{g}_m^{-1}(\mathbf{w}_R[i])$ for each $i \in [128]$. Define the vector $\mathbf{w}_{\mathsf{bin}} = (\tilde{\boldsymbol{w}} \mid \mathbf{w}_{R,\mathsf{bin}} \mid \mathbf{0}^{n-(D+128\cdot m)}) \in \{0,1\}^n$. Commit to it by choosing $\gamma_{\mathsf{bin}} \xleftarrow{R} \mathbb{Z}_p$ and computing

$$\hat{C}_{\mathsf{bin}} = \hat{g}^{\gamma_{\mathsf{bin}}} \cdot \prod_{i=1}^{D+128\cdot m} \hat{g}_i^{\mathbf{w}_{\mathsf{bin}}[i]}$$

8. Generate a proof that, for each $i \in [128]$, $\mathbf{w}_{R,\mathsf{bin}}[(i-1)m+1, im]$ is the binary decomposition of $\mathbf{w}_R[i] \in [-B_{\mathsf{GHL}}, B_{\mathsf{GHL}})$. To do this, define the matrix $\mathbf{H} = \mathbf{I}_{128} \otimes \boldsymbol{g}_m^\top \in \mathbb{Z}_p^{128\times 128\cdot m}$ and prove that $\mathbf{w}_R = \mathbf{H}\cdot\mathbf{w}_{R,\mathsf{bin}}$. To do that, compute $\xi = H_\xi(\mathbf{x}, \hat{C}_e, C_e, \phi, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}) \in \mathbb{Z}_p$ and define $\boldsymbol{\xi} = (\xi_1, \ldots, \xi_{128})$ with $\xi_i = \xi^{i-1}$ for each $i \in [128]$. Then, compute π_{dec} such that

$$\frac{e\big(\prod_{j=1}^{128m} g_{n+1-(D+j)}^{\sum_{i=1}^{128}\boldsymbol{\xi}[i]\cdot\mathbf{H}[i,j]}, \hat{C}_{\mathsf{bin}}\big)}{e\big(C_R, \prod_{i=1}^{128}\hat{g}_{n+1-i}^{\boldsymbol{\xi}[i]}\big)} = e(\pi_{\mathsf{dec}}, \hat{g}). \tag{20}$$

9. Generate a proof that $\hat{C}_{\mathsf{bin}}$ commits to a binary $\mathbf{w}_{\mathsf{bin}}$. Namely,

   a. Compute $y = H\big(\mathbf{x}, \phi, \xi, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}\big) \in \mathbb{Z}_p$. Define the vector $\boldsymbol{y} = (y_1, \ldots, y_n) \in \mathbb{Z}_p^n$ where $y_i = y^{i-1}$ for each $i \in [D + 128\cdot m]$ and $y_i = 0$ for each $i \in [D+128\cdot m+1, n]$. Next, choose $\gamma_y \xleftarrow{R} \mathbb{Z}_p$ and compute

$$C_y = g^{\gamma_y} \cdot \prod_{j=1}^{D+128\cdot m} g_{n+1-j}^{y_j\cdot\mathbf{w}_{\mathsf{bin}}[j]}$$

   Compute

$$t = H_t(\mathbf{x}, y, \phi, \xi, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y) \in \mathbb{Z}_p$$

   and define $\boldsymbol{t} = (t_1, \ldots, t_n) \in \mathbb{Z}_p^n$ where $t_i = t^{i-1}$ for each $i \in [n]$.

   b. Compute $\pi_{eq} \in \mathbb{G}$ such that

$$\frac{e(\prod_{i=1}^{D+128\cdot m} g_{n+1-i}^{t_i\cdot y_i}, \hat{C}_{\mathsf{bin}})}{e(C_y, \prod_{i=1}^{n}\hat{g}_i^{t_i})} = e(\pi_{eq}, \hat{g}). \tag{21}$$

   which shows that C_y commits to the (reversed) product $\boldsymbol{y} \circ \boldsymbol{w} \in \mathbb{Z}_p^n$.

   c. Compute π_y such that

$$e\big(C_y \cdot \prod_{j=1}^{D+128\cdot m} g_{n+1-j}^{-y_j}, \hat{C}_{\mathsf{bin}}\big) = e(\pi_y, \hat{g}) \tag{22}$$

   which shows that $\sum_{i=1}^{n} y_i \cdot \mathbf{w}_{\mathsf{bin}}[i]\cdot(\mathbf{w}_{\mathsf{bin}}[i]-1) = 0$.

10. Compute $\theta = H_{\mathsf{lmap}}\big(\mathbf{x}, y, t, \phi, \xi, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y\big) \in \mathbb{Z}_p$ and define $\boldsymbol{\theta} = (\theta_1, \ldots, \theta_{\bar{d}+\bar{k}}) \in \mathbb{Z}_p^{\bar{d}+\bar{k}}$ where $\theta_i = \theta^{i-1}$ for each $i \in [\bar{d}+\bar{k}]$. Define $\tilde{\mathbf{A}} = [\tilde{\mathbf{A}}_0 \mid -q \cdot \mathbf{I}_{d+k} \mid \mathbf{I}_{d+k}] \in \mathbb{Z}^{(d+k)\times(D+2(d+k))}$ and $\tilde{\mathbf{c}} \in \mathbb{Z}^{d+k}$ as in (16) with $\tilde{\mathbf{A}}_0 \in \mathbb{Z}^{(d+k)\times D}$. Let $\boldsymbol{\theta}_0 \in \mathbb{Z}_p^{d+k}$ the first $d+k$ entries of $\boldsymbol{\theta}$.
11. Let $t_\theta = \boldsymbol{\theta}_0^\top \cdot \tilde{\mathbf{c}} \bmod p$ and $\boldsymbol{a}_\theta^\top = \boldsymbol{\theta}_0^\top \cdot \tilde{\mathbf{A}}_0 \in \mathbb{Z}_p^{1\times D} \bmod p$. Generate a proof $\pi_\theta \in \mathbb{G}$ satisfying

$$e\left(\prod_{i=1}^{D} g_{n+1-i}^{\boldsymbol{a}_\theta[i]},\ \hat{C}_{\mathsf{bin}}\right) \cdot e\left(C_{\tilde{r}}, \prod_{i=1}^{d+k} \hat{g}_{n+1-i}^{-q\cdot\boldsymbol{\theta}_0[i]}\right) \cdot e\left(\prod_{i=1}^{d+k} g_{n+1-i}^{\boldsymbol{\theta}_0[i]}, \hat{C}_e\right) \cdot e(g_1, \hat{g}_n)^{-t_\theta} = e(\pi_\theta, \hat{g}) \tag{23}$$

    which shows that $\boldsymbol{\theta}_0^\top \tilde{\mathbf{A}}_0 \cdot \tilde{\boldsymbol{w}} - q \cdot \boldsymbol{\theta}_0^\top \cdot \tilde{\boldsymbol{r}} + \boldsymbol{\theta}_0^\top \cdot \boldsymbol{e} = \boldsymbol{\theta}_0^\top \cdot \tilde{\boldsymbol{c}} \bmod p$.
12. Compute $\omega = H_\omega\big(\mathbf{x}, y, t, \phi, \xi, \theta, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y\big) \in \mathbb{Z}_p$. Define $\boldsymbol{\omega} = (\omega_1, \ldots, \omega_n) \in \mathbb{Z}_p^n$, where $\omega_i = \omega^{i-1}$ for each $i \in [n]$. Then, generate a proof π_e satisfying

$$\frac{e\big(\prod_{i=1}^{n} g_{n+1-i}^{\omega_i}, \hat{C}_e\big)}{e\big(C_e, \prod_{i=1}^{d+k+4} \hat{g}_i^{\omega_i}\big)} = e(\pi_e, \hat{g}) \tag{24}$$

    which shows that C_e is consistent with $\hat{C}_e$.
13. Generate a proof that $\sum_{i=1}^{d+k+4} \bar{\boldsymbol{e}}[i]^2 = B^2$ by computing π_ℓ such that

$$e(C_e, \hat{C}_e) \cdot e(g_1, \hat{g}_n)^{-B^2} = e(\pi_\ell, \hat{g}) \tag{25}$$

14. Compute $\boldsymbol{\delta} = (\delta_r, \delta_{\mathsf{dec}}, \delta_{eq}, \delta_y, \delta_\theta, \delta_e, \delta_\ell) \in \mathbb{Z}_p^7$ as

$$\boldsymbol{\delta} = H_{\mathrm{agg}}\big(\mathbf{x}, y, t, \phi, \xi, \theta, \omega, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y\big)$$

    and use it to compute an aggregated proof

$$\pi = \pi_r^{\delta_r} \cdot \pi_{\mathsf{dec}}^{\delta_{\mathsf{dec}}} \cdot \pi_y^{\delta_y} \cdot \pi_{eq}^{\delta_{eq}} \cdot \pi_\theta^{\delta_\theta} \cdot \pi_e^{\delta_e} \cdot \pi_\ell^{\delta_\ell}.$$

Output the final proof $\boldsymbol{\pi} = \big(\hat{C}_e, C_e, C_{\tilde{r}}, C_R, \hat{C}_{\mathsf{bin}}, C_y, \pi\big)$.

$\mathsf{Verify}_{\mathsf{pp}}\big(\mathbf{x}, \boldsymbol{\pi}\big)$: Given a statement $\mathbf{x} = (q, d, k, B_\infty, t, \tilde{\mathbf{A}}, \mathbf{c})$ and a candidate $\boldsymbol{\pi}$, return 0 if $\boldsymbol{\pi}$ does not parse properly. Otherwise, set $B = B_\infty \cdot \sqrt{d+k}$.

1. Compute $\bar{\mathbf{R}} = H_R(\mathbf{x}, \hat{C}_e, C_e, C_{\tilde{r}}) \in \{-1, 0, 1\}^{128\times(2(\bar{d}+\bar{k})+4)}$ whose columns are distributed as per $\mathcal{D}^{128}$. Let $\mathbf{R} \in \{-1, 0, 1\}^{128\times(2(d+k)+4)}$ the submatrix consisting of the first $2(d+k)$ columns of $\bar{\mathbf{R}}$.
2. Set $\phi = H_\phi(\mathbf{x}, \mathbf{R}, \hat{C}_e, C_e, C_R, C_{\tilde{r}})$, $\xi = H_\xi(\mathbf{x}, \hat{C}_e, C_e, \phi, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}})$, and $y = H\big(\mathbf{x}, \phi, \xi, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}\big) \in \mathbb{Z}_p$. Then, compute

$$t = H_t(\mathbf{x}, y, \phi, \xi, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y) \in \mathbb{Z}_p,$$

$$\theta = H_{\mathsf{lmap}}\big(\mathbf{x}, y, t, \phi, \xi, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y\big) \in \mathbb{Z}_p$$
$$\omega = H_{\omega}\big(\mathbf{x}, y, t, \phi, \xi, \theta, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y\big) \in \mathbb{Z}_p,$$

and $\boldsymbol{\delta} = (\delta_r, \delta_{\mathsf{dec}}, \delta_{eq}, \delta_y, \delta_\theta, \delta_e, \delta_\ell) \in \mathbb{Z}_p^7$ as

$$\boldsymbol{\delta} = H_{\mathrm{agg}}\big(\mathbf{x}, y, t, \phi, \xi, \theta, \omega, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y\big).$$

Let $\boldsymbol{\phi} = (\phi_1, \ldots, \phi_{128}), \boldsymbol{\xi} = (\xi_1, \ldots, \xi_{128}) \in \mathbb{Z}_p^{128}$ such that $\phi_i = \phi^{i-1}$ and $\xi_i = \xi^{i-1}$ for each $i \in [128]$. Define $\boldsymbol{\theta} = (\theta_1, \ldots, \theta_{\bar{d}+\bar{k}}) \in \mathbb{Z}_p^{\bar{d}+\bar{k}}$, where $\theta_i = \theta^{i-1}$ for each $i \in [\bar{d}+\bar{k}]$. Define $\boldsymbol{\omega} = (\omega_1, \ldots, \omega_n) \in \mathbb{Z}_p^n$, $\boldsymbol{t} = (t_1, \ldots, t_n)$ with $t_i = t^{i-1}$ and $\omega_i = \omega^{i-1}$ for each $i \in [n]$. Define $\boldsymbol{y} = (y_1, \ldots, y_n)$ with $y_i = y^{i-1}$ for each $i \in [D + 128 \cdot m]$ and $y_i = 0$ for each $i \in [D + 128 \cdot m + 1, n]$.

3. Define $\tilde{\mathbf{A}} = [\tilde{\mathbf{A}}_0 \mid -q \cdot \mathbf{I}_{d+k} \mid \mathbf{I}_{d+k}] \in \mathbb{Z}^{(d+k)\times(D+2(d+k))}$ and $\tilde{\mathbf{c}} \in \mathbb{Z}^{d+k}$ as in (16). Let $\tilde{\mathbf{A}}_0 \in \mathbb{Z}^{(d+k)\times D}$. Let the vector $\boldsymbol{\theta}_0 \in \mathbb{Z}_p^{d+k}$ containing the first $d+k$ entries of $\boldsymbol{\theta}$. Let $\bar{\boldsymbol{a}}_\theta^\top = \boldsymbol{\theta}_0^\top \cdot [\tilde{\mathbf{A}}_0 \mid \mathbf{0}^{(d+k)\times 128m}] \in \mathbb{Z}_p^{1\times(D+128\cdot m)} \bmod p$ and $t_\theta = \boldsymbol{\theta}_0^\top \cdot \tilde{\mathbf{c}} \bmod p$, where $\tilde{\mathbf{c}} \in \mathbb{Z}^{d+k}$ is the vector defined in (16).
4. Let $\bar{\boldsymbol{\theta}}_0 = (\boldsymbol{\theta}_0 \mid \mathbf{0}^{n-(d+k)}) \in \mathbb{Z}_p^n$ and define the matrices $\mathbf{R}' \in \mathbb{Z}_p^{128\times n}$ and $\mathbf{H}_\xi \in \mathbb{Z}_p^{128\times(D+128\cdot m)}$ such that

$$\forall i \in [128] : \mathbf{R}'[i,j] = \begin{cases} \mathbf{R}[i,j] & \text{if } \ j \in [1, 2(d+k)+4] \\ 0 & \text{if } \ j \in [2(d+k)+5, n]. \end{cases}$$

and

$$\forall i \in [128] : \mathbf{H}_\xi[i,j] = \begin{cases} \boldsymbol{\xi}[i] \cdot \mathbf{H}[i, j-D] & \text{if } \ j \in [D+1, D+128\cdot m] \\ 0 & \text{if } \ j \in [1, D]. \end{cases}$$

Return 1 if the following equality holds and 0 otherwise:

$$\begin{aligned} & e\left(C_y^{\delta_y} \cdot \prod_{j=1}^{D+128\cdot m} g_{n+1-j}^{\delta_\theta \cdot \bar{\boldsymbol{a}}_\theta[j] - \delta_y \cdot y_j + \delta_{eq} \cdot t_j \cdot y_j + \delta_{\mathsf{dec}} \cdot \sum_{i=1}^{128} \mathbf{H}_\xi[i,j]},\ \hat{C}_{\mathsf{bin}}\right) \\ & \quad \cdot e\left(C_e^{\delta_\ell} \cdot \prod_{j=1}^{n} g_{n+1-j}^{\delta_\theta \cdot \bar{\boldsymbol{\theta}}_0[j] + \sum_{i=1}^{128} \delta_r \cdot \phi_i \cdot \mathbf{R}'[i,j] + \delta_e \cdot \omega_j}, \hat{C}_e\right) \\ & \quad \cdot e\Big(C_{\tilde{r}}, \prod_{j=1}^{d+k} \hat{g}_{n+1-j}^{-\delta_\theta \cdot q \cdot \boldsymbol{\theta}_0[j] + \delta_r \cdot \sum_{i=1}^{128} \phi_i \cdot \mathbf{R}[i, d+k+4+j]}\Big) \\ & \quad \cdot e\big(C_R, \prod_{j=1}^{128} \hat{g}_{n+1-j}^{\delta_r \cdot \phi_j + \delta_{\mathsf{dec}} \cdot \boldsymbol{\xi}[j]}\big)^{-1} \cdot e\big(C_e^{\delta_e}, \prod_{i=1}^{d+k+4} \hat{g}_i^{\omega_i}\big)^{-1} \\ & \quad \cdot e\big(C_y^{\delta_{eq}}, \prod_{j=1}^{n} \hat{g}_j^{t_j}\big)^{-1} \cdot e(g_1, \hat{g}_n)^{-\delta_\theta \cdot t_\theta - \delta_\ell \cdot B^2} = e(\pi, \hat{g}) \end{aligned} \tag{26}$$

Correctness. The final verification equation (26) is obtained by taking a linear combination of individual equations (19), (20), (21), (22), (23), (24), and (25) with the coefficients $(\delta_r, \delta_{\mathsf{dec}}, \delta_{eq}, \delta_y, \delta_\theta, \delta_e, \delta_\ell) \in \mathbb{Z}_p^7$ and aggregating the π-components of these individual equations.

At step 4, the prover fails with negligible probability under the heuristic described in Lemma 2. More precisely, Lemma 2 implies $\|\mathbf{w}_R\|_\infty \leq B_{\mathsf{GHL}}$ with probability $\geq 1 - 2^{-134}$ since $\|\tilde{\boldsymbol{r}}\|_\infty \leq (d+1)/2$ implies

$$\|(\bar{\boldsymbol{e}} \mid \tilde{\boldsymbol{r}})\|_2 \leq \sqrt{B^2 + \tfrac{(d+1)^2(d+k)}{4}} < B_{\mathsf{GHL}}.$$

Efficiency. Instead of computing all proof components separately (which would cost $O(n^2)$ exponentiations), the prover can directly compute the aggregated $\pi \in \mathbb{G}$ using only $2n$ exponentiations and $O(n \log n)$ field operations. Indeed, in the left-hand-side member of (26), each pairing computes a product of polynomials in the exponent for the variable α. By identifying both members, we see that the prover can first compute the coefficients of the polynomial

$$\begin{aligned}
P_\pi[X] = \Big(\delta_y \cdot \gamma_y + \sum_{j=1}^{D+128m} \Big(\delta_y \cdot y_j \cdot (\mathbf{w}_{\mathsf{bin}}[j] - 1) + \delta_\theta \cdot \bar{\boldsymbol{a}}_\theta[j] + \delta_{eq} \cdot t_j \cdot y_j \\
+ \delta_{\mathsf{dec}} \cdot \sum_{i=1}^{128} \mathbf{H}_\xi[i,j]\Big) \cdot X^{n+1-j}\Big) \cdot \Big(\gamma_{\mathsf{bin}} + \sum_{j=1}^{D+128m} \mathbf{w}_{\mathsf{bin}}[j] \cdot X^j\Big) \\
+\Big(\delta_\ell \cdot \big(\gamma_e + \sum_{j=1}^{d+k+4} \bar{\boldsymbol{e}}[j] \cdot X^{n+1-j}\big) + \sum_{j=1}^{n} \big(\delta_\theta \cdot \bar{\boldsymbol{\theta}}_0[j] + \sum_{i=1}^{128} \delta_r \cdot \phi_i \cdot \mathbf{R}'[i,j] + \delta_e \cdot \omega_j\big) \cdot X^{n+1-j}\Big) \\
\cdot \big(\hat{\gamma}_e + \sum_{j=1}^{d+k+4} \bar{\boldsymbol{e}}[j] \cdot X^j\big) \\
+\Big(\gamma_r + \sum_{j=1}^{d+k} \tilde{\boldsymbol{r}}[j] \cdot X^j\Big) \cdot \Big(\sum_{j=1}^{d+k} \Big(-\delta_\theta \cdot q \cdot \boldsymbol{\theta}_0[j] + \sum_{i=1}^{128} \delta_r \cdot \phi_i \cdot \mathbf{R}[i, d+k+4+j]\Big) \cdot X^{n+1-j}\Big) \\
- \Big(\gamma_R + \sum_{j=1}^{128} \mathbf{w}_R[j] \cdot X^j\Big) \cdot \Big(\sum_{j=1}^{128} (\delta_r \cdot \phi_j + \delta_{\mathsf{dec}} \cdot \boldsymbol{\xi}[j]) \cdot X^{n+1-j}\Big) \\
- \delta_e \cdot \big(\gamma_e + \sum_{j=1}^{d+k+4} \bar{\boldsymbol{e}}[j] \cdot X^{n+1-j}\big) \cdot \Big(\sum_{j=1}^{d+k+4} \omega_j \cdot X^j\Big) \\
-\delta_{eq} \cdot \Big(\gamma_y + \sum_{j=1}^{D+128m} y_j \cdot \mathbf{w}_{\mathsf{bin}}[j] \cdot X^{n+1-j}\Big) \cdot \Big(\sum_{j=1}^{n} t_j \cdot X^j\Big) - (\delta_\theta \cdot t_\theta + \delta_\ell \cdot B^2) \cdot X^{n+1},
\end{aligned}$$

From the coefficients of $P_\pi[X] = \sum_{i=1}^{n+D+128m} \nu_i \cdot X^i$, the prover can then compute

$$\pi = g^{P_\pi(\alpha)} = g^{\nu_0} \cdot \prod_{i=1, i \neq n+1}^{n+D+128m} g_i^{\nu_i}, \tag{27}$$

which is computable without g_{n+1}. The second equality of (27) relies on the fact that, in $P_\pi[X]$, the coefficient of the degree-$(n+1)$ term is zero.

Indeed, this coefficient is given by

$$\begin{aligned}
\nu_{n+1} = &\sum_{j=1}^{D+128m} \Big(\delta_y \cdot y_j \cdot \underbrace{(\mathbf{w}_{\mathsf{bin}}[j]-1)\cdot \mathbf{w}_{\mathsf{bin}}[j]}_{=0} + \delta_\theta \cdot \bar{\boldsymbol{a}}_\theta[j]\cdot \mathbf{w}_{\mathsf{bin}}[j] + \delta_{\mathsf{dec}} \cdot \sum_{i=1}^{128} \underbrace{\mathbf{H}_\xi[i,j]\cdot \mathbf{w}_{\mathsf{bin}}[j]}_{=\boldsymbol{\xi}[i]\cdot \mathbf{H}[i,j-D]\cdot \mathbf{w}_{\mathsf{bin}}[j]}\Big) \\
&+ \delta_\ell \cdot \sum_{j=1}^{d+k+4} \bar{\boldsymbol{e}}[j]^2 + \delta_\theta \cdot \sum_{j=1}^{d+k} \bar{\boldsymbol{\theta}}_0[j]\cdot \bar{\boldsymbol{e}}[j] \\
&+ \delta_r \cdot \underbrace{\Big(\sum_{j=1}^{d+k+4}\sum_{i=1}^{128} \phi_i \cdot \mathbf{R}'[i,j]\cdot \bar{\boldsymbol{e}}[j] + \sum_{j=1}^{d+k}\sum_{i=1}^{128} \phi_i \cdot \mathbf{R}[i,d+k+4+j]\cdot \tilde{\boldsymbol{r}}[j] - \sum_{i=1}^{128} \mathbf{w}_R[i]\cdot \phi_i\Big)}_{=0} \\
&- \delta_{\mathsf{dec}} \cdot \sum_{i=1}^{128} \boldsymbol{\xi}[i]\cdot \mathbf{w}_R[i] - (\delta_\theta \cdot t_\theta + \delta_\ell \cdot B^2) \\
&= \delta_\theta \cdot \Big(\sum_{j=1}^{D+128m} \bar{\boldsymbol{a}}_\theta[j]\cdot \mathbf{w}_{\mathsf{bin}}[j] + \sum_{j=1}^{d+k} \bar{\boldsymbol{\theta}}_0[j]\cdot \bar{\boldsymbol{e}}[j] - q\cdot \sum_{j=1}^{d+k} \boldsymbol{\theta}_0[j]\cdot \tilde{\boldsymbol{r}}[j]\Big) \\
&\quad + \delta_\ell \cdot \sum_{j=1}^{d+k+4} \bar{\boldsymbol{e}}[j]^2 - (\delta_\theta \cdot t_\theta + \delta_\ell \cdot B^2) \\
&= \delta_\theta \cdot \Big(\sum_{j=1}^{D+128m} \boldsymbol{\theta}_0^\top \cdot [\tilde{\mathbf{A}}_0 \mid \mathbf{0}^{(d+k)\times 128m}]\cdot \mathbf{w}_{\mathsf{bin}} + \sum_{j=1}^{d+k} \boldsymbol{\theta}_0[j]\cdot \bar{\boldsymbol{e}}[j] - q\cdot \sum_{j=1}^{d+k} \boldsymbol{\theta}_0[j]\cdot \tilde{\boldsymbol{r}}[j]\Big) \\
&\quad + \delta_\ell \cdot \underbrace{\sum_{j=1}^{d+k+4} \bar{\boldsymbol{e}}[j]^2}_{=B^2} - (\delta_\theta \cdot t_\theta + \delta_\ell \cdot B^2) = 0,
\end{aligned}$$

where the second equality holds because $\mathbf{w}_R[i] = \sum_{j=1}^{D+128m} \mathbf{H}[i, j-D]\cdot \mathbf{w}_{\mathsf{bin}}[j]$ for each $i \in [128]$.

Efficiency-wise, the prover only needs to compute $(\hat{C}_e, C_e, C_{\tilde{r}}, C_R, \hat{C}_{\mathsf{bin}}, C_y)$ and the polynomial $P_\pi[X]$ before computing π. The prover's work is dominated by $3n + 2(d+k)$ exponentiations in $\mathbb{G}$ and $d+k+5$ exponentiations in $\hat{\mathbb{G}}$ (or 25088 exponentiations in $\mathbb{G}$ and 2373 exponentiations in $\hat{\mathbb{G}}$).

The verifier has to compute a product of 7 pairings and $n+2(d+k)$ (roughly 11520) exponentiations in $\hat{\mathbb{G}}$ and $2n$ (about 13568) exponentiations in $\mathbb{G}$.

Each proof only costs 2 elements of $\hat{\mathbb{G}}$ and 5 elements of $\mathbb{G}$. On a BLS12-446 curve, the proof fits in 502 bytes and the CRS size is only 1800KB if $n = 6784$.

3.2 Security

The zero-knowledge property holds in the statistical sense (rather than perfectly) and it relies on the heuristic mentioned in Lemma 2 (we will show how to remove

it in Section 3.3). The reason is that, under this heuristic, the real prover fails with probability $\leq 2^{-134}$ (while Hoeffding only guarantees that it fails with probability $\leq 2^{-60}$) while the simulator never fails.

We first note that the verifier learns that

$$\|\mathbf{w}_R\|_\infty = \|\mathbf{R} \cdot (\bar{\boldsymbol{e}}^\top \mid \tilde{\boldsymbol{r}}^\top)^\top \bmod p\|_\infty \leq 2^{m-1} = 2^{\lceil \log B_{\mathsf{GHL}} \rceil}. \tag{28}$$

However, by Lemma 2, (28) is implied w.h.p. by the inequality

$$\|(\bar{\boldsymbol{e}}^\top \mid \tilde{\boldsymbol{r}}^\top)^\top\|_2 \leq \sqrt{B^2 + \tfrac{(d+2)^2(d+k)}{4}}, \tag{29}$$

which is itself implied by the statement. Indeed, with the notations of (16), the statement is that

$$\tilde{\mathbf{c}} = \tilde{\mathbf{A}}_0 \cdot \tilde{\boldsymbol{w}} - q \cdot \tilde{\boldsymbol{r}} + \boldsymbol{e} \quad (\text{over } \mathbb{Z})$$

for some $\tilde{\boldsymbol{r}} \in \mathbb{Z}^{d+k}$, $\tilde{\boldsymbol{w}} \in \{0,1\}^D$ and $\boldsymbol{e} \in \mathbb{Z}^{d+k}$ such that $\|\boldsymbol{e}\|_2 \leq B$. Then, we have $\|\tilde{\mathbf{A}}_0 \cdot \tilde{\boldsymbol{w}} + \boldsymbol{e} - \tilde{\boldsymbol{c}}\|_\infty \leq (d+3) \cdot q/2$ since $\|\tilde{\mathbf{A}}_0 \cdot \tilde{\boldsymbol{w}}\|_\infty \leq (d+1) \cdot q/2$ due to the form of the matrix $\tilde{\mathbf{A}}_0$ in (16). Since all components of $\tilde{\mathbf{A}}_0 \cdot \tilde{\boldsymbol{w}} + \boldsymbol{e} - \tilde{\boldsymbol{c}}$ are divisible by q and d is even, it must be that $\|\tilde{\mathbf{A}}_0 \cdot \tilde{\boldsymbol{w}} + \boldsymbol{e} - \tilde{\boldsymbol{c}}\|_\infty \leq (d+2) \cdot q/2$. In turn, this implies $\|\tilde{\boldsymbol{r}}\|_\infty \leq (d+2)/2$ and thus (29) since the statement also says that $\|(\boldsymbol{e} \mid (v_1, \ldots, v_4))\|_2 = B$. Then, by Lemma 2, (29) implies

$$\|\mathbf{R} \cdot (\bar{\boldsymbol{e}}^\top \mid \tilde{\boldsymbol{r}}^\top)^\top \bmod p\|_\infty \leq B_{\mathsf{GHL}} = 9.75 \cdot \sqrt{B^2 + \tfrac{(d+2)^2(d+k)}{4}},$$

with overwhelming probability and this finally implies (28). Hence, the information (28) is revealed by the statement itself and not by the proof.

Theorem 1. *Under the same heuristic as in Lemma 2, the scheme is statistically zero-knowledge.*

Proof. We show that the secret trapdoor $\alpha \in \mathbb{Z}_p$ of the CRS makes it possible to simulate a proof using only the statement $\mathbf{x}$ and the norm bound B. To do this, the simulator first computes $(\hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_y, C_{\tilde{r}})$ as commitments to the zero vector $\mathbf{0}^n$. Namely, it picks $\hat{\gamma}_e, \gamma_e, \gamma_R, \gamma_r, \gamma_{\mathsf{bin}}, \gamma_y \xleftarrow{R} \mathbb{Z}_p$ and computes

$$\begin{aligned} \hat{C}_e &= \hat{g}^{\hat{\gamma}_e}, & C_e &= g^{\gamma_e}, & C_R &= g^{\gamma_R}, \\ \hat{C}_{\mathsf{bin}} &= \hat{g}^{\gamma_{\mathsf{bin}}}, & C_y &= g^{\gamma_y}, & C_{\tilde{r}} &= g^{\gamma_r} \end{aligned}$$

Next, it computes all scalars by hashing these commitments (together with statement) as the verifier would at steps 1-3 of the Verify algorithm. Then, it computes

$$P[X] = \gamma_{\mathsf{bin}} \cdot \Big(\gamma_y \cdot \delta_y + \sum_{j=1}^{D+128m} \big(\delta_\theta \cdot \bar{\boldsymbol{a}}_\theta[j] - \delta_y \cdot y_j + \delta_{eq} \cdot t_j \cdot y_j + \delta_{\mathsf{dec}} \cdot \sum_{i=1}^{128} \mathbf{H}_\xi[i,j]\big) \cdot X^{n+1-j}\Big)$$

$$
\begin{aligned}
&+ \hat{\gamma}_e \cdot \Big(\delta_\ell \cdot \gamma_e + \sum_{j=1}^{n} \big(\delta_\theta \cdot \bar{\boldsymbol{\theta}}_0[j] + \sum_{i=1}^{128} \delta_r \cdot \phi_i \cdot \mathbf{R}'[i,j] + \delta_e \cdot \omega_j \big) \cdot X^{n+1-j} \Big) \\
&+ \gamma_r \cdot \sum_{j=1}^{d+k} \Big(- \delta_\theta \cdot q \cdot \boldsymbol{\theta}_0[j] + \delta_r \cdot \sum_{i=1}^{128} \phi_i \cdot \mathbf{R}[i, d+k+4+j] \Big) \cdot X^{n+1-j} \\
&- \gamma_R \cdot \sum_{j=1}^{128} \big(\delta_r \cdot \phi_j + \delta_{\mathsf{dec}} \cdot \boldsymbol{\xi}[j] \big) \cdot X^{n+1-j} \\
&- \delta_e \cdot \gamma_e \cdot \Big(\sum_{i=1}^{d+k+4} \omega^i \cdot X^i \Big) - \delta_{eq} \cdot \gamma_y \cdot \Big(\sum_{i=1}^{n} t_i \cdot X^i \Big) - \delta_\theta \cdot t_\theta - \delta_\ell \cdot B^2,
\end{aligned}
$$

which is implicitly evaluated in the exponent for $X = \alpha$ in the left-hand-side member of (26). The simulator obtains an accepting proof by setting $\pi = g^{P(\alpha)}$.

With the above procedure, the simulator obtains exactly the same proof distribution as the real prover since commitments are uniformly distributed in $\mathbb{G}$ or $\hat{\mathbb{G}}$ and π is uniquely determined by the commitments, the statement $\mathbf{x}$ and the scalars derived from hash functions.

We note that the zero-knowlege property does not rely on random oracles and holds against an unbounded verifier. □

In order to prove simulation-extractability, the proof of Theorem 2 builds a trapdoor-less simulator (as defined in [18]), which only proceeds by programming the random oracles and without using the trapdoor of the CRS. This simulator proceeds as in [27, Theorem 6] by programming the random oracles and the committed vectors at the same time.

Theorem 2. *If* $p > \max\big(q \cdot (d+k \cdot (\log t - 1) + 2^{m+1}) + 2B, (d+k+4) \cdot 2^{2m}\big)$, *the scheme provides simulation-extractability in the AGM+ROM under the* $(2n, n)$*-DLOG assumption.* (The proof is given in Supplementary Material B.)

3.3 Removing the Heuristic

We can avoid relying on the heuristic of Lemma 2 (which may be desirable in some applications where we do not want the zero-knowledge property to depend on any unproven claim) if we accept a small loss of efficiency. The Cauchy-Schwarz inequality gives the upper bound

$$
\|\mathbf{w}_R\|_\infty \leq \bar{B}_{\mathsf{CS}} \triangleq \sqrt{2(\bar{d}+\bar{k})+4} \cdot \sqrt{\bar{B}^2 + \tfrac{(\bar{d}+2)^2(\bar{d}+\bar{k})}{4}} \tag{30}
$$

and we have $\sqrt{2(\bar{d}+\bar{k})+4} \approx 2^{6.1}$ for the parameters we consider. However, We have the bound

$$
\|\mathbf{w}_R\|_\infty \leq \sqrt{2\lambda} \cdot \|(\bar{\boldsymbol{e}}^\top \mid \tilde{\boldsymbol{r}}^\top)^\top\| \leq \bar{B}_{\mathsf{H}} \triangleq 13.32 \cdot \sqrt{\bar{B}^2 + \tfrac{(\bar{d}+2)^2(\bar{d}+\bar{k})}{4}},
$$

which holds with overwhelming probability by (7).

Concretely, we can replace $\bar{B}_{\mathsf{GHL}}$ by $\bar{B}_{\mathsf{H}}$ and set $\bar{m} = \lceil 1+\log \bar{B}_{\mathsf{H}} \rceil$. For parameters $d = 2048$, $k = 320$, $p = 2^5$, $\bar{B}_\infty = 2^{17}$, we obtain $\bar{B} = 2^{22.60}$, $\bar{B}_{\mathsf{H}} = 2^{26.34}$ and we can set $\bar{m} = 28$ and $n \geq \bar{d}+\bar{k}\cdot(\log \bar{t}-1)+128\cdot\bar{m} = 6912$. The conditions of Theorem 2 are still fulfilled since $q \cdot (d + k \cdot (\log t - 1) + 2^{m+1}) + 2B < 2^{93}$ and $(d + k + 4) \cdot 2^{2m} < 2^{68}$, which are both smaller than $p > 2^{256}$.

With this dimension $n = 6912$ of committed vectors, the CRS size amounts to 1506 KB and the proof size remains the same. In comparison with the original NIZK construction of [27], we still reduce the CRS size by a factor 9. The prover computes 25472 exponentiations in $\mathbb{G}$ and 2373 exponentiations in $\hat{\mathbb{G}}$. The verifier computes 11648 (resp. 13824) exponentiations in $\hat{\mathbb{G}}$ (resp. $\mathbb{G}$).

3.4 Proving Other Statements About Plaintexts

Without any change in the CRS, the prover is able to prove other statements than just plaintext knowledge by proceeding in the same way as in [27, Appendix G.7]. In short, we can exploit the fact that each proof contains a commitment $\hat{C}_{\mathsf{bin}}$ to $\mathbf{w}_{\mathsf{bin}} \in \{0,1\}^n$, which contains the plaintext bits in its positions $d+1$ to $d + k(\log t - 1)$. We can exploit these properties to prove that two ciphertexts encrypt the same plaintext or that these plaintexts agree in specific bit positions. Indeed, if $\hat{C}_{\mathsf{bin}}$, $\hat{C}'_{\mathsf{bin}}$ commit to the bits of two plaintexts, one can prove that these plaintexts agree in positions $I \subseteq [n]$ by generating a batch-opening proof to $\mathbf{0}^{|I|}$ for $\hat{C}'_{\mathsf{bin}}/\hat{C}_{\mathsf{bin}}$. Using the technique of [27, Appendix G.7], it also remains possible to prove that a plaintext is smaller than another plaintext when they are interpreted as integers. As in [27, Appendix F], we can similarly prove upper bounds on the Hamming weight of plaintexts.

In SNARKs like Groth16 [22], this would not be possible without generating a new CRS. In universal-CRS SNARKs enabling similarly short proofs (e.g., Plonk [17]), a circuit-dependent pre-processing phase would be necessary for each new statement to be proven/verified.

4 A Variant With Faster Verification

A disadvantage of the scheme in Section 3 is that the verifier has to compute multi-exponentiations with $O(n)$ generators. In Supplementary Material C, we describe a variant where these multi-exponentiations are outsourced to the prover and the verifier only computes $O(n)$ field multiplications in order to evaluate degree-n polynomials.

On behalf of the verifier, the prover computes $\hat{C}_t = \prod_{j=1}^{n} \hat{g}_j^{t_j}$ and

$$C_{h,1} = \prod_{j=1}^{D+128\cdot m} g_{n+1-j}^{\delta_\theta \cdot \bar{\boldsymbol{a}}_\theta[j] - \delta_y \cdot y_j + \delta_{eq} \cdot t_j \cdot y_j + \delta_{\mathsf{dec}} \cdot \sum_{i=1}^{128} \mathbf{H}_\xi[i,j]},$$

$$C_{h,2} = \prod_{j=1}^{n} g_{n+1-j}^{\delta_\theta \cdot \bar{\boldsymbol{\theta}}_0[j] + \sum_{i=1}^{128} \delta_r \cdot \phi_i \cdot \mathbf{R}'[i,j] + \delta_e \cdot \omega_j}$$

which appear in the verification equation (26). Then, we interpret $(C_{h,1}, C_{h,2}, \hat{C}_t)$ as deterministic KZG commitments [25] to polynomials that are known to the verifier. To convince the verifier that $(C_{h,1}, C_{h,2}, \hat{C}_t)$ are commitments to the correct polynomials, the prover evaluates them on a random $z \in \mathbb{Z}_p$ and sends a batched KZG evaluation proof demonstrating correct polynomial evaluations. Since the verification time of KZG evaluation proofs does not depend on the degree n of committed polynomials, the verifier computes only $2(d+k)+128$ exponentiations in $\hat{\mathbb{G}}$, which is significantly smaller than n.

With these changes, each proof is now comprised of 8 elements of $\mathbb{G}$ and 3 elements of $\hat{\mathbb{G}}$. On a BLS12-446 curve, each element of $\mathbb{G}$ (resp. $\hat{\mathbb{G}}$) has a 446-bit (resp. 892-bit) representation and the proof fits in 780 bytes. The prover's workload is dominated by $6n + 2(d+k)$ exponentiations in $\mathbb{G}$ and $n+d+k$ exponentiations in $\hat{\mathbb{G}}$. With $n = 6784$ (using the heuristic bound $\bar{B}_{\mathsf{GHL}}$), $d = 2048$ and $k = 320$, the prover has to compute ≈ 45400 exponentiations in $\mathbb{G}$ (resp. 9150 exponentiations in $\hat{\mathbb{G}}$).

Besides the evaluation of degree-n polynomials (each of which takes $O(n)$ field multiplications using Horner's method), the verifier has to compute two multi-exponentiations of dimension $d+k$, one multi-exponentiation of dimension 128 in $\hat{\mathbb{G}}$, and two pairing-products. Using batch verification techniques, the verifier can only compute a single batched product of 8 pairings.

In comparison, the prover of the fast-verifier variant of [27, Appendix G.6] was computing the equivalent of ≈ 400000 exponentiations in $\mathbb{G}$.

A complete description is available in Supplementary Material C. In Supplementary Material D, we give yet another tradeoff where the verifier only computes 128 exponentiations in $\hat{\mathbb{G}}$.

5 Comparisons and Experimental Results

In Table 1, we summarize the number of pairings and exponentiations in each group for our three constructions. The CRS size is identical in all constructions and consists of $2n$ elements of $\mathbb{G}$ and n elements of $\hat{\mathbb{G}}$, where n is the dimension of committed vectors.

The table omits the cost of evaluating degree-n polynomials (which take $3n$ and $5n$ multiplications in $\mathbb{Z}_p$, respectively) in schemes II and III. Since $n \approx 10000$ for $d = 2048$ when we pack up to $k = 1024$ messages per ciphertext, these polynomial evaluations are very negligible compared to group exponentiations.

The scheme of [27] is not included in Table 1 because its prover complexity would be expressed in terms of a much larger n. However, we provide implementation results showing that our new constructions are drastically faster while only slightly increasing the proof length. In the fast-verifier scheme of [27, Appendix G.6], proofs live in $\hat{\mathbb{G}}^2 \times \mathbb{G}^4$ and we thus roughly double its proof length in schemes II and III.

In Table 2, we report implementation timings when we prove the validity of packed ciphertexts containing up to $k = 1024$ message components, each of

Table 1. Efficiency summary for the proposed NIZK arguments

Schemes	Proof size	Prover cost	Verifier cost
Scheme I (Section 3.1)	$5 \times \lvert\mathbb{G}\rvert + 2 \times \lvert\hat{\mathbb{G}}\rvert$	$3n + 2(d+k)$ $\exp_{\mathbb{G}}$ $d + k$ $\exp_{\hat{\mathbb{G}}}$	$n + 2(d+k)$ $\exp_{\hat{\mathbb{G}}}$ $+2n$ $\exp_{\mathbb{G}}$ + 7P
Scheme II (Supplementary Material C)	$8 \times \lvert\mathbb{G}\rvert + 3 \times \lvert\hat{\mathbb{G}}\rvert$	$6n + 2(d+k)$ $\exp_{\mathbb{G}}$ $n + d + k$ $\exp_{\hat{\mathbb{G}}}$	$2(d+k) + 128$ $\exp_{\hat{\mathbb{G}}}$ + 8P
Scheme III (Supplementary Material D)	$8 \times \lvert\mathbb{G}\rvert + 5 \times \lvert\hat{\mathbb{G}}\rvert$	$6n + 2(d+k)$ $\exp_{\mathbb{G}}$ $n + 3(d+k)$ $\exp_{\hat{\mathbb{G}}}$	128 $\exp_{\hat{\mathbb{G}}}$ + 8P

$\exp_{\mathbb{G}}$ and $\exp_{\hat{\mathbb{G}}}$ denote exponentiations in $\mathbb{G}$ and $\hat{\mathbb{G}}$; P stands for a pairing computation;
n denotes the dimension of committed vectors; d is the dimension of the ring $R = \mathbb{Z}[X]/(X^d+1)$;
k is the number of packed messages per ciphertext.

Table 2. Implementation timings in Rust for $k = 1024$ packed messages and $d = 2048$

Schemes	Proving time$^{\diamond}$ (in ms)	Verification time$^{\diamond}$ (in ms)	Verification time$^{\nabla}$ (in ms)
[27, Appendix G.6]	2300	197	151
Scheme II	459	112	54
Scheme III	536	45	37

$^{\diamond}$ This benchmark was run on an c6i.4xlarge AWS instance with 16 vCPUs.
$^{\nabla}$ The verifier was implemented on an hpc7a.96xlarge AWS instance with 192 vCPUs.

which has 2^5 bits (in each slot, only 4 bits are used to carry data since the TFHE-rs library [41] usually leaves one padding bit to 0). We use different machines for the prover and the verifier to mimic a real application. The prover machine has specifications that resemble those of most modern laptops, whereas the server is assumed to be significantly more powerful. These experiments were conducted for RLWE parameters where $d = 2048$, $q = 2^{64}$, a noise of infinity norm $B_\infty \leq 2^{17}$, and the bound $\bar{B}_{\mathsf{CS}}$ of Section 3.3. This corresponds to vectors of dimension $n = 10112$ (instead of ≈ 95000 in [27]) in our second and third constructions.[13] The corresponding Rust code[14] is based on the Arkworks library [3], to which we added an implementation of the BLS12-446 curve that is not natively available.

As for the four-square integer decomposition, we implemented the Rabin-Shallit algorithm with Cornacchia's half-Euclidean GCD (see, e.g., [30]), with the refinements proposed by [37]. Best practical results for our sizes were obtained

[13] As explained in Supplementary Material C, we can obtain a slightly smaller $n = 9728$ using the bound $\bar{B}_{\mathsf{H}}$ which holds with overwhelming probability.
[14] It is available from https://github.com/zama-ai/tfhe-rs/tree/tfhe-zk-pok-0.7.4/tfhe-zk-pok.

for a sieve mask set to 6. We also observe that the same trick as in Cornacchia's algorithm can be used to compute a GCD in the Hürwitz integers using an aborted half-Gaussian GCD, which slightly improves the last (negligible) part of the decomposition algorithm.

In Table 3, we give comparisons when the prover is executed in a browser (we consider browser executions for the prover only). In blockchain applications, this allows users to prove the validity of their ciphertext without having to install a particular software or programming language.

Table 3. Prover timings in WASM for $k = 1024$

Schemes	Proving time$^{\diamond}$ (in s)
[27, Appendix G.6]	7.6
Scheme II	1.7
Scheme III	2.0

$^{\diamond}$ The prover was implemented on an m6i.4xlarge AWS instance with 16 vCPUs.

Comparison with [7]. As mentioned in the introduction, comparing the above results with Greco [7] is difficult since the proven statements are different and the benchmarks of [7] were obtained on a different machine. We have nevertheless run their code (which is native Rust) on the same machine as ours.

For a dimension $d = 2048$ and a modulus $q = 6943179709095039 \approx 2^{52}$, we obtained proving times between 2.21s and 7.94s (depending on a parameter K that dictates the number of rows in the advice column given to halo2-lib) for a verification time ranging between 4 and 11ms. As shown by Table 4, the best proving times (from $K = 12$ to $K = 16$) give significantly longer proofs (between 2.6 and 27KB) and the shortest proofs (for $K = 18$) take 7.9s to compute.

Table 4. Timings (on a C6i.4xlarge) and Proof sizes for Greco

K	Proof size (bytes)	Proving time (s)	Verification time (ms)
12	27072	2.51864059	11.150076
13	14208	2.212683187	7.7747
14	7776	2.213476644	5.686808
15	4096	2.323916362	4.462565
16	2624	2.960160366	4.026018
17	2048	4.693492132	3.543564
18	1696	7.943805176	5.315869

K is a parameter for the circuit generation by halo2-lib.

References

1. S. Agrawal, D. Boneh, and X. Boyen. Efficient lattice (H)IBE in the standard model. In *Eurocrypt*, 2010.
2. E. Alkim, L. Ducas, T. Pöppelmann, and P. Schwabe. Post-quantum key exchange - a new hope. In *USENIX Security*, 2016.
3. arkworks contributors. `arkworks` zksnark ecosystem, 2022.
4. C. Baum and V. Lyubashevsky. Simple amortized proofs of shortness for linear relations over polynomial rings. Cryptology ePrint Archive Report 2017/759, 2017.
5. W. Beullens and G. Seiler. LaBRADOR: Compact Proofs for R1CS from Module-SIS. In *Crypto*, 2023.
6. D. Boneh and V. Venkatesan. Breaking RSA may not be equivalent to factoring. In *Eurocrypt*, 1998.
7. E. Bottazzi. Greco: Fast zero-knowledge proofs for valid FHE RLWE ciphertexts formation. Cryptology ePrint Archive Report 2024/594, 2024.
8. B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *IEEE S&P*, 2018.
9. D. Catalano and D. Fiore. Vector commitments and their applications. In *PKC*, 2013.
10. I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène. TFHE: Fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1), 2020.
11. M. Dahl, C. Danjou, D. Demmler, T. Frederiksen, P. Ivanov, M. Joye, D. Rotaru, N. Smart, and L. Tremblay-Thibault. fhEVM: Confidential EVM Smart Contracts Using Fully Homomorphic Encryption. Zama white paper, https://github.com/zama-ai/fhevm/raw/main/fhevm-whitepaper.pdf, 2023. Accessed: 2023-11-22.
12. W. Dai. PESCA: a privacy-enhancing smart-contract architecture. Cryptology ePrint Archive Report 2022/1119, 2022.
13. S. Das, P. Camacho, Z. Xiang, J. Nieto, B. Bünz, and L. Ren. Threshold signatures from inner product argument: Succinct, weighted, and multi-threshold. In *ACM-CCS*, 2023.
14. R. del Pino, V. Lyubashevsky, and G. Seiler. Short discrete log proofs for FHE and Ring-LWE ciphertexts. In *PKC*, 2019.
15. J. Fan and F. Vercauteren. Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive Report 2012/144, 2012.
16. G. Fuchsbauer, E. Kiltz, and J. Loss. The algebraic group model and its applications. In *Crypto*, 2018.
17. G. Gabizon, Z. Williamson, and O. Ciobotaru. PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive, Report 2019/953,2019.
18. C. Ganesh, H. Khoshakhlagh, M. Kohlweiss, A. Nitulescu, and M. Zajac. What makes Fiat-Shamir zkSNARKs (Updatable SRS) simulation extractable? In *SCN*, 2022.
19. C. Ganesh, C. Orlandi, M. Pancholi, A. Takahashi, and D. Tschudi. Fiat–Shamir Bulletproofs are Non-Malleable (in the Algebraic Group Model). In *Eurocrypt*, 2022.
20. C. Gentry, S. Halevi, and V. Lyubashevsky. Practical non-interactive publicly verifiable secret sharing with thousands of parties. In *Eurocrypt*, 2022.
21. S. Gorbunov, L. Reyzin, H. Wee, and Z. Zhang. PointProofs: Aggregating Proofs for Multiple Vector Commitments. In *ACM-CCS*, 2020.

22. J. Groth. On the size of pairing-based non-interactive arguments. In *Eurocrypt*, 2016.
23. W. Johnson and J. Lindenstrauss. Extensions of Lipschitz mappings into a Hilbert space. *Contemporary Mathematics*, (26), 1984.
24. M. Joye. TFHE public key encryption revisited. In *CT-RSA*, 2024.
25. A. Kate, G. Zaverucha, and I. Goldberg. Constant-size commitments to polynomialsand applications. In *Asiacrypt*, 2010.
26. R.-W. Lai and G. Malavolta. Subvector commitments with application to succinct arguments. In *Crypto*, 2019.
27. B. Libert. Vector commitments with proofs of smallness: short range proofs and more. In *PKC*, 2024.
28. B. Libert, S. Ramanna, and M. Yung. Functional commitment schemes: From polynomial commitments to pairing-based accumulators from simple assumptions. In *ICALP*, 2016.
29. B. Libert and M. Yung. Concise mercurial vector commitments and independent zero-knowledge sets with short proofs. In *TCC*, 2010.
30. H. Lipmaa. On Diophantine Complexity and Statistical Zero-Knowledge Arguments. In *Asiacrypt*, 2003.
31. V. Lyubashevsky, N.-K. Nguyen, and G. Seiler. Shorter lattice-based zero-knowledge proofs via one-time commitments. In *PKC*, 2021.
32. V. Lyubashevsky, C. Peikert, and O. Regev. On ideal lattices and learning with errors over rings. In *Eurocrypt*, 2010.
33. M. Maller, S. Bowe, M. Kohlweiss, and S. Meiklejohn. Sonic: Zero-knowledge SNARKs from linear-size universal and updateable structured reference strings. In *ACM-CCS*, 2019.
34. N.-K. Nguyen and G. Seiler. Greyhound: Fast polynomial commitments from lattices. In *Crypto*, 2024.
35. P. Paillier and D. Vergnaud. Discrete-log-based signatures may not be equivalent to discrete log. In *Asiacrypt*, 2005.
36. T. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *Crypto*, 1991.
37. P. Pollack and E. Treviño. Finding the four squares in Lagrange's theorem. *Integers*, 18(#A15), 2018.
38. M. Rabin and J. Shallit. Randomized algorithms in number theory. *Communications in Pure and Applied Mathematics*, 39, 1986.
39. V. Shoup. Lower bounds for discrete logarithms and related problems. In *Eurocrypt*, 1997.
40. R. Solomon, R. Weber, and G. Almashaqbeh. smartFHE: Privacy-Preserving Smart Contracts from Fully Homomorphic Encryption. In *EuroS&P*, 2023.
41. Zama. TFHE-rs: A Pure Rust Implementation of the TFHE Scheme for Boolean and Integer Arithmetics Over Encrypted Data. https://github.com/zama-ai/tfhe-rs, 2020.

Supplementary Material

A Joye's Public-Key TFHE

For polynomials $r, s \in R$, we denote by $\bar{r}, \bar{s}$ the polynomials obtained by reversing the coefficients of r and s respectively. The public key consists of $a \in R_q$ and

$b = a \cdot \bar{s} + e$, for a binary secret key $s \in R/(2R)$ and where $e \in R$ is a noise.

For a plaintext $m \in \mathbb{Z}_t$, a random $r \in R/(2R)$, and noise terms $e_1 \in R$ and $e_2 \in \mathbb{Z}$, ciphertexts are of the form

$$(c_1, c_2) = \big(a \cdot \bar{r} + e_1, \langle \phi(b),\ \phi(r)\rangle + e_2 + \Delta \cdot m\big) \in R_q \times \mathbb{Z}_q \tag{31}$$

where $\Delta = \lceil q/t \rfloor$ (for a plaintext modulus t) and $\phi(b) = (b_0, \ldots, b_{d-1}) \in \mathbb{Z}_q^d$ contains the coefficients of $b(x) = b_0 + b_1 X + \cdots + b_{n-1} X^{d-1} \in R_q$. Note that the $c_2 \in \mathbb{Z}_q$ component of (31) can be seen as the extraction of the last coefficient from the second component $b \cdot \bar{r} + \Delta \cdot m + \mathsf{noise}$ of an LPR ciphertext since the degree-$(d-1)$ coefficient of the polynomial product $b \cdot \bar{r} \in R_q$ is $\langle \phi(b), \phi(r)\rangle$. Decryption proceeds by computing

$$\begin{aligned}
\left\lceil \frac{t}{q} \cdot \big(c_2 - \langle \phi(s), \phi(c_1)\rangle\big) \right\rfloor &= \left\lceil \frac{t}{q} \cdot \big(c_2 - (c_1 \cdot \bar{s})_{d-1}\big) \right\rfloor \\
&= \left\lceil \frac{t}{q} \cdot \big(c_2 - (a \cdot \bar{r} \cdot \bar{s} + e_1 \cdot \bar{s})_{d-1}\big) \right\rfloor \\
&= \left\lceil \frac{t}{q} \cdot \big(c_2 - ((b-e) \cdot \bar{r} + e_1 \cdot \bar{s})_{d-1}\big) \right\rfloor \\
&= \left\lceil \frac{t}{q} \cdot \big(c_2 - ((b \cdot \bar{r})_{d-1} + (e_1 \cdot \bar{s} - e \cdot \bar{r})_{d-1})\big) \right\rfloor \\
&= \left\lceil \frac{t}{q} \cdot \big(c_2 - \langle \phi(b), \phi(r)\rangle - (e_1 \cdot \bar{s} - e \cdot \bar{r})_{d-1}\big) \right\rfloor \\
&= \left\lceil \frac{t}{q} \cdot \big(e_2 + \Delta \cdot m - (e_1 \cdot \bar{s} - e \cdot \bar{r})_{d-1}\big) \right\rfloor = m
\end{aligned}$$

where $(a \cdot b)_{d-1}$ denotes the coefficient of degree $d-1$ in the product $a(X) \cdot b(X)$.

In a packed version of the scheme that encrypts k messages using the same randomness $r \in R/(2R)$, the scheme exploits the property that the coefficient vector of the ring element $b \cdot \bar{r} \in R_q$ can be obtained as

$$\phi(b \cdot \bar{r}) = \underbrace{\begin{pmatrix} b_0 & -b_{d-1} & \ldots & -b_2 & -b_1 \\ \vdots & \ddots & \ddots & & \vdots \\ b_{d-2} & b_{d-3} & \ddots & b_0 & -b_{d-1} \\ b_{d-1} & b_{d-2} & \ldots & b_1 & b_0 \end{pmatrix}}_{\triangleq\ \mathsf{rot}(b)} \cdot \underbrace{\begin{pmatrix} r_{d-1} \\ r_{d-2} \\ \vdots \\ r_0 \end{pmatrix}}_{\triangleq\ \phi(\bar{r})} \tag{32}$$

If we denote by $\phi_i(b)$ the i-th row of the matrix $\mathsf{rot}(b) \in \mathbb{Z}_q^{d \times d}$ in (32), the degree-i coefficient of $b \cdot \bar{r} \in R_q$ is equal to $\langle \phi_{i+1}(b), \phi(\bar{r})\rangle$. Therefore, we obtain a k-slot variant of the scheme by considering the last k rows of (32). Namely, one encrypts $(m_1, \ldots, m_k) \in \mathbb{Z}_t^k$ as

$$\begin{aligned}
c_1 &= a \cdot \bar{r} + e_1 \in R_q \\
c_{2,i} &= \langle \phi_{d+1-i}(b),\ \phi(\bar{r})\rangle + e_{2,i} + \Delta \cdot m_i \in \mathbb{Z}_q \qquad \forall i \in [k]
\end{aligned} \tag{33}$$

for independent noise terms $\{e_{2,i}\}_{i=1}^{k}$. Decryption proceeds by computing the i-th plaintext slot as

$$\begin{aligned}
\left\lceil \frac{t}{q} \cdot \left(c_{2,i} - \langle \phi_{d+1-i}(c_1), \phi(\bar{s}) \rangle\right) \right\rfloor &= \left\lceil \frac{t}{q} \cdot \left(c_{2,i} - (c_1 \cdot \bar{s})_{d-i}\right) \right\rfloor \\
&= \left\lceil \frac{t}{q} \cdot \left(c_{2,i} - (a \cdot \bar{r} \cdot \bar{s} + e_1 \cdot \bar{s})_{d-i}\right) \right\rfloor \\
&= \left\lceil \frac{t}{q} \cdot \left(c_{2,i} - ((b-e) \cdot \bar{r} + e_1 \cdot \bar{s})_{d-i}\right) \right\rfloor \\
&= \left\lceil \frac{t}{q} \cdot \left(c_{2,i} - ((b \cdot \bar{r})_{d-i} + (e_1 \cdot \bar{s} - e \cdot \bar{r})_{d-i})\right) \right\rfloor \\
&= \left\lceil \frac{t}{q} \cdot \left(c_{2,i} - \langle \phi_{d+1-i}(b), \phi(\bar{r}) \rangle - (e_1 \cdot \bar{s} - e \cdot \bar{r})_{d-i}\right) \right\rfloor \\
&= \left\lceil \frac{t}{q} \cdot \left(e_{2,i} + \Delta \cdot m_i - (e_1 \cdot \bar{s} - e \cdot \bar{r})_{d-i}\right) \right\rfloor = m_i
\end{aligned} \tag{34}$$

for each $i \in [k]$.

B Proof of Theorem 2

Proof. In the AGM+ROM model, we show that, under the $(2n, n)$-DLOG assumption, there is an extractor that can extract a witness from any adversarially-generated proof $\boldsymbol{\pi}$ and statement $\mathbf{x}$. Concretely, we show an algorithm $\mathcal{B}$ that either extracts a witness or solves an $(2n, n)$-DLOG instance by computing $\alpha \in \mathbb{Z}_p$ from $\{(g, g_1, \ldots, g_{2n}), (\hat{g}_1, \ldots, \hat{g}_n)\}$, where $g_i = g^{(\alpha^i)}$ and $\hat{g}_i = \hat{g}^{(\alpha^i)}$ for all i.

The problem instance $\{(g, g_1, \ldots, , g_{2n}), (\hat{g}_1, \ldots, \hat{g}_n)\}$ is used to define pp. Note that $g_{n+1} = g^{(\alpha^{n+1})}$ is not included in pp although it is part of $\mathcal{B}$'s input. Our reduction/extractor $\mathcal{B}$ interacts with $\mathcal{A}$ as follows.

Queries: At any time, $\mathcal{A}$ can choose a statement $\mathbf{x} = (q, d, k, B_\infty, t, \tilde{\mathbf{A}}, \mathbf{c})$ and request a simulated proof that $\mathbf{c} \in R_q \times \mathbb{Z}_q^k$ is a valid ciphertext. To simulate a proof, $\mathcal{B}$ chooses a random $\theta \xleftarrow{R} \mathbb{Z}_p$ and defines $\boldsymbol{\theta} = (1, \theta, \ldots, \theta^{\bar{d}+\bar{k}}) \in \mathbb{Z}_p^{\bar{d}+\bar{k}}$. It then defines the public-key-dependent matrices $\tilde{\mathbf{A}} = [\tilde{\mathbf{A}}_0 \mid -q \cdot \mathbf{I}_{d+k} \mid \mathbf{I}_{d+k}]$, where $\tilde{\mathbf{A}}_0 \in \mathbb{Z}^{(d+k) \times D}$, and the vector $\tilde{\mathbf{c}} \in \mathbb{Z}^{d+k}$ as in (16). It defines $\boldsymbol{\theta}_0 \in \mathbb{Z}_p^{d+k}$ as the first $d+k$ entries of $\boldsymbol{\theta}$ and computes $\boldsymbol{a}_\theta^\top = \boldsymbol{\theta}_0^\top \cdot \tilde{\mathbf{A}}_0 \in \mathbb{Z}_p^{1 \times D} \bmod p$ and $t_\theta = \boldsymbol{\theta}_0^\top \cdot \tilde{\mathbf{c}} \bmod p$, where $\tilde{\mathbf{c}} \in \mathbb{Z}^{d+k}$ is the vector defined in (16). We note that, even for a maliciously chosen statement $\mathbf{x}$, the first column of the matrix $\tilde{\mathbf{A}}$ is non-zero since the proven language (17) only considers non-zero public keys $(a, b) \in (R_q^*)^2$.

If the first column of $\tilde{\mathbf{A}}$ is non-zero, the first component $\boldsymbol{a}_\theta[1] \in \mathbb{Z}_p$ of $\boldsymbol{a}_\theta$ is non-zero with overwhelming probability $\geq 1 - (d+k)/p$. If $\boldsymbol{\theta}_0^\top \cdot \tilde{\mathbf{A}}_0[1] = t_\theta$, $\mathcal{B}$ can generate a real proof using the witness $\tilde{\boldsymbol{w}} = (1, 0, \ldots, 0) \in \{0,1\}^D$ and $\tilde{\boldsymbol{r}} = \boldsymbol{e} = 0$. We thus assume $\boldsymbol{\theta}_0^\top \cdot \tilde{\mathbf{A}}_0[1] \neq t_\theta$, so that $\mathcal{B}$ can compute the non-binary

$w_1 \in \mathbb{Z}_p$ satisfying the equation

$$\boldsymbol{a}_\theta[1] \cdot w_1 = t_\theta \bmod p.$$

Next, $\mathcal{B}$ simulates other proof elements as follows.

1. Choose random $t, y \xleftarrow{R} \mathbb{Z}_p$ and define vectors $\boldsymbol{t} = (t_1, \ldots, t_n) \in \mathbb{Z}_p^n$, $\boldsymbol{y} = ((y_1, \ldots, y_{D+128m}) \mid \mathbf{0}^{n-(D+128m)})$ such that $y_i = y^{i-1}$ for $i \in [D + 128m]$ and $t_i = t^{i-1}$ for each $i \in [n]$.
2. Find integers $v_1, \ldots, v_4 \in [0, B]$ such that $\sum_{i=1}^4 v_i^2 = B^2$ over $\mathbb{Z}$. Then, choose $\gamma_e, \hat{\gamma}_e, \gamma_r \xleftarrow{R} \mathbb{Z}$ and compute simulated commitments

$$C_e = g^{\gamma_e} \cdot \prod_{i=1}^{4} g_{n+1-(d+k+i)}^{v_i}, \qquad \hat{C}_e = \hat{g}^{\hat{\gamma}_e} \cdot \prod_{i=1}^{4} \hat{g}_{d+k+i}^{v_i}, \qquad C_{\tilde{r}} = g^{\gamma_r}.$$

   which commit to $\bar{\boldsymbol{e}} = (\mathbf{0}^{d+k} \mid (v_1, \ldots, v_4))$ and $\tilde{\boldsymbol{r}} = \mathbf{0}^{d+k}$.
3. Compute the product $\mathbf{w}_R = \mathbf{R} \cdot (\mathbf{0}^{d+k} \mid (v_1, \ldots, v_4)^\top \mid \mathbf{0}^{d+k})^\top \in \mathbb{Z}^{128}$, where $\mathbf{R} \in \{-1, 0, 1\}^{128 \times (2(d+k)+4)}$ is extracted from $\bar{\mathbf{R}} = H_R(\mathbf{x}, \hat{C}_e, C_e, C_{\tilde{r}}) \in \{-1, 0, 1\}^{128 \times (2(\bar{d}+\bar{k})+4)}$. Then, choose $\gamma_R \xleftarrow{R} \mathbb{Z}_p$ and compute

$$C_R = g^{\gamma_R} \cdot \prod_{i=1}^{128} g_i^{\mathbf{w}_R[i]}.$$

   Let $\mathbf{w}_{R,\mathsf{bin}} \in \{0,1\}^{128 \cdot m}$ such that $\mathbf{w}_{R,\mathsf{bin}}[(i-1) \cdot m + 1, i \cdot m] = \boldsymbol{g}_m^{-1}(\mathbf{w}_R[i])$ for each $i \in [128]$ (and thus $\mathbf{w}_R = \mathbf{H} \cdot \mathbf{w}_{R,\mathsf{bin}}$). Compute

$$\phi = H_\phi(\mathbf{x}, \mathbf{R}, \hat{C}_e, C_e, C_R, C_{\tilde{r}}) \in \mathbb{Z}_p$$

   and define $\boldsymbol{\phi} = (\phi_1, \ldots, \phi_{128}) = (1, \phi, \phi^2, \ldots, \phi^{127})$
4. Define the fake witness $\bar{\mathbf{w}} = (w_1 \mid \mathbf{0}^{D-1} \mid \mathbf{w}_{R,\mathsf{bin}}) \in \mathbb{Z}_p^{D+128m}$ (which is not a valid witness since $w_1 \notin \{0,1\}$). Then, commit to $\bar{\mathbf{w}}$ by computing

$$\hat{C}_{\mathsf{bin}} = \hat{g}^{\gamma} \cdot \hat{g}_1^{w_1} \cdot \prod_{i=D+1}^{D+128m} \hat{g}_i^{\mathbf{w}_{R,\mathsf{bin}}[i-D]}$$

   for a random $\gamma \xleftarrow{R} \mathbb{Z}_p$. Then, compute

$$\xi = H_\xi(\mathbf{x}, \hat{C}_e, C_e, \phi, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}) \in \mathbb{Z}_p$$

   and define $\boldsymbol{\xi} = (1, \xi, \xi^2, \ldots, \xi^{127})$.
5. Set $z_n = y_1$ and $z_{n+1-i} = \mathbf{w}_{R,\mathsf{bin}}[i - D] \cdot y_i$ for each $i \in [D+1, D+128m]$. Then, find an arbitrary $(z_i)_{i \in [2,n] \setminus [D+1, D+128m]} \in \mathbb{Z}_p^{n-128m-1}$ such that

$$\sum_{i \in [2,n] \setminus [D+1, D+128m]} t_i \cdot z_{n+1-i} = t_1 \cdot y_1 \cdot (w_1 - 1)$$

6. Choose random $z_0 \xleftarrow{R} \mathbb{Z}_p$ and compute a simulated commitment

$$C_y = g^{z_0} \cdot \prod_{i=1}^{n} g_i^{z_i}.$$

Program the hash values

$$H\big(\mathbf{x}, \phi, \xi, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}\big) = y,$$
$$H_t(\mathbf{x}, y, \phi, \xi, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y) = t$$
$$H_{\mathsf{lmap}}\big(\mathbf{x}, y, t, \phi, \xi, \hat{C}_e, C_e,\ C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y\big) = \theta$$

and abort if one of them was already defined. Otherwise, faithfully compute

$$\omega = H_\omega\big(\mathbf{x}, y, t, \phi, \xi, \theta, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y\big) \in \mathbb{Z}_p$$

and define $\boldsymbol{\omega} = (\omega_1, \ldots, \omega_n) = (1, \omega, \ldots, \omega^{n-1})$.

7. Define the polynomials

$$\begin{aligned}
Q_{eq}[X] &= \Big(\gamma + \sum_{i=1}^{D+128m} \bar{\mathbf{w}}[i] \cdot X^i\Big) \cdot \Big(\sum_{i=1}^{D+128m} t_i \cdot y_i \cdot X^{n+1-i}\Big) \\
&\qquad - \Big(\sum_{i=0}^{n} z_i \cdot X^i\Big) \cdot \Big(\sum_{i=1}^{n} t_i \cdot X^i\Big) = \sum_{i=0}^{n+D+128m} \rho_{eq,i} \cdot X^i, \\
Q_y[X] &= \Big(\sum_{i=0}^{n} z_i \cdot X^i - \sum_{i=1}^{D+128m} y_i \cdot X^{n+1-i}\Big) \cdot \Big(\gamma + \sum_{i=1}^{D+128m} \bar{\mathbf{w}}[i] \cdot X^i\Big) \\
&= \Big(z_0 + \sum_{i=1}^{D+128m} \big(z_{n+1-i} - y_i\big) \cdot X^{n+1-i} \\
&\qquad + \sum_{i=D+128m+1}^{n} z_{n+1-i} \cdot X^{n+1-i}\Big) \cdot \Big(\gamma + \sum_{i=1}^{D+128m} \bar{\mathbf{w}}[i] \cdot X^i\Big) \\
&= \sum_{i=0}^{n+D+128m} \rho_{y,i} \cdot X^i \\
Q_\theta[X] &= \Big(\sum_{i=1}^{D} \boldsymbol{a}_\theta[i] \cdot X^{n+1-i}\Big) \cdot \Big(\sum_{i=0}^{D+128m} w_i \cdot X^i\Big) - t_\theta \cdot X^{n+1} \\
&= \sum_{i=0}^{n+D+128m} \rho_{\theta,i} \cdot X^i
\end{aligned}$$

for which the left-hand-side members of (21), (22) and (23) can be written

$$e\Big(\prod_{i=1}^{D+128\cdot m} g_{n+1-i}^{t_i \cdot y_i}, \hat{C}_{\mathsf{bin}}\Big) \cdot e\Big(C_y, \prod_{i=1}^{n} \hat{g}_i^{t_i}\Big)^{-1} = e(g, \hat{g})^{Q_{eq}(\alpha)},$$

$$e\big(C_y \cdot \prod_{j=1}^{D+128\cdot m} g_{n+1-j}^{-y_j}, \hat{C}_{\mathsf{bin}}\big) = e(g,\hat{g})^{Q_y(\alpha)}$$

$$e\left(\prod_{i=1}^{D} g_{n+1-i}^{\boldsymbol{a}_\theta[i]},\ \hat{C}_{\mathsf{bin}}\right)\cdot e\left(C_{\tilde{r}}, \prod_{i=1}^{d+k} \hat{g}_{n+1-i}^{-q\cdot\boldsymbol{\theta}_0[i]}\right)$$

$$\cdot e\left(\prod_{i=1}^{d+k} g_{n+1-i}^{\boldsymbol{\theta}_0[i]}, \hat{C}_e\right)\cdot e(g_1,\hat{g}_n)^{-t_\theta} = e(g,\hat{g})^{Q_\theta(\alpha)}$$

The degree-$(n+1)$ coefficients of these polynomials are

$$\begin{aligned}
\rho_{eq,n+1} &= w_1 t_1 y_1 + \sum_{i=D+1}^{D+128m} \mathbf{w}_{R,\mathsf{bin}}[i-D]\cdot t_i \cdot y_i - \sum_{i=1}^{n} t_i \cdot z_{n+1-i} \\
&= t_1\cdot y_1\cdot(w_1-1) + \sum_{i=D+1}^{D+128m} t_i\cdot(\mathbf{w}_{R,\mathsf{bin}}[i-D]\cdot y_i - z_{n+1-i}) \\
&\quad - \sum_{i\in[2,n]\setminus[D+1,D+128m]} t_i\cdot z_{n+1-i} = 0 \\
\rho_{y,n+1} &= w_1\cdot(z_n - y_1) + \sum_{i=D+1}^{D+128m} \mathbf{w}_{R,\mathsf{bin}}[i-D]\cdot(z_{n+1-i}-y_i) \\
&= w_1\cdot(z_n-y_1) + \sum_{i=D+1}^{D+128m} \mathbf{w}_{R,\mathsf{bin}}[i-D]\cdot(\mathbf{w}_{R,\mathsf{bin}}[i-D]-1)\cdot y_i = 0 \\
\rho_{\theta,n+1} &= \boldsymbol{a}_\theta[1]\cdot w_1 - t_\theta = 0
\end{aligned}$$

due to the definition of committed vectors $\bar{\mathbf{w}}$ (where $\bar{\mathbf{w}}[1] = \boldsymbol{a}_\theta[1]^{-1}\cdot t_\theta$ and $\mathbf{w}_{R,\mathsf{bin}}[i]\in\{0,1\}$ for each $i\in[128m]$) and $\boldsymbol{z} = (z_1,\ldots,z_n)$. This allows $\mathcal{B}$ to compute $\pi_{eq} = g^{Q_{eq}(\alpha)}$, $\pi_y = g^{Q_y(\alpha)}$ and $\pi_\theta = g^{Q_\theta(\alpha)}$ from $(g,\{g_i\}_{i=1,i\neq n+1}^{2n})$.

8. Compute π_r, π_{dec}, π_e, π_ℓ by exploiting the fact that commitments C_e, $\hat{C}_e$, C_R were honestly computed. Namely, compute the polynomials

$$\begin{aligned}
Q_r[X] &= \Big(\sum_{j=1}^{d+k+4}\big(\sum_{i=1}^{128}\phi_i\cdot\mathbf{R}[i,j]\big)\cdot X^{n+1-j}\Big)\cdot\Big(\hat{\gamma}_e + \sum_{j=1}^{4} v_j\cdot X^{d+k+j}\Big) \\
&\quad + \gamma_r\cdot\Big(\sum_{j=1}^{d+k}\big(\sum_{i=1}^{128}\phi_i\cdot\mathbf{R}[i,d+k+4+j]\big)\cdot X^{n+1-j}\Big) \\
&\quad - \Big(\gamma_R + \sum_{i=1}^{128}\mathbf{w}_R[i]\cdot X^i\Big)\cdot\Big(\sum_{i=1}^{128}\phi_i\cdot X^{n+1-i}\Big) \\
Q_{\mathsf{dec}}[X] &= \Big(\sum_{j=1}^{128m}\big(\sum_{i=1}^{128}\boldsymbol{\xi}[i]\cdot\mathbf{H}[i,j]\cdot X^{n+1-(D+j)}\Big)\cdot\big(\gamma + \sum_{j=1}^{D+128m}\bar{\mathbf{w}}[j]\cdot X^j\big)
\end{aligned}$$

$$- \Big(\gamma_R + \sum_{i=1}^{128} \mathbf{w}_R[i] \cdot X^i\Big) \cdot \big(\sum_{i=1}^{128} \boldsymbol{\xi}[i] \cdot X^{n+1-i}\big)$$

$$\begin{aligned}
Q_e[X] &= \Big(\sum_{i=1}^{n} \omega_i \cdot X^{n+1-i}\Big) \cdot \Big(\hat{\gamma}_e + \sum_{i=1}^{4} v_i \cdot X^{d+k+i}\Big) \\
&\quad - \Big(\gamma_e + \sum_{i=1}^{4} v_i \cdot X^{n+1-(d+k+i)}\Big) \cdot \Big(\sum_{i=1}^{d+k+4} \omega_i \cdot X^i\Big) \\
Q_\ell[X] &= \Big(\gamma_e + \sum_{i=1}^{4} v_i \cdot X^{n+1-(d+k+i)}\Big) \cdot \Big(\hat{\gamma}_e + \sum_{i=1}^{4} v_i \cdot X^{d+k+i}\Big) - B^2 \cdot X^{n+1}
\end{aligned}$$

of which the degree-$(n+1)$ coefficients are

$$\begin{aligned}
\rho_{r,n+1} &= \sum_{i=1}^{128} \phi_i \cdot \Big(\underbrace{\big(\sum_{j=d+k+1}^{d+k+4} \mathbf{R}[i,j] \cdot v_j\big) - \mathbf{w}_R[i]}_{=\,0} \Big) = 0 \\
\rho_{\mathsf{dec},n+1} &= \sum_{i=1}^{128} \boldsymbol{\xi}[i] \cdot \Big(\underbrace{\sum_{j=D+1}^{D+128m} \mathbf{H}[i, j-D] \cdot \bar{\mathbf{w}}[j] - \mathbf{w}_R[i]}_{=\,0} \Big) = 0 \\
\rho_{e,n+1} &= \sum_{i=1}^{4} v_i \cdot \omega_{d+k+i} - \sum_{i=1}^{4} v_i \cdot \omega_{d+k+i} = 0 \\
\rho_{\ell,n+1} &= \sum_{i=1}^{4} v_i^4 - B^2 = 0
\end{aligned}$$

This allows computing $(\pi_r, \pi_{\mathsf{dec}}, \pi_e, \pi_\ell) = (g^{Q_r(\alpha)}, g^{Q_{\mathsf{dec}}(\alpha)}, g^{Q_e(\alpha)}, g^{Q_\ell(\alpha)})$ from $(g, \{g_i\}_{i=1, i\neq n+1}^{2n})$ without programming any random oracle.

8. Compute

$$\begin{aligned}
\boldsymbol{\delta} &= (\delta_r, \delta_{\mathsf{dec}}, \delta_{eq}, \delta_y, \delta_\theta, \delta_e, \delta_\ell) \\
&= H_{\mathrm{agg}}\big(\mathbf{x}, y, t, \phi, \xi, \theta, \omega, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y\big)
\end{aligned}$$

Output the simulated proof $\boldsymbol{\pi} = \big(\hat{C}_e, C_e, C_{\tilde{r}}, C_R, \hat{C}_{\mathsf{bin}}, C_y, \pi\big)$.

The proof $\boldsymbol{\pi}$ has the same distribution as an output of the NIZK simulator described in the proof of Theorem 1. Indeed, π is uniquely determined by the statement $\mathbf{x}$, the commitments $(\hat{C}_e, C_e, C_{\tilde{r}}, C_R, \hat{C}_{\mathsf{bin}}, C_y)$ and the hash values. Moreover, while the committed $\boldsymbol{w}, \boldsymbol{z} \in \mathbb{Z}_p^n$ are programmed in a special way, they are independent of $\mathcal{A}$'s view due to the randomness γ and z_0 in $(\hat{C}_{\mathsf{bin}}, C_y)$.

Consequently, the simulation is perfect, unless one of the random oracles has to be programmed on an input where it was previously defined. If Q_S (reps. Q_H)

is the number of queries made by $\mathcal{A}$ to the simulator (resp. to random oracles), this happens with probability $\leq (Q_S + Q_H) \cdot Q_H/p$.

We also observe that the only random oracles that are programmed by the simulator are H, H_t and H_{lmap} (at step 6). In all other random oracles, the output is always defined after the input.

Output: When $\mathcal{A}$ terminates, it outputs a statement $\mathbf{x} = (q, d, k, B_\infty, t, \tilde{\mathbf{A}}, \mathbf{c})$ and a proof $\boldsymbol{\pi} = \big(\hat{C}_e, C_e, C_{\tilde{r}}, C_R, \hat{C}_{\mathsf{bin}}, C_y, \pi\big)$. Parse the public-key dependent matrix $\tilde{\mathbf{A}}$ as $\tilde{\mathbf{A}} = [\tilde{\mathbf{A}}_0 \mid -q \cdot \mathbf{I}_{d+k} \mid \mathbf{I}_{d+k}] \in \mathbb{Z}^{(d+k)\times(D+2(d+k))}$ and let $\tilde{\mathbf{c}}$ the ciphertext-dependent vector in (16).

Since we are in the AGM, $\mathcal{A}$ must provide representations of all commitments w.r.t to the set of all $\hat{\mathbb{G}}$-elements that it could observe during the game. Since the simulator used by $\mathcal{B}$ is algebraic, it also knows a representation of each simulated $(C_e^{(i)}, C_R^{(i)}, \pi^{(i)})$ w.r.t $(g, \{g_i\}_{i\in[2n]\setminus\{n+1\}})$. It also knows a representation of $(\hat{C}_e^{(i)}, \hat{C}_{\mathsf{bin}}^{(i)})$ w.r.t. $(\hat{g}, \{\hat{g}_i\}_{i\in[n]})$. From $\mathcal{A}$'s output and the randomness of the simulation, $\mathcal{B}$ can therefore compute scalars $\{(e_i, \psi_i, z_i, w_{R,i}, \tilde{r}_i) \in \mathbb{Z}_p^4\}_{i\in[0,2n]\setminus\{n+1\}}$ and $\{(\hat{e}_i, w_i) \in \mathbb{Z}_p^2\}_{i\in[0,n]}$ such that

$$\hat{C}_{\mathsf{bin}} = \prod_{i=0}^{n} \hat{g}_i^{w_i}, \qquad C_y = \prod_{i=0, i\neq n+1}^{2n} g_i^{z_i}, \qquad \pi = \prod_{i=0, i\neq n+1}^{2n} g_i^{\psi_i},$$

$$C_e = \prod_{i=0, i\neq n+1}^{2n} g_i^{e_i}, \qquad \hat{C}_e = \prod_{i=0}^{n} \hat{g}_i^{\hat{e}_i}, \qquad C_R = \prod_{i=0, i\neq n+1}^{2n} g_i^{w_{R,i}},$$

$$C_{\tilde{r}} = \prod_{i=0, i\neq n+1}^{2n} g_i^{\tilde{r}_i},$$

where $g_0 = g$ and $\hat{g}_0 = \hat{g}$. Define the vectors $\hat{\boldsymbol{e}} = (\hat{e}_1, \ldots, \hat{e}_{d+k})^\top$, and

$$\bar{\boldsymbol{e}} \triangleq (\hat{\boldsymbol{e}}^\top \mid (\hat{e}_{d+k+1}, \ldots, \hat{e}_{d+k+4})^\top)^\top, \qquad \tilde{\boldsymbol{r}} = (\tilde{r}_1, \ldots, \tilde{r}_{d+k})^\top.$$

Solving $(2n, n)$-DLOG: We note that a non-trivial valid proof $\boldsymbol{\pi}$ cannot recycle $(\mathbf{x}, \hat{C}_e, C_e, C_{\tilde{r}}, C_R, \hat{C}_{\mathsf{bin}}, C_y)$ from a simulated proof. Said otherwise, for all queries $i \in [Q_S]$, we must have

$$(\mathbf{x}, \hat{C}_e, C_e, C_{\tilde{r}}, C_R, \hat{C}_{\mathsf{bin}}, C_y) \neq (\mathbf{x}^{(i)}, \hat{C}_e^{(i)}, C_e^{(i)}, C_{\tilde{r}}^{(i)}, C_R^{(i)}, \hat{C}_{\mathsf{bin}}^{(i)}, C_y^{(i)})$$

since the left-hand-side member of the verification equation (26) is uniquely determined by $(\mathbf{x}^{(i)}, \hat{C}_e^{(i)}, C_e^{(i)}, C_{\tilde{r}}^{(i)}, C_R^{(i)}, \hat{C}_{\mathsf{bin}}^{(i)}, C_y^{(i)})$ and it in turn determines a unique valid $\pi^{(i)} \in \mathbb{G}$ in the right-hand-side member. This implies that

$$t = H_t(\mathbf{x}, y, \phi, \xi, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y)$$

is not one of the hash values programmed by the simulator and neither is

$$H_{\mathsf{lmap}}\big(\mathbf{x}, y, t, \phi, \xi, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y\big).$$

Let the vector $\boldsymbol{a}_\theta^\top = \boldsymbol{\theta}_0^\top \cdot \tilde{\mathbf{A}}_0 \in \mathbb{Z}_p^{1\times D} \bmod p$ and the scalar $t_\theta = \boldsymbol{\theta}_0^\top \cdot \tilde{\mathbf{c}} \bmod p$ defined in the Verify algorithm. From the algebraic representations of $\mathcal{A}$'s commitments and proof π, $\mathcal{B}$ can compute

$$\begin{aligned}
P_r[X] = &\Big(\sum_{j=1}^{d+k+4}\big(\sum_{i=1}^{128}\phi_i\cdot\mathbf{R}[i,j]\big)\cdot X^{n+1-j}\Big)\cdot\Big(\sum_{j=0}^{n}\hat{e}_j\cdot X^j\Big)\\
&+\Big(\sum_{j=1}^{d+k}\big(\sum_{i=1}^{128}\phi_i\cdot\mathbf{R}[i,d+k+4+j]\big)\cdot X^{n+1-j}\Big)\cdot\Big(\sum_{j=0,j\neq n+1}^{2n}\tilde{r}_j\cdot X^j\Big)\\
&-\Big(\sum_{i=0,i\neq n+1}^{2n}w_{R,i}\cdot X^i\Big)\cdot\Big(\sum_{i=1}^{128}\phi_i\cdot X^{n+1-i}\Big)\\
P_{\mathsf{dec}}[X] = &\Big(\sum_{j=1}^{128m}\big(\sum_{i=1}^{128}\boldsymbol{\xi}[i]\cdot\mathbf{H}[i,j]\cdot X^{n+1-(D+j)}\Big)\cdot\big(\sum_{j=0}^{n}w_j\cdot X^j\big)\\
&-\Big(\sum_{i=0,i\neq n+1}^{2n}w_{R,i}\cdot X^i\Big)\cdot\big(\sum_{i=1}^{128}\boldsymbol{\xi}[i]\cdot X^{n+1-i}\big)
\end{aligned}$$

as well as the polynomials

$$\begin{aligned}
P_{eq}[X] = &\Big(\sum_{i=0}^{n}w_i\cdot X^i\Big)\cdot\Big(\sum_{i=1}^{D+128m}t_i\cdot y_i\cdot X^{n+1-i}\Big)\\
&-\Big(\sum_{i=0,i\neq n+1}^{2n}z_i\cdot X^i\Big)\cdot\Big(\sum_{i=1}^{n}t_i\cdot X^i\Big),\\
P_y[X] = &\Big(\sum_{i=0,i\neq n+1}^{2n}z_i\cdot X^i-\sum_{i=1}^{D+128m}y_i\cdot X^{n+1-i}\Big)\cdot\Big(\sum_{i=0}^{n}w_i\cdot X^i\Big)\\
= &\Big(z_0+\sum_{i=1}^{D+128m}\big(z_{n+1-i}-y_i\big)\cdot X^{n+1-i}\\
&+\sum_{i=D+128m+1}^{n}z_{n+1-i}\cdot X^{n+1-i}+\sum_{i=n+2}^{2n}z_i\cdot X^i\Big)\cdot\Big(\sum_{i=0}^{n}w_i\cdot X^i\Big)
\end{aligned}$$

and

$$\begin{aligned}
P_\theta[X] = &\Big(\sum_{i=1}^{D}\boldsymbol{a}_\theta[i]\cdot X^{n+1-i}\Big)\cdot\Big(\sum_{i=0}^{n}w_i\cdot X^i\Big)\\
&-q\cdot\Big(\sum_{i=1}^{d+k}\boldsymbol{\theta}_0[i]\cdot X^{n+1-i}\Big)\cdot\Big(\sum_{i=0,i\neq n+1}^{2n}\tilde{r}_i\cdot X^i\Big)\\
&+\Big(\sum_{i=1}^{d+k}\boldsymbol{\theta}_0[i]\cdot X^{n+1-i}\Big)\cdot\Big(\sum_{i=0}^{n}\hat{e}_i\cdot X^i\Big)-t_\theta\cdot X^{n+1}
\end{aligned}$$

$$P_e[X] = \Big(\sum_{i=1}^{n} \omega_i \cdot X^{n+1-i}\Big) \cdot \Big(\sum_{i=0}^{2n} \hat{e}_i \cdot X^i\Big) - \Big(\sum_{i=0, i\neq n+1}^{2n} e_i \cdot X^i\Big) \cdot \Big(\sum_{i=1}^{d+k+4} \omega_i \cdot X^i\Big)$$

$$P_\ell[X] = \Big(\sum_{i=0, i\neq n+1}^{2n} e_i \cdot X^i\Big) \cdot \Big(\sum_{i=0}^{n} \hat{e}_i \cdot X^i\Big) - B^2 \cdot X^{n+1}$$

for which the left-hand-side members of (19)-(25) can be written $e(g,\hat{g})^{P_r(\alpha)}$, $e(g,\hat{g})^{P_{\mathsf{dec}}(\alpha)}$, $e(g,\hat{g})^{P_{eq}(\alpha)}$, $e(g,\hat{g})^{P_y(\alpha)}$, $e(g,\hat{g})^{P_\theta(\alpha)}$, $e(g,\hat{g})^{P_e(\alpha)}$, and $e(g,\hat{g})^{P_\ell(\alpha)}$, respectively.

The right-hand-side member of (26) can be written $e(g,\hat{g})^{P_{\text{agg}}(\alpha)}$, where $P_{\text{agg}}[X] = \sum_{i=0}^{3n} \nu_i \cdot X^i$ is the polynomial

$$P_{\text{agg}}[X] = \delta_r \cdot P_r[X] + \delta_{\mathsf{dec}} \cdot P_{\mathsf{dec}}[X] + \delta_{eq} \cdot P_{eq}[X] + \delta_y \cdot P_y[X] + \delta_\theta \cdot P_\theta[X] + \delta_e \cdot P_e[X] + \delta_\ell \cdot P_\ell[X] = \sum_{i=0}^{3n} \nu_i \cdot X^i$$

In the summed polynomials, the coefficients of degree-$(n+1)$ monomials can be written

$$\gamma_{r,n+1} \triangleq \sum_{j=1}^{d+k+4} \sum_{i=1}^{128} \phi_i \cdot \mathbf{R}[i,j] \cdot \hat{e}_j + \sum_{j=1}^{d+k} \sum_{i=1}^{128} \phi_i \cdot \mathbf{R}[i, d+k+4+j] \cdot \tilde{r}_j - \sum_{i=1}^{128} \phi_i \cdot w_{R,i}$$
$$= \sum_{i=1}^{128} \phi_i \cdot \Big(\sum_{j=1}^{d+k+4} \big(\mathbf{R}[i,j] \cdot \hat{e}_j + \sum_{j=1}^{d+k} \mathbf{R}[i, d+k+4+j] \cdot \tilde{r}_j - w_{R,i}\big)\Big)$$

$$\gamma_{\mathsf{dec},n+1} \triangleq \sum_{j=1}^{128m} \sum_{i=1}^{128} \boldsymbol{\xi}[i] \cdot \mathbf{H}[i,j] \cdot w_{D+j} - \sum_{i=1}^{128} \boldsymbol{\xi}[i] \cdot w_{R,i}$$
$$= \sum_{i=1}^{128} \boldsymbol{\xi}[i] \cdot \Big(\sum_{j=1}^{128m} \mathbf{H}[i,j] \cdot w_{D+j} - w_{R,i}\Big)$$

$$\gamma_{eq,n+1} \triangleq \sum_{i=1}^{D+128m} t_i \cdot y_i \cdot w_i - \sum_{i=1}^{n} t_i \cdot z_{n+1-i}$$
$$= \sum_{i=1}^{D+128m} t_i \cdot (y_i \cdot w_i - z_{n+1-i}) - \sum_{i=D+128m+1}^{n} t_i \cdot z_{n+1-i}$$

$$\gamma_{y,n+1} \triangleq \sum_{i=1}^{D+128m} (z_{n+1-i} - y_i) \cdot w_i + \sum_{i=D+128m+1}^{n} z_{n+1-i} \cdot w_i$$

$$\gamma_{\theta,n+1} = \sum_{i=1}^{D} \boldsymbol{a}_\theta[i] \cdot w_i - q \cdot \sum_{i=1}^{d+k} \boldsymbol{\theta}_0[i] \cdot \tilde{r}_i + \sum_{i=1}^{d+k} \boldsymbol{\theta}_0[i] \cdot \hat{e}_i - t_\theta$$

$$= \boldsymbol{\theta}_0^\top \cdot \tilde{\mathbf{A}}_0 \cdot (w_1, \ldots, w_D)^\top - q \cdot \boldsymbol{\theta}_0^\top \cdot \tilde{\boldsymbol{r}} + \boldsymbol{\theta}_0^\top \cdot \hat{\boldsymbol{e}} - \boldsymbol{\theta}_0^\top \cdot \tilde{\boldsymbol{c}}$$

$$\gamma_{e,n+1} \triangleq \sum_{i=1}^{n} \omega_i \cdot \hat{e}_i - \sum_{i=1}^{d+k+4} \omega_i \cdot e_{n+1-i} = \sum_{i=1}^{d+k+4} \omega_i \cdot (\hat{e}_i - e_{n+1-i}) + \sum_{i=d+k+5}^{n} \omega_i \cdot \hat{e}_i$$

$$\gamma_{\ell,n+1} \triangleq \sum_{i=1}^{n} e_{n+1-i} \cdot \hat{e}_i - B^2$$

In $P_{\mathrm{agg}}[X]$, the coefficient ν_{n+1} of the degree-$(n+1)$ term can be written

$$\nu_{n+1} = \delta_r \cdot \gamma_{r,n+1} + \delta_{\mathsf{dec}} \cdot \gamma_{\mathsf{dec},n+1} + \delta_{eq} \cdot \gamma_{eq,n+1} + \delta_y \cdot \gamma_{y,n+1} + \delta_\theta \cdot \gamma_{\theta,n+1} + \delta_e \cdot \gamma_{e,n+1} + \delta_\ell \cdot \gamma_{\ell,n+1}.$$

We first claim that, if $\nu_{n+1} \neq 0$, then the reduction $\mathcal{B}$ can solve its $(2n, n)$-DLOG instance. Indeed, the AGM representations of all group elements contained in $\boldsymbol{\pi} = \big(\hat{C}_e, C_e, C_{\tilde{r}}, C_R, \hat{C}_{\mathsf{bin}}, C_y, \pi\big)$ allow $\mathcal{B}$ to compute $P_{\mathrm{agg}}[X] = \sum_{i=0}^{3n} \nu_i \cdot X^i$ whose coefficients form an algebraic representation of π as $\pi = g^{\nu_0} \cdot \prod_{i=1}^{3n} g_i^{\nu_i}$. However, as part of its output, $\mathcal{A}$ must have supplied a different algebraic representation of π as $\pi = g \cdot \prod_{i=1, i\neq n+1}^{2n} g_i^{\psi_i}$ (note that that this representation $\{\psi_i\}_{i\neq n+1}$ does not depend on g_{n+1} since $\mathcal{A}$ has never seen any group element that depends on g_{n+1} in the experiment). These two distinct AGM representations of π provide $\mathcal{B}$ with a non-zero polynomial

$$R[X] = \sum_{i=0}^{n} (\nu_i - \psi_i) \cdot X^i + \nu_{n+1} \cdot X^{n+1} + \sum_{i=n+2}^{2n} (\nu_i - \psi_i) \cdot X^i + \sum_{i=2n+1}^{3n} \nu_i \cdot X^i$$

of which $\alpha \in \mathbb{Z}_p$ is a root.

We now assume that $\nu_{n+1} = 0$ and argue that, in this case, $\mathcal{B}$ can reconstruct a witness with overwhelming probability. First, the aggregation coefficients

$$\boldsymbol{\delta} = (\delta_r, \delta_{\mathsf{dec}}, \delta_{eq}, \delta_y, \delta_\theta, \delta_e, \delta_\ell) = H_{\mathrm{agg}}\big(\mathbf{x}, y, t, \phi, \xi, \theta, \omega, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y\big)$$

were derived from a random oracle after the algebraic representations of the commitments $(\hat{C}_e, C_{\tilde{r}}, C_R, \hat{C}_{\mathsf{bin}}, C_y, C_e)$ which, together with the statement $\mathbf{x}$, determine the randomizers $(\mathbf{R}, \phi, \xi, y, t, \theta, \omega)$ and then the vector

$$\boldsymbol{\gamma} \triangleq (\gamma_{r,n+1}, \gamma_{\mathsf{dec},n+1}, \gamma_{eq,n+1}, \gamma_{y,n+1}, \gamma_{\theta,n+1}, \gamma_{e,n+1}, \gamma_{\ell,n+1}).$$

Therefore, since $\boldsymbol{\delta} = (\delta_r, \delta_{\mathsf{dec}}, \delta_{eq}, \delta_y, \delta_\theta, \delta_e, \delta_\ell)$ is chosen uniformly after $\boldsymbol{\gamma}$, we must have

$$(\gamma_{r,n+1}, \gamma_{\mathsf{dec},n+1}, \gamma_{eq,n+1}, \gamma_{y,n+1}, \gamma_{\theta,n+1}, \gamma_{e,n+1}, \gamma_{\ell,n+1}) = \mathbf{0}$$

with probability $\geq 1 - 1/p$. In particular, $\gamma_{\theta,n+1} = 0$ implies

$$\boldsymbol{\theta}_0^\top \cdot \big(\tilde{\mathbf{A}}_0 \cdot (w_1, \ldots, w_D)^\top - q \cdot \tilde{\boldsymbol{r}} + \hat{\boldsymbol{e}} - \tilde{\boldsymbol{c}}\big) = 0,$$

where $\hat{\boldsymbol{e}} = (\hat{e}_1, \ldots, \hat{e}_{d+k})$ and $\tilde{\boldsymbol{r}} = (\tilde{r}_1, \ldots, \tilde{r}_{d+k})$. Since $\boldsymbol{\theta}_0 = (1, \theta, \ldots, \theta^{d+k-1}) \in \mathbb{Z}_p^{d+k}$ was obtained from $\theta = H_{\mathsf{lmap}}\big(\mathbf{x}, y, t, \phi, \xi, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y\big) \in \mathbb{Z}_p$ (of which the inputs determine $\tilde{\mathbf{A}}_0$, $(w_1, \ldots, w_D)$, $\hat{\boldsymbol{e}}$, $\tilde{\boldsymbol{r}}$ and $\tilde{\boldsymbol{c}}$), we must have

$$\tilde{\mathbf{A}}_0 \cdot (w_1, \ldots, w_D)^\top - q \cdot \tilde{\boldsymbol{r}} + \hat{\boldsymbol{e}} = \tilde{\boldsymbol{c}} \mod p. \tag{35}$$

except with probability $\leq (d+k)/p$. To complete the proof, we show that (35) holds over $\mathbb{Z}$ and that $(w_1, \ldots, w_D) \in \{0,1\}^D$ and $\|\hat{\boldsymbol{e}}\| \leq B$, which implies $\|\hat{\boldsymbol{e}}\|_\infty \leq B = B_\infty \sqrt{d+k}$.

We first prove that $(w_1, \ldots, w_D) \in \{0,1\}^D$ with overwhelming probability. Since

$$\gamma_{eq,n+1} = \sum_{i=1}^{D+128m} t_i \cdot (y_i \cdot w_i - z_{n+1-i}) - \sum_{i=D+128m+1}^{n} t_i \cdot z_{n+1-i} = 0,$$

and $t = H_t(\mathbf{x}, y, \phi, \xi, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y)$ was derived from a random oracle *after* the algebraic representations $\{w_i\}_{i=0}^n$, $\{z_i\}_{i=0, i\neq n+1}^{2n}$ have been fixed and after the choice of y, we must have $z_{n+1-i} = y_i \cdot w_i$ for all $i \in [1, D+128m]$ and $z_{n+1-i} = 0$ for all $i \in [D+128m+1, n]$, except with negligible probability n/p. Then, the equality

$$\gamma_{y,n+1} = \sum_{i=1}^{D+128m} (z_{n+1-i} - y_i) \cdot w_i + \sum_{i=D+128m+1}^{n} z_{n+1-i} \cdot w_i = 0$$

becomes

$$\gamma_{y,n+1} = \sum_{i=1}^{D+128m} (y_i \cdot w_i - y_i) \cdot w_i = \sum_{i=1}^{D+128m} y_i \cdot (w_i - 1) \cdot w_i = 0. \tag{36}$$

We now consider two cases:

- If $y = H\big(\mathbf{x}, \phi, \xi, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}\big) \in \mathbb{Z}_p$ is a random oracle value programmed by the simulator (i.e., if

  $$(\mathbf{x}, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}) = (\mathbf{x}^{(i)}, \hat{C}_e^{(i)}, C_e^{(i)}, C_R^{(i)}, \hat{C}_{\mathsf{bin}}^{(i)}, C_{\tilde{r}}^{(i)})$$

  and $C_y \neq C_y^{(i)}$ for some $i \in [Q_S]$), then (36) implies

  $$0 = \gamma_{y,n+1} = \sum_{j=1}^{D+128m} y_j^{(i)} \cdot (\bar{\mathbf{w}}[j]^{(i)} - 1) \cdot \bar{\mathbf{w}}[j]^{(i)} \tag{37}$$

  $$= y_1^{(i)} \cdot (w_1^{(i)} - 1) \cdot w_1^{(i)} + \sum_{j=D+1}^{D+128m} y_j^{(i)} \cdot \underbrace{(\mathbf{w}_{R,\mathsf{bin}}[j-D]^{(i)} - 1) \cdot \mathbf{w}_{R,\mathsf{bin}}[j-D]^{(i)}}_{=\,0}$$

  (where the superscript (i) is used to denote values chosen by the simulator at the i-th query), where the second term is 0 since the simulator chose

$\mathbf{w}_{R,\mathsf{bin}}^{(i)} \in \{0,1\}^{128m}$ at the i-th query. Then, (37) can only hold if $y_1^{(i)} = 0$ since $w_1^{(i)} \notin \{0,1\}$. However, the simulator chose $y_1^{(i)} = 1$ at the i-th simulation query, meaning that we cannot have (37). This shows that $\mathcal{A}$ cannot tamper with a simulated proof and only modify the commitment $C_y^{(i)}$, which is the only case where we could have $\boldsymbol{y} = \boldsymbol{y}^{(i)}$.

- If $y = H\big(\mathbf{x}, \phi, \xi, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}\big)$ is not a random oracle value programmed by the simulator, then its output was derived *after* the algebraic representation $\{w_i\}_{i=0}^n$ of $\hat{C}_{\mathsf{bin}}$. In this case, (36) can only hold with probability $(D+128m)/p \leq n/p$ if there exists $i \in [D+128m]$ such that $w_i \notin \{0,1\}$.

We are left with showing that $\|\hat{\boldsymbol{e}}\| \leq B$ and that $\tilde{\boldsymbol{r}}$ is sufficiently small to ensure that (35) holds over $\mathbb{Z}$. Since

$$\gamma_{\mathsf{dec},n+1} = \sum_{i=1}^{128} \boldsymbol{\xi}[i] \cdot \Big(\sum_{j=1}^{128m} \mathbf{H}[i,j] \cdot w_{D+j} - w_{R,i} \Big) = 0$$

and $\xi = H_\xi(\mathbf{x}, \hat{C}_e, C_e, C_R, \boldsymbol{\phi}, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}) \in \mathbb{Z}_p$ was derived after $\{w_{D+j}\}_{i=1}^{128m}$ and $\{w_{R,i}\}_{i=1}^{128}$ (which are fixed by the algebraic representation of the inputs), we must have

$$\forall i \in [128] : w_{R,i} = \sum_{j=1}^{128m} \mathbf{H}[i,j] \cdot w_{D+j} \bmod p$$

and we previously showed that $w_{D+j} \in \{0,1\}$ for all $j \in [128m]$ w.h.p. Since $\mathbf{H} = \mathbf{I}_{128} \otimes \boldsymbol{g}_m$, this implies $w_{R,i} \in [-2^{m-1}, 2^{m-1})$ for each $i \in [128]$, except with negligible probability $127/p$ since $\boldsymbol{\xi} = (1, \xi, \ldots, \xi^{127})$. Now, we also have

$$\gamma_{r,n+1} = \sum_{i=1}^{128} \phi_i \cdot \Big(\sum_{j=1}^{d+k+4} \big(\mathbf{R}[i,j] \cdot \hat{e}_j + \sum_{j=1}^{d+k} \mathbf{R}[i, d+k+4+j] \cdot \tilde{r}_j - w_{R,i}\big) \Big) = 0.$$

Since $\boldsymbol{\phi} = (1, \phi, \ldots, \phi^{127}) \in \mathbb{Z}_p^{128}$ is defined from $\phi = H_\phi(\mathbf{x}, \mathbf{R}, \hat{C}_e, C_e, C_R, C_{\tilde{r}}) \in \mathbb{Z}_p$, which was derived randomly after $\mathbf{R}$ and the algebraic representation $\{\hat{e}_i\}_{i=1}^n$, $\{w_{R,i}, \tilde{r}_i\}_{i \neq n+1}$ of $\hat{C}_e$, C_R and $C_{\tilde{r}}$, we must have

$$\forall i \in [128] : w_{R,i} = \sum_{j=1}^{d+k+4} \mathbf{R}[i,j] \cdot \hat{e}_j + \sum_{j=1}^{d+k} \mathbf{R}[i, d+k+4+j] \cdot \tilde{r}_j \bmod p,$$

except with probability $127/p$. If we now define the vector $\mathbf{w}_R = (w_{R,1}, \ldots, w_{R,128})^\top$, we have shown so far that

$$\|\mathbf{w}_R\|_\infty = \|\mathbf{R} \cdot \big(\bar{\boldsymbol{e}}^\top \mid \tilde{\boldsymbol{r}}^\top\big)^\top \bmod p\|_\infty \leq 2^{m-1} = 2^{\lceil \log B_{\mathsf{GHL}} \rceil},$$

where $\bar{\boldsymbol{e}} \triangleq (\hat{\boldsymbol{e}}^\top \mid (\hat{e}_{d+k+1}, \ldots, \hat{e}_{d+k+4})^\top)^\top \in \mathbb{Z}_p^{d+k+4}$. By applying Lemma 1 with $\mathbf{u} = \mathbf{0}^{128}$, we find that the inequality

$$\Big\| \big(\bar{\boldsymbol{e}}^\top \mid \tilde{\boldsymbol{r}}^\top\big)^\top \Big\|_\infty \leq 2 \cdot \Big\| \mathbf{R} \cdot \big(\bar{\boldsymbol{e}}^\top \mid \tilde{\boldsymbol{r}}^\top\big)^\top \bmod p \Big\|_\infty \leq 2 \cdot 2^{m-1} = 2^m$$

holds except with probability 2^{-128} over the choice of $\mathbf{R}$ (which occurs after $\bar{\boldsymbol{e}}$ and $\tilde{\boldsymbol{r}}$ have been fixed since $\bar{\mathbf{R}} = H_R(\mathbf{x}, \hat{C}_e, C_e, C_{\tilde{r}})$).

We then obtain a tight bound $\|\hat{\boldsymbol{e}}\|_2 \leq B$ by exploiting the equalities

$$\gamma_{e,n+1} = \sum_{i=1}^{d+k+4} \omega_i \cdot (\hat{e}_i - e_{n+1-i}) + \sum_{i=d+k+5}^{n} \omega_i \cdot \hat{e}_i = 0 \mod p \tag{38}$$

$$\gamma_{\ell,n+1} = \sum_{i=1}^{n} e_{n+1-i} \cdot \hat{e}_i - B^2 = 0 \mod p \tag{39}$$

Since $\boldsymbol{\omega} = (\omega_1, \ldots, \omega_n) = (1, \omega, \ldots, \omega^{n-1})$ is defined from

$$\omega = H_\omega\big(\mathbf{x}, y, t, \phi, \xi, \theta, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y\big),$$

which was derived after the AGM representations $\{\hat{e}_i\}_{i=0}^{n}$, $\{e_i\}_{i=0, i\neq n+1}^{2n}$ of $\hat{C}_e$ and C_e, (38) ensures that $\hat{e}_i = 0$ for each $i \in [d+k+5, n]$ and $\hat{e}_i = e_{n+1-i}$ for each $i \in [d+k+4]$, except with negligible probability n/p. By plugging these equalities into (39), we find $\sum_{i=1}^{d+k+4} \hat{e}_i^2 = B^2 \bmod p$. Over the integers, this can be written $\sum_{i=1}^{d+k+4} \hat{e}_i^2 = B^2 + \mu \cdot p$ for some $\mu \in \mathbb{Z}$.

However, we have previously shown that

$$\|\bar{\boldsymbol{e}}\|_\infty \leq \left\| \left(\bar{\boldsymbol{e}}^\top \mid \tilde{\boldsymbol{r}}^\top\right)^\top \right\|_\infty \leq 2^m,$$

which implies[15]

$$\sum_{i=1}^{d+k+4} \hat{e}_i^2 < (d+k+4) \cdot 2^{2m} < p.$$

Therefore we must have $\mu = 0$, so that $\|\bar{\boldsymbol{e}}\|_2 = B$ and thus $\|\hat{\boldsymbol{e}}\|_2 \leq B$.

We finally show that the equality

$$\tilde{\mathbf{A}}_0 \cdot (w_1, \ldots, w_D)^\top + \hat{\boldsymbol{e}} - q \cdot \tilde{\boldsymbol{r}} = \tilde{\boldsymbol{c}} \mod p. \tag{40}$$

also holds over $\mathbb{Z}$ and not only mod p. Since $\|\hat{\boldsymbol{e}}\|_\infty \leq \|\hat{\boldsymbol{e}}\|_2 \leq B$, the left-hand-side member of (40) has infinity norm smaller than

$$D \cdot \tfrac{q}{2} + B + q \cdot \|\tilde{\boldsymbol{r}}\|_\infty = q \cdot \tfrac{d+k\cdot\log t}{2} + B + q \cdot \|\tilde{\boldsymbol{r}}\|_\infty.$$

We have shown before that

$$\|\tilde{\boldsymbol{r}}\|_\infty \leq \left\| \left(\bar{\boldsymbol{e}}^\top \mid \tilde{\boldsymbol{r}}^\top\right)^\top \right\|_\infty \leq 2 \cdot 2^{m-1} = 2^m.$$

Since we chose p such that[16]

$$\tfrac{p}{2} > q \cdot \left(\tfrac{D}{2} + 2^m\right) + B = q \cdot \left(\tfrac{d+k\cdot\log t}{2} + 2^m\right) + B,$$

[15] For the parameters of interest $d = 2048$, $k = 320$, $\sigma = 2^{12}$, $B = \sigma\sqrt{2(d+k)} \approx 2^{18.10}$, $B_{\mathsf{GHL}} \approx 2^{21.21}$ and $m = 23$, we have $(d+k+4) \cdot 2^{2m} < 2^{58}$, which is way smaller than $p > 2^{256}$.

[16] For parameters $d = 2048$, $k = 320$, $t = 2^5$, $B \approx 2^{18.10}$, $B_{\mathsf{GHL}} \approx 2^{21.31}$, $m = 23$ and $q \approx 2^{64}$, we have $q \cdot (d + k \cdot \log t + 2^{m+1}) + 2B < 2^{89} \ll p$.

(40) becomes

$$\tilde{\mathbf{A}}_0 \cdot (w_1, \ldots, w_D)^\top + \hat{\boldsymbol{e}} - q \cdot \tilde{\boldsymbol{r}} = \tilde{\boldsymbol{c}}$$

over $\mathbb{Z}$ (without any modular reduction).

Then, when reducing modulo q, we have $\tilde{\mathbf{A}}_0 \cdot (w_1, \ldots, w_D)^\top + \hat{\boldsymbol{e}} = \tilde{\boldsymbol{c}} \bmod q$ with $\|\hat{\boldsymbol{e}}\|_2 \leq B$. From the binary vector $(w_1, \ldots, w_D) \in \{0,1\}^D$ obtained from the algebraic representation of $\hat{C}_{\mathsf{bin}}$, $\mathcal{B}$ can extract a valid witness by setting $\bar{r} = \phi^{-1}((w_1, \ldots, w_d)) \in R/(2R)$ and

$$(m_1, \ldots, m_k) = (\boldsymbol{g}_{\log t} \cdot (w_{d+1}, \ldots, w_{d+\log t})^\top, \\ \ldots, \boldsymbol{g}_{\log t} \cdot (w_{d+(k-1)\log t+1}, \ldots, w_{d+k\log t})^\top).$$

□

C Description of the First Fast-Verifier Variant

In this section, we describe our first verifier-optimized construction, where verification only requires $O(d+k)$ exponentiations in $\hat{\mathbb{G}}$, instead of $O(n)$ exponentiations in each group in our first construction.

To shorten the description, we directly write the most efficient way to compute the aggregate proof $\pi = g^{P_\pi(\alpha)}$ from the polynomial $P_\pi[X]$ at step 13. The prover is identical to that of the (optimized) construction of Section 3.1 in its steps 1 to 13. From step 14 onwards, the prover verifiably computes exponentiations on behalf of the verifier. As part of the process, step 16 computes a batch evaluation proof for three KZG commitments on a common input.

CRS-Gen$(\lambda, (\bar{q}, \bar{d}, \bar{k}, \bar{B}_\infty, \bar{t}))$**:** Given a security parameter λ, a maximal dimension $\bar{d} \in \mathsf{poly}(\lambda)$, integers $\bar{q}, \bar{t}, \bar{k} \in \mathsf{poly}(\lambda)$, set $\bar{B} = \bar{B}_\infty \cdot \sqrt{\bar{d}+\bar{k}} \in \mathsf{poly}(\lambda)$, and $\bar{B}_{\mathsf{GHL}} = 9.75 \cdot \sqrt{\bar{B}^2 + \frac{(\bar{d}+2)^2(\bar{d}+\bar{k})}{4}}$, $\bar{m} = \lceil 1 + \log \bar{B}_{\mathsf{GHL}} \rceil$. Then, do the following.

1. Generate asymmetric pairing-friendly groups $(\mathbb{G}, \hat{\mathbb{G}}, \mathbb{G}_T)$ of prime order

   $$p > \max\left(2^{l(\lambda)}, \bar{q} \cdot (\bar{d} + \bar{k} \cdot (\log \bar{t} - 1) + 2^{\bar{m}+1}) + 2\bar{B}, (\bar{d} + \bar{k} + 4) \cdot 2^{2\bar{m}}\right),$$

   for some polynomial $l : \mathbb{N} \to \mathbb{N}$. Let

   $$n \geq \max(\bar{d} + \bar{k} \cdot (\log \bar{t} - 1) + 128 \cdot \bar{m}, 2(\bar{d} + \bar{k}) + 4).$$

2. Pick a random $\alpha \xleftarrow{R} \mathbb{Z}_p$ and compute $g_1, \ldots, g_n, g_{n+2}, \ldots, g_{2n} \in \mathbb{G}$ as well as $\hat{g}_1, \ldots, \hat{g}_n \in \hat{\mathbb{G}}$, where $g_i = g^{(\alpha^i)}$ for each $i \in [2n] \setminus \{n+1\}$ and $\hat{g}_i = \hat{g}^{(\alpha^i)}$ for each $i \in [n]$.
3. Choose a hash function $H_R : \{0,1\}^* \to \mathcal{D}^{128 \times (2(\bar{d}+\bar{k})+4)}$, where $\mathcal{D}$ is the distribution that outputs 0 with probability 1/2 and 1 or -1 with probability 1/4 each.

4. Choose hash functions $H, H_t, H_\omega : \{0,1\}^* \to \mathbb{Z}_p$, $H_{\mathrm{agg}} : \{0,1\}^* \to \mathbb{Z}_p^7$, $H_{\mathsf{lmap}} : \{0,1\}^* \to \mathbb{Z}_p$, $H_\phi, H_\xi : \{0,1\}^* \to \mathbb{Z}_p$ and $H_z, H_\chi : \{0,1\}^* \to \mathbb{Z}_p$ that will be modeled as random oracles.

Output the CRS

$$\mathsf{pp} = \Big((\mathbb{G}, \hat{\mathbb{G}}, \mathbb{G}_T), g, \hat{g}, \{g_i\}_{i \in [2n] \setminus \{n+1\}}, \{\hat{g}_i\}_{i \in [n]}, \mathbf{H} \Big),$$

where $\mathbf{H} = \{H, H_R, H_\phi, H_\xi, H_t, H_\omega, H_{\mathrm{agg}}, H_{\mathsf{lmap}}, H_z, H_\chi\}$.

$\mathsf{Prove}_{\mathsf{pp}}(\mathbf{x}, \mathbf{w})$: Given a statement $\mathbf{x} = (q, d, k, B_\infty, t, \tilde{\mathbf{A}}, \mathbf{c})$ consisting of dimensions $d \leq \bar{d}$, moduli $q \leq \bar{q}$ and $t \leq \bar{t}$, a ciphertext $\mathbf{c} = (c_1, (c_{2,i})_{i=1}^k) \in R_q \times \mathbb{Z}_q^k$, a norm bound $B_\infty \leq \bar{B}_\infty$ for the vector $\boldsymbol{e} \triangleq (\phi(e_1), e_{2,1}, \ldots, e_{2,k}) \in \mathbb{Z}^{d+k}$, a matrix $\tilde{\mathbf{A}}$ of the form (16), as well as a witness $\mathbf{w} = (\bar{r}, e_1, (m_i, e_{2,i})_{i=1}^k) \in R^2 \times (\mathbb{Z}_t \times \mathbb{Z})^k$ such that (13) holds with $\bar{r} \in \{0,1\}^d$, return $\perp$ if $\|\boldsymbol{e}\|_\infty > B_\infty$ or if there exists $i \in [k]$ such that $\mathsf{msb}(m_i) \neq 0$. Otherwise, set the ℓ_2-norm bound $B = B_\infty \sqrt{d+k}$ and do the following.

1. Define $B_{\mathsf{GHL}} = 9.75 \cdot \sqrt{B^2 + \frac{(d+2)^2(d+k)}{4}}$ and $m = \lceil 1 + \log B_{\mathsf{GHL}} \rceil \leq \bar{m}$.
2. Compute the quotients $r_1 \in R$ and $(r_{2,1}, \ldots, r_{2,k}) \in \mathbb{Z}^k$ such that $\|r_1\|_\infty, \|r_{2,i}\|_\infty \leq \frac{d+1}{2}$ for each $i \in [k]$ and satisfying (14). Encode the noise terms $(e_1, \{e_{2,i}\}_{i=1}^k)$ as $\boldsymbol{e} = \big(\phi(e_1)^\top \mid (e_{2,i}^\top)_{i=1}^k\big)^\top \in \mathbb{Z}^{d+k}$ and the quotients $(r_1, \{r_{2,i}\}_{i=1}^k)$ as $\tilde{\boldsymbol{r}} = \big(\phi(r_1)^\top \mid (r_{2,i}^\top)_{i=1}^k\big)^\top \in \mathbb{Z}^{d+k}$. Then, encode $(r, m_1, \ldots, m_k)$ as a vector

$$\tilde{\boldsymbol{w}} = \big[\ \phi(\bar{r}) \mid \boldsymbol{m}_1 \mid \ldots \mid \boldsymbol{m}_k\ \big]^\top \in \{0,1\}^D,$$

of dimension

$$D = d + k \cdot (\log t - 1)$$

using bit decompositions $\boldsymbol{m}_i = \bar{\boldsymbol{g}}_{\log t}^{-1}(m_i) \in \{0,1\}^{\log t - 1}$ for each $i \in [k]$. Note that we have $\tilde{\mathbf{c}} = \tilde{\mathbf{A}}_0 \cdot \tilde{\boldsymbol{w}} - q \cdot \tilde{\boldsymbol{r}} + \boldsymbol{e}$ over $\mathbb{Z}$ in (16).
3. Choose $\hat{\gamma}_e, \gamma_e \xleftarrow{R} \mathbb{Z}_p$ and commit to $\bar{\boldsymbol{e}} = (\boldsymbol{e} \mid (v_1, \ldots, v_4)) \in \mathbb{Z}^{d+k+4}$ by computing

$$\hat{C}_e = \hat{g}^{\hat{\gamma}_e} \cdot \prod_{i=1}^{d+k+4} \hat{g}_i^{\bar{\boldsymbol{e}}[i]}, \qquad C_e = g^{\gamma_e} \cdot \prod_{i=1}^{d+k+4} g_{n+1-i}^{\bar{\boldsymbol{e}}[i]},$$

where $v_1, \ldots, v_4 \in [0, B]$ are integers such that $\sum_{i=1}^4 v_i^2 = B^2 - \|\boldsymbol{e}\|^2$. Also, choose $\gamma_r \xleftarrow{R} \mathbb{Z}_p$ and compute

$$C_{\tilde{r}} = g^{\gamma_r} \cdot \prod_{i=1}^{d+k} g_i^{\tilde{\boldsymbol{r}}[i]}.$$

4. Compute $\bar{\mathbf{R}} = H_R(\mathbf{x}, \hat{C}_e, C_e, C_{\tilde{r}}) \in \{-1,0,1\}^{128\times(2(\bar{d}+\bar{k})+4)}$ of which the columns are distributed as per $\mathcal{D}^{128}$. Let $\mathbf{R} \in \{-1,0,1\}^{128\times(2(d+k)+4)}$ denote the submatrix consisting of the first $d+k+4$ columns of $\bar{\mathbf{R}}$. Compute
$$\mathbf{w}_R = \mathbf{R} \cdot \begin{bmatrix} \bar{\boldsymbol{e}} \\ \tilde{\boldsymbol{r}} \end{bmatrix} \in \mathbb{Z}^{128}$$
as a compressed version of $\bar{\boldsymbol{e}} \in \mathbb{Z}^{d+k+4}$ and $\tilde{\boldsymbol{r}} \in \mathbb{Z}^{d+k}$. If $\|\mathbf{w}_R\|_\infty > B_{\mathsf{GHL}}$, abort and return $\perp$.
5. Choose $\gamma_R \xleftarrow{R} \mathbb{Z}_p$ and compute
$$C_R = g^{\gamma_R} \cdot \prod_{i=1}^{128} g_i^{\mathbf{w}_R[i]}$$
Then, compute $\phi = H_\phi(\mathbf{x}, \mathbf{R}, \hat{C}_e, C_e, C_R, C_{\tilde{r}}) \in \mathbb{Z}_p$ and define $\boldsymbol{\phi} = (\phi_1, \ldots, \phi_{128}) \in \mathbb{Z}_p^{128}$ where $\phi_i = \phi^{i-1}$ for each $i \in [128]$.
6. Let $\mathbf{w}_{R,\mathsf{bin}} \in \{0,1\}^{128\cdot m}$ the binary decomposition of $\mathbf{w}_R$ such that $\mathbf{w}_{R,\mathsf{bin}}[(i-1)\cdot m + 1, i\cdot m] = \boldsymbol{g}_m^{-1}(\mathbf{w}_R[i])$ for each $i \in [128]$. Define the vector $\mathbf{w}_{\mathsf{bin}} = (\tilde{\boldsymbol{w}} \mid \mathbf{w}_{R,\mathsf{bin}} \mid \mathbf{0}^{n-(D+128\cdot m)}) \in \{0,1\}^n$. Commit to it by computing
$$\hat{C}_{\mathsf{bin}} = \hat{g}^{\gamma_{\mathsf{bin}}} \cdot \prod_{i=1}^{D+128\cdot m} \hat{g}_i^{\mathbf{w}_{\mathsf{bin}}[i]}$$
where $\gamma_{\mathsf{bin}} \xleftarrow{R} \mathbb{Z}_p$.
7. Compute $\xi = H_\xi(\mathbf{x}, \hat{C}_e, C_e, \phi, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}) \in \mathbb{Z}_p$ and define the vector $\boldsymbol{\xi} = (\xi_1, \ldots, \xi_{128})$ with $\xi_i = \xi^{i-1}$ for each $i \in [128]$.
8. Compute $y = H\big(\mathbf{x}, \phi, \xi, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}\big) \in \mathbb{Z}_p$. Define $\boldsymbol{y} = (y_1, \ldots, y_n) \in \mathbb{Z}_p^n$ where $y_i = y^{i-1}$ for each $i \in [D + 128\cdot m]$ and $y_i = 0$ for each $i \in [D + 128\cdot m + 1, n]$. Next, choose $\gamma_y \xleftarrow{R} \mathbb{Z}_p$ and compute
$$C_y = g^{\gamma_y} \cdot \prod_{j=1}^{D+128\cdot m} g_{n+1-j}^{y_j \cdot \mathbf{w}_{\mathsf{bin}}[j]}$$
Compute
$$t = H_t(\mathbf{x}, y, \phi, \xi, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y) \in \mathbb{Z}_p$$
and define $\boldsymbol{t} = (t_1, \ldots, t_n) \in \mathbb{Z}_p^n$ where $t_i = t^{i-1}$ for each $i \in [n]$.
9. Compute $\theta = H_{\mathsf{lmap}}\big(\mathbf{x}, y, t, \phi, \xi, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y\big) \in \mathbb{Z}_p$ and define $\boldsymbol{\theta} = (\theta_1, \ldots, \theta_{\bar{d}+\bar{k}}) \in \mathbb{Z}_p^{\bar{d}+\bar{k}}$ where $\theta_i = \theta^{i-1}$ for each $i \in [\bar{d}+\bar{k}]$. Define $\tilde{\mathbf{A}} = [\tilde{\mathbf{A}}_0 \mid -q\cdot\mathbf{I}_{d+k} \mid \mathbf{I}_{d+k}] \in \mathbb{Z}^{(d+k)\times(D+2(d+k))}$ and $\tilde{\mathbf{c}} \in \mathbb{Z}^{d+k}$ as in (16) with $\tilde{\mathbf{A}}_0 \in \mathbb{Z}^{(d+k)\times D}$. Let $\boldsymbol{\theta}_0 \in \mathbb{Z}_p^{d+k}$ the first $d+k$ entries of $\boldsymbol{\theta}$.
10. Let $t_\theta = \boldsymbol{\theta}_0^\top \cdot \tilde{\mathbf{c}} \bmod p$ and $\boldsymbol{a}_\theta^\top = \boldsymbol{\theta}_0^\top \cdot \tilde{\mathbf{A}}_0 \in \mathbb{Z}_p^{1\times D} \bmod p$.

11. Compute

$$\omega = H_\omega\big(\mathbf{x}, y, t, \phi, \xi, \theta, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y\big) \in \mathbb{Z}_p.$$

Define $\boldsymbol{\omega} = (\omega_1, \ldots, \omega_n) \in \mathbb{Z}_p^n$, where $\omega_i = \omega^{i-1}$ for each $i \in [n]$.

12. Let

$$\begin{aligned}\boldsymbol{\delta} &= (\delta_r, \delta_{\mathsf{dec}}, \delta_{eq}, \delta_y, \delta_\theta, \delta_e, \delta_\ell) \\ &= H_{\mathrm{agg}}\big(\mathbf{x}, y, t, \phi, \xi, \theta, \omega, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y\big).\end{aligned}$$

13. Let $\bar{\boldsymbol{\theta}}_0 = (\boldsymbol{\theta}_0 \mid \mathbf{0}^{n-(d+k)}) \in \mathbb{Z}_p^n$ and define the matrices $\mathbf{R}' \in \mathbb{Z}_p^{128\times n}$ and $\mathbf{H}_\xi \in \mathbb{Z}_p^{128\times(D+128\cdot m)}$ such that

$$\forall i \in [128] : \mathbf{R}'[i,j] = \begin{cases} \mathbf{R}[i,j] & \text{if } \ j \in [1, 2(d+k)+4] \\ 0 & \text{if } \ j \in [2(d+k)+5, n]. \end{cases}$$

and

$$\forall i \in [128] : \mathbf{H}_\xi[i,j] = \begin{cases} \boldsymbol{\xi}[i] \cdot \mathbf{H}[i, j-D] & \text{if } \ j \in [D+1, D+128\cdot m] \\ 0 & \text{if } \ j \in [1, D]. \end{cases}$$

where $\mathbf{H} = \mathbf{I}_{128} \otimes \boldsymbol{g}_m^\top \in \mathbb{Z}_p^{128\times 128\cdot m}$. Compute the polynomial

$$\begin{aligned}P_\pi[X] = \Big(\delta_y \cdot \gamma_y + \Big(&\sum_{j=1}^{D+128m} \delta_y \cdot y_j \cdot (\mathbf{w}_{\mathsf{bin}}[j] - 1) + \delta_\theta \cdot \bar{\boldsymbol{a}}_\theta[j] + \delta_{eq} \cdot t_j \cdot y_j \\ &+ \delta_{\mathsf{dec}} \cdot \sum_{i=1}^{128} \mathbf{H}_\xi[i,j]\Big) \cdot X^{n+1-j}\Big) \cdot \Big(\gamma_{\mathsf{bin}} + \sum_{j=1}^{D+128m} \mathbf{w}_{\mathsf{bin}}[j] \cdot X^j\Big) \\ + \Big(\delta_\ell \cdot \big(\gamma_e + &\sum_{j=1}^{d+k+4} \bar{\boldsymbol{e}}[j] \cdot X^{n+1-j}\big) + \sum_{j=1}^{n} \big(\delta_\theta \cdot \bar{\boldsymbol{\theta}}_0[j] + \sum_{i=1}^{128} \delta_r \cdot \phi_i \cdot \mathbf{R}'[i,j] + \delta_e \cdot \omega_j\big) \cdot X^{n+1-j}\Big) \\ &\cdot \big(\hat{\gamma}_e + \sum_{j=1}^{d+k+4} \bar{\boldsymbol{e}}[j] \cdot X^j\big) \\ + \Big(\gamma_r + \sum_{j=1}^{d+k} \tilde{\boldsymbol{r}}[j] \cdot X^j\Big) \cdot \Big(&\sum_{j=1}^{d+k} \Big(-\delta_\theta \cdot q \cdot \boldsymbol{\theta}_0[j] + \sum_{i=1}^{128} \delta_r \cdot \phi_i \cdot \mathbf{R}[i, d+k+4+j]\Big) \cdot X^{n+1-j}\Big) \\ &- \Big(\gamma_R + \sum_{j=1}^{128} \mathbf{w}_R[j] \cdot X^j\Big) \cdot \Big(\sum_{j=1}^{128} (\delta_r \cdot \phi_j + \delta_{\mathsf{dec}} \cdot \boldsymbol{\xi}[j]) \cdot X^{n+1-j}\Big) \\ &- \delta_e \cdot \big(\gamma_e + \sum_{j=1}^{d+k+4} \bar{\boldsymbol{e}}[j] \cdot X^{n+1-j}\big) \cdot \Big(\sum_{j=1}^{d+k+4} \omega_j \cdot X^j\Big) \\ - \delta_{eq} \cdot \Big(\gamma_y + &\sum_{j=1}^{D+128m} y_j \cdot \mathbf{w}_{\mathsf{bin}}[j] \cdot X^{n+1-j}\Big) \cdot \Big(\sum_{j=1}^{n} t_j \cdot X^j\Big) - (\delta_\theta \cdot t_\theta + \delta_\ell \cdot B^2) \cdot X^{n+1},\end{aligned}$$

From the coefficients of $P_\pi[X] = \sum_{i=1}^{n+D+128m} \nu_i \cdot X^i$, compute

$$\pi = g^{P_\pi(\alpha)} = g^{\nu_0} \cdot \prod_{i=1, i\neq n+1}^{n+D+128m} g_i^{\nu_i}, \tag{41}$$

14. Compute the deterministic commitments $\hat{C}_t = \prod_{j=1}^{n} \hat{g}_j^{t_j}$ and

$$C_{h,1} = \prod_{j=1}^{D+128\cdot m} g_{n+1-j}^{\delta_\theta \cdot \bar{\boldsymbol{a}}_\theta[j] - \delta_y \cdot y_j + \delta_{eq} \cdot t_j \cdot y_j + \delta_{\mathsf{dec}} \cdot \sum_{i=1}^{128} \mathbf{H}_\xi[i,j]}$$

$$C_{h,2} = \prod_{j=1}^{n} g_{n+1-j}^{\delta_\theta \cdot \bar{\boldsymbol{\theta}}_0[j] + \sum_{i=1}^{128} \delta_r \cdot \phi_i \cdot \mathbf{R}'[i,j] + \delta_e \cdot \omega_j}$$

15. Compute an evaluation input

$$z = H_z\big(\mathbf{x}, y, t, \phi, \xi, \theta, \omega, \boldsymbol{\delta}, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y, C_{h,1}, C_{h,2}, \hat{C}_t\big) \in \mathbb{Z}_p$$

and evaluate the polynomials

$$P_{h,1}[X] = \sum_{j=1}^{D+128m} \Big(\delta_\theta \cdot \bar{\boldsymbol{a}}_\theta[j] - \delta_y \cdot y_j + \delta_{eq} \cdot t_j \cdot y_j + \delta_{\mathsf{dec}} \cdot \sum_{i=1}^{128} \mathbf{H}_\xi[i,j]\Big) \cdot X^{n+1-i}$$

$$P_{h,2}[X] = \sum_{j=1}^{n} \Big(\delta_\theta \cdot \bar{\boldsymbol{\theta}}_0[j] + \sum_{i=1}^{128} \delta_r \cdot \phi_i \cdot \mathbf{R}'[i,j] + \delta_e \cdot \omega_j\Big) \cdot X^{n+1-j}$$

$$P_t[X] = \sum_{i=1}^{n} t_i \cdot X^i, \tag{42}$$

on the input z. Let the valuations $(p_{h,1}, p_{h,2}, p_t) = (P_{h,1}(z), P_{h,2}(z), P_t(z))$.

16. Compute

$$\chi = H_\chi\big(\mathbf{x}, y, t, \phi, \xi, \theta, \omega, \boldsymbol{\delta}, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y, C_{h,1}, C_{h,2}, \hat{C}_t, z, p_{h,1}, p_{h,2}, p_t\big).$$

Then, compute the polynomial $Q[X] = \sum_{i=0}^{n-1} q_i \cdot X^i$ such that

$$Q_{\mathsf{KZG}}[X] = \frac{(P_{h,1}[X] + \chi \cdot P_{h,2}[X] + \chi^2 \cdot P_t[X]) - (p_{h,1} + p_{h,2} \cdot \chi + \chi^2 \cdot p_t)}{X - z}$$

Finally, compute $\pi_{\mathsf{KZG}} = g^{q_0} \cdot \prod_{i=1}^{n-1} g_i^{q_i} = g^{Q_{\mathsf{KZG}}(\alpha)}$.

Output the final proof

$$\boldsymbol{\pi} = \big(\hat{C}_e, C_e, C_{\tilde{r}}, C_R, \hat{C}_{\mathsf{bin}}, C_y, C_{h,1}, C_{h,2}, \hat{C}_t, \pi, \pi_{\mathsf{KZG}}\big).$$

$\mathsf{Verify}_{\mathsf{pp}}(\mathbf{x}, \boldsymbol{\pi})$: Given a statement $\mathbf{x} = (q, d, k, B_\infty, t, \tilde{\mathbf{A}}, \mathbf{c})$ and a candidate $\boldsymbol{\pi}$, return 0 if $\boldsymbol{\pi}$ does not parse properly. Otherwise, set $B = B_\infty \cdot \sqrt{d+k}$ and do the following.

1. Compute $\bar{\mathbf{R}} = H_R(\mathbf{x}, \hat{C}_e, C_e, C_{\tilde{r}}) \in \{-1, 0, 1\}^{128 \times (2(\bar{d}+\bar{k})+4)}$ whose columns are distributed as per $\mathcal{D}^{128}$. Let $\mathbf{R} \in \{-1, 0, 1\}^{128 \times (2(d+k)+4)}$ the submatrix consisting of the first $2(d+k)+4$ columns of $\bar{\mathbf{R}}$.
2. Set $\phi = H_\phi(\mathbf{x}, \mathbf{R}, \hat{C}_e, C_e, C_R, C_{\tilde{r}})$, $\xi = H_\xi(\mathbf{x}, \hat{C}_e, C_e, \phi, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}})$, and $y = H\big(\mathbf{x}, \phi, \xi, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}\big) \in \mathbb{Z}_p$. Then, compute

$$\begin{aligned} t &= H_t(\mathbf{x}, y, \phi, \xi, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y) \in \mathbb{Z}_p, \\ \theta &= H_{\mathsf{lmap}}\big(\mathbf{x}, y, t, \phi, \xi, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y\big) \in \mathbb{Z}_p \\ \omega &= H_\omega\big(\mathbf{x}, y, t, \phi, \xi, \theta, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y\big) \in \mathbb{Z}_p, \end{aligned}$$

   and

$$\begin{aligned} \boldsymbol{\delta} &= (\delta_r, \delta_{\mathsf{dec}}, \delta_{eq}, \delta_y, \delta_\theta, \delta_e, \delta_\ell) \\ &= H_{\mathrm{agg}}\big(\mathbf{x}, y, t, \phi, \xi, \theta, \omega, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y\big). \end{aligned}$$

   Let $\boldsymbol{\phi} = (\phi_1, \ldots, \phi_{128}), \boldsymbol{\xi} = (\xi_1, \ldots, \xi_{128}) \in \mathbb{Z}_p^{128}$ such that $\phi_i = \phi^{i-1}$ and $\xi_i = \xi^{i-1}$ for each $i \in [128]$. Define $\boldsymbol{\theta} = (\theta_1, \ldots, \theta_{\bar{d}+\bar{k}}) \in \mathbb{Z}_p^{\bar{d}+\bar{k}}$, where $\theta_i = \theta^{i-1}$ for each $i \in [\bar{d}+\bar{k}]$. Define $\boldsymbol{\omega} = (\omega_1, \ldots, \omega_n) \in \mathbb{Z}_p^n$, $\boldsymbol{t} = (t_1, \ldots, t_n)$ with $t_i = t^{i-1}$ and $\omega_i = \omega^{i-1}$ for each $i \in [n]$. Define $\boldsymbol{y} = (y_1, \ldots, y_n)$ with $y_i = y^{i-1}$ for each $i \in [D + 128 \cdot m]$ and $y_i = 0$ for each $i \in [D + 128 \cdot m + 1, n]$.
3. Define $\tilde{\mathbf{A}} = [\tilde{\mathbf{A}}_0 \mid -q \cdot \mathbf{I}_{d+k} \mid \mathbf{I}_{d+k}] \in \mathbb{Z}^{(d+k) \times (D+2(d+k))}$ and $\tilde{\mathbf{c}} \in \mathbb{Z}^{d+k}$ as in (16) with $\tilde{\mathbf{A}}_0 \in \mathbb{Z}^{(d+k) \times D}$. Let $\boldsymbol{\theta}_0 \in \mathbb{Z}_p^{d+k}$ the first $d+k$ entries of $\boldsymbol{\theta}$. Let $t_\theta = \boldsymbol{\theta}_0^\top \cdot \tilde{\mathbf{c}} \bmod p$ and $\boldsymbol{a}_\theta^\top = \boldsymbol{\theta}_0^\top \cdot \tilde{\mathbf{A}}_0 \in \mathbb{Z}_p^{1 \times D} \bmod p$.
4. Let $\bar{\boldsymbol{\theta}}_0 = (\boldsymbol{\theta}_0 \mid \mathbf{0}^{n-(d+k)}) \in \mathbb{Z}_p^n$ and define the matrices $\mathbf{R}' \in \mathbb{Z}_p^{128 \times n}$ and $\mathbf{H}_\xi \in \mathbb{Z}_p^{128 \times (D+128 \cdot m)}$ as in step 13 of Prove. Return 1 if the following equality holds and 0 otherwise:

$$\begin{aligned} & e\left(C_y^{\delta_y} \cdot C_{h,1},\ \hat{C}_{\mathsf{bin}}\right) \cdot e\left(C_e^{\delta_\ell} \cdot C_{h,2}, \hat{C}_e\right) \\ & \qquad \cdot e\Big(C_{\tilde{r}}, \prod_{j=1}^{d+k} \hat{g}_{n+1-j}^{-\delta_\theta \cdot q \cdot \boldsymbol{\theta}_0[j] + \delta_r \cdot \sum_{i=1}^{128} \phi_i \cdot \mathbf{R}[i, d+k+4+j]}\Big) \\ & \qquad \cdot e\big(C_R, \prod_{j=1}^{128} \hat{g}_{n+1-j}^{\delta_r \cdot \phi_j + \delta_{\mathsf{dec}} \cdot \boldsymbol{\xi}[j]}\big)^{-1} \cdot e\big(C_e^{\delta_e}, \prod_{i=1}^{d+k+4} \hat{g}_i^{\omega_i}\big)^{-1} \\ & \qquad\qquad \cdot e\big(C_y^{\delta_{eq}}, \hat{C}_t\big)^{-1} \cdot e(g_1, \hat{g}_n)^{-\delta_\theta \cdot t_\theta - \delta_\ell \cdot B^2} = e(\pi, \hat{g}) \end{aligned} \tag{43}$$

5. Compute

$$z = H_z\big(\mathbf{x}, y, t, \phi, \xi, \theta, \omega, \boldsymbol{\delta}, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y, C_{h,1}, C_{h,2}, \hat{C}_t\big) \in \mathbb{Z}_p$$

and

$$\chi = H_\chi\big(\mathbf{x}, y, t, \phi, \xi, \theta, \omega, \boldsymbol{\delta}, \hat{C}_e, C_e, C_R, \\ \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y, C_{h,1}, C_{h,2}, \hat{C}_t, z, p_{h,1}, p_{h,2}, p_t\big).$$

Compute $(p_{h,1}, p_{h,2}, p_t) = (P_{h,1}(z), P_{h,2}(z), P_t(z))$ by evaluating the polynomials in (42).

6. Return 0 if the following equality does not hold:

$$e\big(C_{h,1} \cdot C_{h,2}^{\chi} \cdot g^{-p_{h,1}-\chi \cdot p_{h,2}}, \hat{g}\big) \\ \cdot e(g, \hat{C}_t \cdot \hat{g}^{-p_t})^{\chi^2} = e(\pi_{\mathsf{KZG}}, \hat{g}_1 \cdot \hat{g}^{-z}). \qquad (44)$$

Otherwise, return 1.

Efficiency. The CRS size remains the same as in Section 3 (i.e., 1450 KB). The proof consists of 8 elements of $\mathbb{G}$ and 3 elements of $\hat{\mathbb{G}}$. On a BLS12-446 curve, each element of $\mathbb{G}$ (resp. $\hat{\mathbb{G}}$) has a 446-bit (resp. 892-bit) representation and the proof fits in 780 bytes. We note that, after the initial verification, the verifier can discard the helper commitments $(C_{h,1}, C_{h,2}, \hat{C}_t)$ and only store the shorter proof $\big(\hat{C}_e, C_e, C_{\tilde{r}}, C_R, \hat{C}_{\mathsf{bin}}, C_y, \pi\big)$ that can still be verified using the slower verification algorithm of Section 3.1.

The prover's work is dominated by $6n + 2(d + k)$ exponentiations[17] in $\mathbb{G}$ and $n + d + k$ exponentiations (actually, one multi-exponentiation with n generators) in $\hat{\mathbb{G}}$. With $n = 6784$, $d = 2048$ and $k = 320$, the prover has to compute 45440 exponentiations in $\mathbb{G}$ (resp. 9280 exponentiations in $\hat{\mathbb{G}}$), but only ≈ 57600 (resp. 6784) of them involve exponents that are not small with respect to p. In comparison, the prover of the fast-verifier variant of [27] was computing the equivalent of 456000 exponentiations in $\mathbb{G}$.

The verifier has to compute two multi-exponentiations of dimension $d + k$ and one of dimension 128 in $\hat{\mathbb{G}}$ in addition to two pairing-products. Using batch verification techniques, the two verification equations (43)-(44) can be merged into a single batched product of 8 pairings. To do this, the verifier can choose a random $\eta \xleftarrow{R} \mathbb{Z}_p$ and compute

$$e\left(C_y^{\delta_y} \cdot C_{h,1},\ \hat{C}_{\mathsf{bin}}\right) \cdot e\left(C_e^{\delta_\ell} \cdot C_{h,2}, \hat{C}_e\right) \\ \cdot e\Big(C_{\tilde{r}}, \prod_{j=1}^{d+k} \hat{g}_{n+1-j}^{-\delta_\theta \cdot q \cdot \boldsymbol{\theta}_0[j] + \delta_r \cdot \sum_{i=1}^{128} \phi_i \cdot \mathbf{R}[i, d+k+4+j]}\Big) \cdot e\Big(C_R, \prod_{j=1}^{128} \hat{g}_{n+1-j}^{\delta_r \cdot \phi_j + \delta_{\mathsf{dec}} \cdot \boldsymbol{\xi}[j]}\Big)^{-1} \\ \cdot e\Big(C_e^{\delta_e}, \prod_{i=1}^{d+k+4} \hat{g}_i^{\omega_i}\Big)^{-1} \cdot e\big(C_y^{-\delta_{eq}} \cdot g^{\eta \cdot \chi^2}, \hat{C}_t\big) \cdot e(g_n^{-\delta_\theta \cdot t_\theta - \delta_\ell \cdot B^2} \cdot \pi_{\mathsf{KZG}}^{-\eta}, \hat{g}_1)$$

[17] In fact, 4 multi-exponentiations with n generators and one multi-exponentiation with $2n$ generators. The multi-exponentiations that compute C_e and $C_{\tilde{r}}$ are cheaper since they only involve $d + k$ generators and the exponents are small w.r.t. p.

$$= e\Big(\pi \cdot C_{h,1}^{-\eta} \cdot g^{\eta\cdot(p_{h,1}+\chi\cdot p_{h,2}+\chi^2\cdot p_t)} \cdot C_{h,2}^{-\chi\cdot\eta} \cdot \pi_{\mathsf{KZG}}^{-z\cdot\eta}\ ,\ \hat{g}\Big) \tag{45}$$

We note that, if the verifier stores the full proof (instead of its compressed version distributed as in Section 3.1), it does not have to store the full CRS but only (g, g_n) and $(\hat{g}, (\hat{g}_i)_{i\in[1,d+k+4]}, (\hat{g}_{n+1-i})_{i\in[1,d+k]})$.

As we show in Supplementary Material D, it is possible to further decrease the verification time by also having the prover verifiably compute

$$\prod_{j=1}^{d+k} \hat{g}_{n+1-j}^{-\delta_\theta\cdot q\cdot\boldsymbol{\theta}_0[j]+\delta_r\cdot\sum_{i=1}^{128}\phi_i\cdot\mathbf{R}[i,d+k+4+j]}, \qquad \prod_{i=1}^{d+k+4} \hat{g}_i^{\omega_i}$$

on behalf of the verifier. In this case, the verifier only computes ≈ 128 exponentiations and it only needs to store a very small number of CRS components.

Efficiency Without the Heuristic. If we set $d = 2048$, $k = 320$ and derive $\bar{m} = 30$ from the bound $\bar{B}_{\mathsf{CS}}$ given by (30) so as to not rely on the heuristic of Lemma 2, we obtain and $n = 7168$. The verification cost remains the same. The prover computes 47744 exponentiations in $\mathbb{G}$ and 9536 exponentiations in $\hat{\mathbb{G}}$. Among these, the number of exponentiations with full-size exponents is only $6n \approx 43000$ in $\mathbb{G}$ and $n = 7168$ in $\hat{\mathbb{G}}$ (or the equivalent of ≈ 57350 exponentiations in $\mathbb{G}$ on BLS12-446 curves).

Encrypting Longer Messages. The above variant of the scheme scales better than the one of [27] when we want to increase the parameter k so as to pack more message components in a given ciphertext. In order to have $\bar{k} = 640$ while keeping $\bar{d} = 2048$, $\bar{t} = 2^5$ and $\bar{B}_\infty = 2^{17}$, we have $\bar{B} = \bar{B}_\infty\sqrt{\bar{d}+\bar{k}} \approx 2^{22.69}$ and $\bar{B}_{\mathsf{CS}} < 2^{28.89}$, so that we can still set $\bar{m} = \lceil 1+\log(\bar{B}_{\mathsf{CS}})\rceil = 30$. Then, the minimal dimension of committed vectors becomes $n \geq \bar{d} + \bar{k}\cdot(\log\bar{t} - 1) + 128\cdot\bar{m} = 8448$ and the prover now computes 56064 (resp. 11136) exponentiations in $\mathbb{G}$ (resp. $\hat{\mathbb{G}}$). For $\bar{k} = 640$, we would have $n \geq 75136$ in the scheme of [27].

For $\bar{k} = 1024$, we have $\bar{B} = 2^{22.79}$, $\bar{B}_{\mathsf{CS}} \approx 2^{29.08}$ and $\bar{m} = 31$, which leads to vectors of dimension $n \geq 10112$. Then, the CRS size amounts to 2676KB when only the x-coordinate of each point is stored. For $\bar{k} = 1024$, the scheme of [27] would require $n \geq 87000$ and a CRS size ≈ 18.9MB.

C.1 Security

Theorem 3. *Under the same conditions as in Theorem 2, the above variant provides simulation-extractability in the AGM+ROM if the $(2n, n)$-DLOG assumption holds.*

Proof. The proof proceeds identically to the proof of Theorem 2 with the difference that it additionally considers the case of adversarially-generated proofs containing malformed commitments $(C_{h,1}, C_{h,2}, \hat{C}_t)$. Let the real polynomials $(P_{h,1}[X], P_{h,2}[X], P_t[X])$ defined in (42). From the algebraic representations of

commitments $(C_{h,1}, C_{h,2}, \hat{C}_t)$, the reduction can compute polynomials $Q_{h,1}[X]$, $Q_{h,2}[X]$ and $Q_t[X]$ such that $C_{h,i} = g^{Q_{h,i}(\alpha)}$ for each $i \in \{1,2\}$ and $\hat{C}_t = \hat{g}^{Q_t(\alpha)}$. If at least one of the commitments $C_{h,1}$, $C_{h,2}$ or $\hat{C}_t$ is malformed, we must have $(Q_{h,1}[X], Q_{h,2}[X], Q_t[X]) \neq (P_{h,1}[X], P_{h,2}[X], P_t[X])$. By the Schwartz-Zippel lemma, this implies

$$Q_{h,1}[X] + \chi \cdot Q_{h,2}[X] + \chi^2 \cdot Q_t[X] \neq P_{h,1}[X] + \chi \cdot P_{h,2}[X] + \chi^2 \cdot P_t[X] \tag{46}$$

with overwhelming probability $\geq 1 - 2/p$ over the choice of χ since

$$(\mathbf{x}, C_{h,1}, C_{h,2}, \hat{C}_t, \phi, y, t, \boldsymbol{\delta}, \omega, \xi, \theta)$$

are included in the inputs of H_χ and they completely determine the polynomials $(P_{h,1}[X], P_{h,2}[X], P_t[X])$ (via $(\mathbf{x}, \phi, y, t, \boldsymbol{\delta}, \omega, \xi, \theta)$ in and the matrix $\mathbf{R}$, which is hashed using H_ϕ to derive ϕ) and $(Q_{h,1}[X], Q_{h,2}[X], Q_t[X])$ (via the algebraic representations of $C_{h,1}$, $C_{h,2}$ and $\hat{C}_t$).

From the KZG verification equation (44), we know that

$$(Q_{h,1}(\alpha) - p_{h,1}) + \chi \cdot (Q_{h,2}(\alpha) - p_{h,2}) + \chi^2 \cdot (Q_t(\alpha) - p_t) = \pi(\alpha) \cdot (\alpha - z)$$

where $\pi[X]$ is defined by the algebraic representation of $\pi_{\mathsf{KZG}} = g^{\pi(\alpha)}$. Then, the polynomial identity

$$(Q_{h,1}[X] - p_{h,1}) + \chi \cdot (Q_{h,2}[X] - p_{h,2}) + \chi^2 \cdot (Q_t[X] - p_t) = \pi[X] \cdot (X - z) \tag{47}$$

must hold unless the reduction can compute $\alpha \in \mathbb{Z}_p$ by factoring a non-zero polynomial. Since $(p_{h,1}, p_{h,2}, p_t) = (P_{h,1}(z), P_{h,2}, P_t(z))$, the identity (47) then implies

$$(Q_{h,1}(z) - P_{h,1}(z)) + \chi \cdot (Q_{h,2}(z) - P_{h,2}(z)) + \chi^2 \cdot (Q_t(z) - P_t(z)) = 0$$

Due to the inequality (46), the Schwartz-Zippel lemma implies that the above equality is only possible with negligible probability $\leq n/p$ because the input

$$z = H_z\big(\mathbf{x}, y, t, \phi, \xi, \theta, \omega, \boldsymbol{\delta}, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y, C_{h,1}, C_{h,2}, \hat{C}_t\big) \in \mathbb{Z}_p$$

is defined after $(Q_{h,1}[X], Q_{h,2}[X], Q_t[X], P_{h,1}[X], P_{h,2}[X], P_t[X])$.

This shows that, if an algebraic cheating prover sends malformed commitments $(C_{h,1}, C_{h,2}, \hat{C}_t) \neq (g^{P_{h,1}(\alpha)}, g^{P_{h,2}(\alpha)}, \hat{g}^{P_t(\alpha)})$, it can only provide an accepting π_{KZG} (i.e., satisfying (44)) with negligible probability under the $(2n, n)$-DLOG assumption. □

D Yet Another Tradeoff Reducing the Number of Exponentiations at the Verifier

In this section, we slightly modify the proof system of Section C in such a way that the prover only computes about 128 exponentiations in $\hat{\mathbb{G}}$. To do this, we ask the prover to compute $2(d+k)$ additional exponentiations in $\hat{\mathbb{G}}$. The modified construction is as follows.

$\mathsf{CRS\text{-}Gen}(\lambda, (\bar{q}, \bar{d}, \bar{k}, \bar{B}_\infty, \bar{t}))$: Given a security parameter λ, a maximal dimension $\bar{d} \in \mathsf{poly}(\lambda)$, integers $\bar{q}, \bar{t}, \bar{k} \in \mathsf{poly}(\lambda)$, set $\bar{B} = \bar{B}_\infty \cdot \sqrt{\bar{d} + \bar{k}} \in \mathsf{poly}(\lambda)$, and $\bar{B}_{\mathsf{GHL}} = 9.75 \cdot \sqrt{\bar{B}^2 + \frac{(\bar{d}+2)^2(\bar{d}+\bar{k})}{4}}$, $\bar{m} = \lceil 1 + \log \bar{B}_{\mathsf{GHL}} \rceil$. Then, do the following.

1. Generate asymmetric pairing-friendly groups $(\mathbb{G}, \hat{\mathbb{G}}, \mathbb{G}_T)$ of prime order

$$p > \max\left(2^{l(\lambda)}, \bar{q} \cdot (\bar{d} + \bar{k} \cdot (\log \bar{t} - 1) + 2^{\bar{m}+1}) + 2\bar{B}, (\bar{d} + \bar{k} + 4) \cdot 2^{2\bar{m}}\right),$$

   for some polynomial $l : \mathbb{N} \to \mathbb{N}$. Let

$$n \geq \max(\bar{d} + \bar{k} \cdot (\log \bar{t} - 1) + 128 \cdot \bar{m}, 2(\bar{d} + \bar{k}) + 4).$$

2. Pick a random $\alpha \xleftarrow{R} \mathbb{Z}_p$ and compute $g_1, \ldots, g_n, g_{n+2}, \ldots, g_{2n} \in \mathbb{G}$ as well as $\hat{g}_1, \ldots, \hat{g}_n \in \hat{\mathbb{G}}$, where $g_i = g^{(\alpha^i)}$ for each $i \in [2n] \setminus \{n+1\}$ and $\hat{g}_i = \hat{g}^{(\alpha^i)}$ for each $i \in [n]$.
3. Choose a hash function $H_R : \{0,1\}^* \to \mathcal{D}^{128 \times (2(\bar{d}+\bar{k})+4)}$, where $\mathcal{D}$ is the distribution that outputs 0 with probability 1/2 and 1 or -1 with probability 1/4 each.
4. Choose hash functions $H, H_t, H_\omega : \{0,1\}^* \to \mathbb{Z}_p$, $H_{\mathrm{agg}} : \{0,1\}^* \to \mathbb{Z}_p^7$, $H_{\mathsf{lmap}} : \{0,1\}^* \to \mathbb{Z}_p$, $H_\phi, H_\xi : \{0,1\}^* \to \mathbb{Z}_p$ and $H_z, H_\chi : \{0,1\}^* \to \mathbb{Z}_p$. that will be modeled as random oracles.

Output the CRS

$$\mathsf{pp} = \left((\mathbb{G}, \hat{\mathbb{G}}, \mathbb{G}_T), g, \hat{g}, \{g_i\}_{i \in [2n] \setminus \{n+1\}}, \{\hat{g}_i\}_{i \in [n]}, \mathbf{H}\right),$$

where $\mathbf{H} = \{H, H_R, H_\phi, H_\xi, H_t, H_\omega, H_{\mathrm{agg}}, H_{\mathsf{lmap}}, H_z, H_\chi\}$.

$\mathsf{Prove}_{\mathsf{pp}}(\mathbf{x}, \mathbf{w})$: Given a statement $\mathbf{x} = (q, d, k, B_\infty, t, \tilde{\mathbf{A}}, \mathbf{c})$ consisting of dimensions $d \leq \bar{d}$, moduli $q \leq \bar{q}$ and $t \leq \bar{t}$, a ciphertext $\mathbf{c} = (c_1, (c_{2,i})_{i=1}^k) \in R_q \times \mathbb{Z}_q^k$, a norm bound $B \leq \bar{B}$ for the vector $\boldsymbol{e} \triangleq (\phi(e_1), e_{2,1}, \ldots, e_{2,k}) \in \mathbb{Z}^{d+k}$, as well as a witness $\mathbf{w} = (\bar{r}, e_1, (m_i, e_{2,i})_{i=1}^k) \in R^2 \times (\mathbb{Z}_t \times \mathbb{Z})^k$ such that (13) holds with $\bar{r} \in \{0,1\}^d$, return $\perp$ if $\|\boldsymbol{e}\|_\infty > B_\infty$ or if there exists $i \in [k]$ such that $\mathsf{msb}(m_i) \neq 0$. Otherwise, set $B = B_\infty \sqrt{d+k}$ and do the following.

1. Define $B_{\mathsf{GHL}} = 9.75 \cdot \sqrt{B^2 + \frac{(d+2)^2(d+k)}{4}}$ and $m = \lceil 1 + \log B_{\mathsf{GHL}} \rceil \leq \bar{m}$.
2. Compute the quotients $r_1 \in R$ and $(r_{2,1}, \ldots, r_{2,k}) \in \mathbb{Z}^k$ such that $\|r_1\|_\infty, \|r_{2,i}\|_\infty \leq \frac{d+1}{2}$ for each $i \in [k]$ and satisfying (14). Encode the noise terms $(e_1, \{e_{2,i}\}_{i=1}^k)$ as $\boldsymbol{e} = \left(\phi(e_1)^\top \mid (e_{2,i}^\top)_{i=1}^k\right)^\top \in \mathbb{Z}^{d+k}$ and the quotients $(r_1, \{r_{2,i}\}_{i=1}^k)$ as $\tilde{\boldsymbol{r}} = \left(\phi(r_1)^\top \mid (r_{2,i}^\top)_{i=1}^k\right)^\top \in \mathbb{Z}^{d+k}$. Then, encode $(r, m_1, \ldots, m_k)$ as a vector

$$\tilde{\boldsymbol{w}} = \left[\ \phi(\bar{r}) \mid \boldsymbol{m}_1 \mid \ldots \mid \boldsymbol{m}_k\ \right]^\top \in \{0,1\}^D,$$

   of dimension

$$D = d + k \cdot (\log t - 1)$$

using bit decompositions $\boldsymbol{m}_i = \bar{\boldsymbol{g}}_{\log t}^{-1}(m_i) \in \{0,1\}^{\log t - 1}$ for each $i \in [k]$. Note that we have $\tilde{\mathbf{c}} = \tilde{\mathbf{A}}_0 \cdot \tilde{\boldsymbol{w}} - q \cdot \tilde{\boldsymbol{r}} + \boldsymbol{e}$ over $\mathbb{Z}$ in (16).

3. Choose $\hat{\gamma}_e, \gamma_e \xleftarrow{R} \mathbb{Z}_p$ and commit to $\bar{\boldsymbol{e}} = (\boldsymbol{e} \mid (v_1, \ldots, v_4)) \in \mathbb{Z}^{d+k+4}$ by computing

$$\hat{C}_e = \hat{g}^{\hat{\gamma}_e} \cdot \prod_{i=1}^{d+k+4} \hat{g}_i^{\bar{\boldsymbol{e}}[i]}, \qquad C_e = g^{\gamma_e} \cdot \prod_{i=1}^{d+k+4} g_{n+1-i}^{\bar{\boldsymbol{e}}[i]},$$

where $v_1, \ldots, v_4 \in [0, B]$ are integers such that $\sum_{i=1}^{4} v_i^2 = B^2 - \|\boldsymbol{e}\|^2$. Also, choose $\gamma_r \xleftarrow{R} \mathbb{Z}_p$ and compute

$$C_{\tilde{r}} = g^{\gamma_r} \cdot \prod_{i=1}^{d+k} g_i^{\tilde{\boldsymbol{r}}[i]}.$$

4. Compute $\bar{\mathbf{R}} = H_R(\mathbf{x}, \hat{C}_e, C_e, C_{\tilde{r}}) \in \{-1,0,1\}^{128 \times (2(\bar{d}+\bar{k})+4)}$ whose columns are distributed as per $\mathcal{D}^{128}$. Let $\mathbf{R} \in \{-1,0,1\}^{128 \times (2(d+k)+4)}$ denote the submatrix consisting of the first $d+k+4$ columns of $\bar{\mathbf{R}}$. Compute

$$\mathbf{w}_R = \mathbf{R} \cdot \begin{bmatrix} \bar{\boldsymbol{e}} \\ \tilde{\boldsymbol{r}} \end{bmatrix} \in \mathbb{Z}^{128}$$

as a compressed version of $\bar{\boldsymbol{e}} \in \mathbb{Z}^{d+k+4}$ and $\tilde{\boldsymbol{r}} \in \mathbb{Z}^{d+k}$. If $\|\mathbf{w}_R\|_\infty > B_{\mathsf{GHL}}$, abort and return $\perp$.

5. Choose $\gamma_R \xleftarrow{R} \mathbb{Z}_p$ and compute

$$C_R = g^{\gamma_R} \cdot \prod_{i=1}^{128} g_i^{\mathbf{w}_R[i]}$$

Then, compute $\phi = H_\phi(\mathbf{x}, \mathbf{R}, \hat{C}_e, C_e, C_R, C_{\tilde{r}}) \in \mathbb{Z}_p$ and define $\boldsymbol{\phi} = (\phi_1, \ldots, \phi_{128}) \in \mathbb{Z}_p^{128}$ where $\phi_i = \phi^{i-1}$ for each $i \in [128]$.

6. Let $\mathbf{w}_{R,\mathsf{bin}} \in \{0,1\}^{128 \cdot m}$ the binary decomposition of $\mathbf{w}_R$ such that $\mathbf{w}_{R,\mathsf{bin}}[(i-1) \cdot m + 1, i \cdot m] = \boldsymbol{g}_m^{-1}(\mathbf{w}_R[i])$ for each $i \in [128]$. Define the vector $\mathbf{w}_{\mathsf{bin}} = (\tilde{\boldsymbol{w}} \mid \mathbf{w}_{R,\mathsf{bin}} \mid \mathbf{0}^{n-(D+128 \cdot m)}) \in \{0,1\}^n$. Commit to it by computing

$$\hat{C}_{\mathsf{bin}} = \hat{g}^{\gamma_{\mathsf{bin}}} \cdot \prod_{i=1}^{D+128 \cdot m} \hat{g}_i^{\mathbf{w}_{\mathsf{bin}}[i]}$$

where $\gamma_{\mathsf{bin}} \xleftarrow{R} \mathbb{Z}_p$.

7. Compute $\xi = H_\xi(\mathbf{x}, \hat{C}_e, C_e, \phi, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}) \in \mathbb{Z}_p$ and define the vector $\boldsymbol{\xi} = (\xi_1, \ldots, \xi_{128})$ with $\xi_i = \xi^{i-1}$ for each $i \in [128]$.
8. Compute $y = H\big(\mathbf{x}, \phi, \xi, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}\big) \in \mathbb{Z}_p$ and use it to define $\boldsymbol{y} = (y_1, \ldots, y_n) \in \mathbb{Z}_p^n$ where $y_i = y^{i-1}$ for each $i \in [D + 128 \cdot m]$ and $y_i = 0$ for each $i \in [D + 128 \cdot m + 1, n]$. Next, choose $\gamma_y \xleftarrow{R} \mathbb{Z}_p$ and compute

$$C_y = g^{\gamma_y} \cdot \prod_{j=1}^{D+128 \cdot m} g_{n+1-j}^{y_j \cdot \mathbf{w}_{\mathsf{bin}}[j]}$$

Compute

$$t = H_t(\mathbf{x}, y, \phi, \xi, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y) \in \mathbb{Z}_p$$

and define $\boldsymbol{t} = (t_1, \ldots, t_n) \in \mathbb{Z}_p^n$ where $t_i = t^{i-1}$ for each $i \in [n]$.

9. Compute $\theta = H_{\mathsf{lmap}}\big(\mathbf{x}, y, t, \phi, \xi, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y\big) \in \mathbb{Z}_p$ and define $\boldsymbol{\theta} = (\theta_1, \ldots, \theta_{\bar{d}+\bar{k}}) \in \mathbb{Z}_p^{\bar{d}+\bar{k}}$ where $\theta_i = \theta^{i-1}$ for each $i \in [\bar{d}+\bar{k}]$. Define $\tilde{\mathbf{A}} = [\tilde{\mathbf{A}}_0 \mid -q \cdot \mathbf{I}_{d+k} \mid \mathbf{I}_{d+k}] \in \mathbb{Z}^{(d+k)\times(D+2(d+k))}$ and $\tilde{\mathbf{c}} \in \mathbb{Z}^{d+k}$ as in (16) with $\tilde{\mathbf{A}}_0 \in \mathbb{Z}^{(d+k)\times D}$. Let $\boldsymbol{\theta}_0 \in \mathbb{Z}_p^{d+k}$ the first $d+k$ entries of $\boldsymbol{\theta}$.
10. Let $t_\theta = \boldsymbol{\theta}_0^\top \cdot \tilde{\mathbf{c}} \bmod p$ and $\boldsymbol{a}_\theta^\top = \boldsymbol{\theta}_0^\top \cdot \tilde{\mathbf{A}}_0 \in \mathbb{Z}_p^{1\times D} \bmod p$.
11. Compute

$$\omega = H_\omega\big(\mathbf{x}, y, t, \phi, \xi, \theta, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y\big) \in \mathbb{Z}_p.$$

Define $\boldsymbol{\omega} = (\omega_1, \ldots, \omega_n) \in \mathbb{Z}_p^n$, where $\omega_i = \omega^{i-1}$ for each $i \in [n]$.
12. Let

$$\begin{aligned}\boldsymbol{\delta} &= (\delta_r, \delta_{\mathsf{dec}}, \delta_{eq}, \delta_y, \delta_\theta, \delta_e, \delta_\ell) \\ &= H_{\mathrm{agg}}\big(\mathbf{x}, y, t, \phi, \xi, \theta, \omega, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y\big).\end{aligned}$$

13. Let $\bar{\boldsymbol{\theta}}_0 = (\boldsymbol{\theta}_0 \mid \mathbf{0}^{n-(d+k)}) \in \mathbb{Z}_p^n$ and define the matrices $\mathbf{R}' \in \mathbb{Z}_p^{128\times n}$ and $\mathbf{H}_\xi \in \mathbb{Z}_p^{128\times(D+128\cdot m)}$ such that

$$\forall i \in [128] : \mathbf{R}'[i,j] = \begin{cases} \mathbf{R}[i,j] & \text{if } \ j \in [1, 2(d+k)+4] \\ 0 & \text{if } \ j \in [2(d+k)+5, n]. \end{cases}$$

and

$$\forall i \in [128] : \mathbf{H}_\xi[i,j] = \begin{cases} \boldsymbol{\xi}[i] \cdot \mathbf{H}[i, j-D] & \text{if } \ j \in [D+1, D+128\cdot m] \\ 0 & \text{if } \ j \in [1, D]. \end{cases}$$

where $\mathbf{H} = \mathbf{I}_{128} \otimes \boldsymbol{g}_m^\top \in \mathbb{Z}_p^{128\times 128\cdot m}$. Compute the polynomial

$$\begin{aligned} P_\pi[X] = \Big(\delta_y \cdot \gamma_y + \sum_{j=1}^{D+128m} \Big(\delta_y \cdot y_j \cdot (\mathbf{w}_{\mathsf{bin}}[j]-1) + \delta_\theta \cdot \bar{\boldsymbol{a}}_\theta[j] + \delta_{eq} \cdot t_j \cdot y_j \\ + \delta_{\mathsf{dec}} \cdot \sum_{i=1}^{128} \mathbf{H}_\xi[i,j]\Big) \cdot X^{n+1-j}\Big) \cdot \Big(\gamma_{\mathsf{bin}} + \sum_{j=1}^{D+128m} \mathbf{w}_{\mathsf{bin}}[j] \cdot X^j\Big) \\ + \Big(\delta_\ell \cdot \big(\gamma_e + \sum_{j=1}^{d+k+4} \bar{\boldsymbol{e}}[j] \cdot X^{n+1-j}\big) + \sum_{j=1}^{n} \big(\delta_\theta \cdot \bar{\boldsymbol{\theta}}_0[j] + \sum_{i=1}^{128} \delta_r \cdot \phi_i \cdot \mathbf{R}'[i,j] + \delta_e \cdot \omega_j\big) \cdot X^{n+1-j}\Big) \\ \cdot \big(\hat{\gamma}_e + \sum_{j=1}^{d+k+4} \bar{\boldsymbol{e}}[j] \cdot X^j\big) \end{aligned}$$

$$+\Big(\gamma_r+\sum_{j=1}^{d+k}\tilde{\boldsymbol{r}}[j]\cdot X^j\Big)\cdot\Big(\sum_{j=1}^{d+k}\Big(-\delta_\theta\cdot q\cdot\boldsymbol{\theta}_0[j]+\sum_{i=1}^{128}\delta_r\cdot\phi_i\cdot\mathbf{R}[i,d+k+4+j]\Big)\cdot X^{n+1-j}\Big)$$

$$-\Big(\gamma_R+\sum_{j=1}^{128}\mathbf{w}_R[j]\cdot X^j\Big)\cdot\Big(\sum_{j=1}^{128}(\delta_r\cdot\phi_j+\delta_{\mathsf{dec}}\cdot\boldsymbol{\xi}[j])\cdot X^{n+1-j}\Big)$$

$$-\delta_e\cdot\big(\gamma_e+\sum_{j=1}^{d+k+4}\bar{\boldsymbol{e}}[j]\cdot X^{n+1-j}\big)\cdot\Big(\sum_{j=1}^{d+k+4}\omega_j\cdot X^j\Big)$$

$$-\delta_{eq}\cdot\Big(\gamma_y+\sum_{j=1}^{D+128m}y_j\cdot\mathbf{w}_{\mathsf{bin}}[j]\cdot X^{n+1-j}\Big)\cdot\Big(\sum_{j=1}^{n}t_j\cdot X^j\Big)-(\delta_\theta\cdot t_\theta+\delta_\ell\cdot B^2)\cdot X^{n+1},$$

From the coefficients of $P_\pi[X]=\sum_{i=1}^{n+D+128m}\nu_i\cdot X^i$, compute

$$\pi=g^{P_\pi(\alpha)}=g^{\nu_0}\cdot\prod_{i=1,i\neq n+1}^{n+D+128m}g_i^{\nu_i},\tag{48}$$

14. Compute the deterministic commitments

$$C_{h,1}=\prod_{j=1}^{D+128\cdot m}g_{n+1-j}^{\delta_\theta\cdot\bar{\boldsymbol{a}}_\theta[j]-\delta_y\cdot y_j+\delta_{eq}\cdot t_j\cdot y_j+\delta_{\mathsf{dec}}\cdot\sum_{i=1}^{128}\mathbf{H}_\xi[i,j]}$$

$$C_{h,2}=\prod_{j=1}^{n}g_{n+1-j}^{\delta_\theta\cdot\bar{\boldsymbol{\theta}}_0[j]+\sum_{i=1}^{128}\delta_r\cdot\phi_i\cdot\mathbf{R}'[i,j]+\delta_e\cdot\omega_j}$$

$$\hat{C}_t=\prod_{j=1}^{n}\hat{g}_j^{t_j}$$

$$\hat{C}_{h,3}=\prod_{j=1}^{d+k}\hat{g}_{n+1-j}^{-\delta_\theta\cdot q\cdot\boldsymbol{\theta}_0[j]+\delta_r\cdot\sum_{i=1}^{128}\phi_i\cdot\mathbf{R}[i,d+k+4+j]}$$

$$\hat{C}_\omega=\prod_{i=1}^{d+k+4}\hat{g}_i^{\omega_i}$$

15. Compute an evaluation input

$$z=H_z\big(\mathbf{x},y,t,\phi,\xi,\theta,\omega,\boldsymbol{\delta},\hat{C}_e,C_e,C_R,\hat{C}_{\mathsf{bin}},C_{\tilde{r}},C_y,C_{h,1},C_{h,2},\hat{C}_t,\hat{C}_{h,3},\hat{C}_\omega\big)\in\mathbb{Z}_p$$

and evaluate the polynomials

$$P_{h,1}[X]=\sum_{j=1}^{D+128m}\Big(\delta_\theta\cdot\bar{\boldsymbol{a}}_\theta[j]-\delta_y\cdot y_j+\delta_{eq}\cdot t_j\cdot y_j+\delta_{\mathsf{dec}}\cdot\sum_{i=1}^{128}\mathbf{H}_\xi[i,j]\Big)\cdot X^{n+1-i}$$

$$P_{h,2}[X]=\sum_{j=1}^{n}\Big(\delta_\theta\cdot\bar{\boldsymbol{\theta}}_0[j]+\sum_{i=1}^{128}\delta_r\cdot\phi_i\cdot\mathbf{R}'[i,j]+\delta_e\cdot\omega_j\Big)\cdot X^{n+1-j}$$

$$P_t[X] = \sum_{i=1}^{n} t_i \cdot X^i, \tag{49}$$

$$P_{h,3}[X] = \sum_{j=1}^{d+k} \big(- \delta_\theta \cdot q \cdot \boldsymbol{\theta}_0[j] + \delta_r \cdot \sum_{i=1}^{128} \phi_i \cdot \mathbf{R}[i, d+k+4+j] \big) \cdot X^{n+1-j}$$

$$P_\omega[X] = \sum_{i=1}^{d+k+4} \omega_i \cdot X^i$$

on the input z. Let the valuations

$$(p_{h,1}, p_{h,2}, p_t, p_{h,3}, p_\omega) = (P_{h,1}(z), P_{h,2}(z), P_t(z), P_{h,3}(z), P_\omega(z)).$$

16. Compute

$$\chi = H_\chi\big(\mathbf{x}, y, t, \phi, \xi, \theta, \omega, \boldsymbol{\delta}, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y, C_{h,1}, C_{h,2}, \hat{C}_t, \hat{C}_{h,3}, \hat{C}_\omega, z, p_{h,1}, p_{h,2}, p_t, p_{h,3}, p_\omega\big).$$

Then, compute the polynomial $Q[X] = \sum_{i=0}^{n-1} q_i \cdot X^i$ such that

$$Q_{\mathsf{KZG}}[X] = \Big(P_{h,1}[X] + \chi \cdot P_{h,2}[X] + \chi^2 \cdot P_{h,3}[X] + \chi^3 \cdot P_t[X] + \chi^4 \cdot P_\omega[X]) - (p_{h,1} + p_{h,2} \cdot \chi + \chi^2 \cdot p_{h,3} + \chi^3 \cdot p_t + \chi^4 \cdot p_\omega) \Big) / (X - z)$$

Finally, compute $\pi_{\mathsf{KZG}} = g^{q_0} \cdot \prod_{i=1}^{n-1} g_i^{q_i} = g^{Q_{\mathsf{KZG}}(\alpha)}$.

Output the final proof

$$\boldsymbol{\pi} = \big(\hat{C}_e, C_e, C_{\tilde{r}}, C_R, \hat{C}_{\mathsf{bin}}, C_y, C_{h,1}, C_{h,2}, \hat{C}_t, \hat{C}_{h,3}, \hat{C}_\omega, \pi, \pi_{\mathsf{KZG}}\big).$$

$\mathsf{Verify}_{\mathsf{pp}}\big(\mathbf{x}, \boldsymbol{\pi}\big)$: Given $\mathbf{x} = (q, d, k, B_\infty, t, \tilde{\mathbf{A}}, \mathbf{c})$ and a candidate $\boldsymbol{\pi}$, return 0 if $\boldsymbol{\pi}$ does not parse properly. Otherwise, set $B = B_\infty \cdot \sqrt{d+k}$ and conduct the following steps.

1. Compute $\bar{\mathbf{R}} = H_R(\mathbf{x}, \hat{C}_e, C_e, C_{\tilde{r}}) \in \{-1, 0, 1\}^{128 \times (2(\bar{d}+\bar{k})+4)}$ of which the columns are distributed as per $\mathcal{D}^{128}$. Let $\mathbf{R} \in \{-1, 0, 1\}^{128 \times (2(d+k)+4)}$ the submatrix consisting of the first $2(d+k)+4$ columns of $\bar{\mathbf{R}}$.
2. Set $\phi = H_\phi(\mathbf{x}, \mathbf{R}, \hat{C}_e, C_e, C_R, C_{\tilde{r}})$, $\xi = H_\xi(\mathbf{x}, \hat{C}_e, C_e, \phi, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}})$, and $y = H\big(\mathbf{x}, \phi, \xi, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}\big) \in \mathbb{Z}_p$. Then, compute

$$t = H_t(\mathbf{x}, y, \phi, \xi, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y) \in \mathbb{Z}_p,$$
$$\theta = H_{\mathsf{lmap}}\big(\mathbf{x}, y, t, \phi, \xi, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y\big) \in \mathbb{Z}_p$$
$$\omega = H_\omega\big(\mathbf{x}, y, t, \phi, \xi, \theta, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y\big) \in \mathbb{Z}_p,$$

and

$$\boldsymbol{\delta} = (\delta_r, \delta_{\mathsf{dec}}, \delta_{eq}, \delta_y, \delta_\theta, \delta_e, \delta_\ell) \\ = H_{\mathrm{agg}}\big(\mathbf{x}, y, t, \phi, \xi, \theta, \omega, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y\big).$$

Let $\boldsymbol{\phi} = (\phi_1, \ldots, \phi_{128}), \boldsymbol{\xi} = (\xi_1, \ldots, \xi_{128}) \in \mathbb{Z}_p^{128}$ such that $\phi_i = \phi^{i-1}$ and $\xi_i = \xi^{i-1}$ for each $i \in [128]$. Define $\boldsymbol{\theta} = (\theta_1, \ldots, \theta_{\bar{d}+\bar{k}}) \in \mathbb{Z}_p^{\bar{d}+\bar{k}}$, where $\theta_i = \theta^{i-1}$ for each $i \in [\bar{d}+\bar{k}]$. Define $\boldsymbol{\omega} = (\omega_1, \ldots, \omega_n) \in \mathbb{Z}_p^n$, $\boldsymbol{t} = (t_1, \ldots, t_n)$ with $t_i = t^{i-1}$ and $\omega_i = \omega^{i-1}$ for each $i \in [n]$. Define $\boldsymbol{y} = (y_1, \ldots, y_n)$ with $y_i = y^{i-1}$ for each $i \in [D + 128 \cdot m]$ and $y_i = 0$ for each $i \in [D + 128 \cdot m + 1, n]$.

3. Define $\tilde{\mathbf{A}} = [\tilde{\mathbf{A}}_0 \mid -q \cdot \mathbf{I}_{d+k} \mid \mathbf{I}_{d+k}] \in \mathbb{Z}^{(d+k)\times(D+2(d+k))}$ and $\tilde{\mathbf{c}} \in \mathbb{Z}^{d+k}$ as in (16) with $\tilde{\mathbf{A}}_0 \in \mathbb{Z}^{(d+k)\times D}$. Let $\boldsymbol{\theta}_0 \in \mathbb{Z}_p^{d+k}$ the first $d+k$ entries of $\boldsymbol{\theta}$. Let $t_\theta = \boldsymbol{\theta}_0^\top \cdot \tilde{\mathbf{c}} \bmod p$ and $\boldsymbol{a}_\theta^\top = \boldsymbol{\theta}_0^\top \cdot \tilde{\mathbf{A}}_0 \in \mathbb{Z}_p^{1\times D} \bmod p$.

4. Let $\bar{\boldsymbol{\theta}}_0 = (\boldsymbol{\theta}_0 \mid \mathbf{0}^{n-(d+k)}) \in \mathbb{Z}_p^n$ and define the matrices $\mathbf{R}' \in \mathbb{Z}_p^{128\times n}$ and $\mathbf{H}_\xi \in \mathbb{Z}_p^{128\times(D+128\cdot m)}$ as in step 13 of Prove. Return 1 if the following equality holds and 0 otherwise:

$$e\left(C_y^{\delta_y} \cdot C_{h,1},\ \hat{C}_{\mathsf{bin}}\right) \cdot e\left(C_e^{\delta_\ell} \cdot C_{h,2}, \hat{C}_e\right) \cdot e\Big(C_{\tilde{r}}, \hat{C}_{h,3}\Big) \\ \cdot e\big(C_R, \prod_{j=1}^{128} \hat{g}_{n+1-j}^{\delta_r\cdot\phi_j + \delta_{\mathsf{dec}}\cdot\boldsymbol{\xi}[j]}\big)^{-1} \cdot e\big(C_e^{\delta_e}, \hat{C}_\omega\big)^{-1} \\ \cdot e\big(C_y^{\delta_{eq}}, \hat{C}_t\big)^{-1} \cdot e(g_1, \hat{g}_n)^{-\delta_\theta\cdot t_\theta - \delta_\ell \cdot B^2} = e(\pi, \hat{g}) \qquad (50)$$

5. Compute

$$z = H_z\big(\mathbf{x}, y, t, \phi, \xi, \theta, \omega, \boldsymbol{\delta}, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, \\ C_{\tilde{r}}, C_y, C_{h,1}, C_{h,2}, \hat{C}_t, \hat{C}_{h,3}, \hat{C}_\omega\big) \in \mathbb{Z}_p$$

and

$$\chi = H_\chi\big(\mathbf{x}, y, t, \phi, \xi, \theta, \omega, \boldsymbol{\delta}, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, \\ C_{\tilde{r}}, C_y, C_{h,1}, C_{h,2}, \hat{C}_t, \hat{C}_{h,3}, \hat{C}_\omega, z, p_{h,1}, p_{h,2}, p_t, p_{h,3}, p_\omega\big).$$

Compute $(p_{h,1}, p_{h,2}, p_{h,3}, p_t, p_\omega) = (P_{h,1}(z), P_{h,2}(z), P_{h,3}(z), P_t(z), P_\omega(z))$ by evaluating the polynomials in (49).

6. Return 0 if the following equality does not hold:

$$e\big(C_{h,1} \cdot C_{h,2}^{\chi} \cdot g^{-p_{h,1} - \chi\cdot p_{h,2}}, \hat{g}\big) \\ \cdot e\big(g, \hat{C}_{h,3}^{(\chi^2)} \cdot \hat{C}_t^{(\chi^3)} \cdot \hat{C}_\omega^{(\chi^4)} \cdot \hat{g}^{-(\chi^2)\cdot p_{h,3} - (\chi^3)\cdot p_t - (\chi^4)\cdot p_\omega}\big) = e(\pi_{\mathsf{KZG}}, \hat{g}_1 \cdot \hat{g}^{-z}). \qquad (51)$$

Otherwise, return 1.

We can optimize the verification algorithm by batching the two pairing product equations (50)-(51) and testing if

$$e\left(C_y^{\delta_y}\cdot C_{h,1},\ \hat{C}_{\mathsf{bin}}\right)\cdot e\left(C_e^{\delta_\ell}\cdot C_{h,2},\hat{C}_e\right)\cdot e\Big(C_{\tilde{r}}\cdot g^{\eta\cdot\chi^2},\hat{C}_{h,3}\Big)\cdot e\Big(C_R,\prod_{j=1}^{128}\hat{g}_{n+1-j}^{\delta_r\cdot\phi_j+\delta_{\mathsf{dec}}\cdot\boldsymbol{\xi}[j]}\Big)^{-1}$$
$$\cdot e\Big(C_e^{\delta_e}\cdot g^{-\eta\cdot\chi^4},\hat{C}_\omega\Big)^{-1}\cdot e\big(C_y^{\delta_{eq}}\cdot g^{-\eta\cdot\chi^3},\hat{C}_t\big)^{-1}\cdot e(g_n^{-\delta_\theta\cdot t_\theta-\delta_\ell\cdot B^2}\cdot\pi_{\mathsf{KZG}}^{-\eta},\hat{g}_1)$$
$$=e\big(\pi\cdot C_{h,1}^{-\eta}\cdot g^{\eta\cdot(p_{h,1}+\chi\cdot p_{h,2}+\chi^2\cdot p_{h,3}+\chi^3 p_t+\chi^4\cdot p_\omega)}\cdot C_{h,2}^{-\chi\cdot\eta}\cdot\pi_{\mathsf{KZG}}^{-z\cdot\eta},\hat{g}\big) \tag{52}$$

for a random $\eta\xleftarrow{R}\mathbb{Z}_p$.

Compared to the variant of Section 4, the proof size has slightly increased as each proof now requires 8 elements of $\mathbb{G}$ and 5 elements of $\hat{\mathbb{G}}$. On a BLS12-446 curve, it fits in 1003.5 bytes.

After the initial verification, the verifier can discard the helper commitments $(C_{h,1},C_{h,2},\hat{C}_{h,3},\hat{C}_t,\hat{C}_\omega)$ and only store the shorter proof $\big(\hat{C}_e,C_e,C_{\tilde{r}},C_R,\hat{C}_{\mathsf{bin}},C_y,\pi\big)$ that has the same shape as in Section 3.1 and can be verified using the slower verification algorithm.

We finally note that the verifier only needs to store $(g,g_n,\hat{g},\hat{g}_1,(g_{n+1-i})_{i=1}^{128})$ in order to verify a proof. However, the full CRS is needed to verify the shorter proof $\big(\hat{C}_e,C_e,C_{\tilde{r}},C_R,\hat{C}_{\mathsf{bin}},C_y,\pi\big)$.

Theorem 4. *Under the same conditions as in Theorem 2, the above variant provides simulation-extractability in the AGM+ROM if the $(2n,n)$-DLOG assumption holds.*

Proof. The proof proceeds identically to the proof of Theorem 3 and considers possibly malformed deterministic commitments $(C_{h,1},C_{h,2},\hat{C}_t,\hat{C}_{h,3},\hat{C}_\omega)$. Let the real polynomials $(P_{h,1}[X],P_{h,2}[X],P_t[X],P_{h,3}[X],P_\omega[X])$ defined in (49). From the algebraic representations of commitments $(C_{h,1},C_{h,2},\hat{C}_t,\hat{C}_{h,3},\hat{C}_\omega)$, the reduction can compute $Q_{h,1}[X]$, $Q_{h,2}[X]$, $Q_t[X]$, $Q_{h,3}[X]$ and $Q_\omega[X]$ such that $C_{h,i}=g^{Q_{h,i}(\alpha)}$ for each $i\in\{1,2\}$, $\hat{C}_{h,3}=\hat{g}^{Q_{h,3}(\alpha)}$, $\hat{C}_\omega=\hat{g}^{Q_\omega(\alpha)}$ and $\hat{C}_t=\hat{g}^{Q_t(\alpha)}$. If at least one of the deterministic commitments is malformed, we must have

$$(Q_{h,1}[X],Q_{h,2}[X],Q_t[X],Q_{h,3}[X],Q_\omega[X])$$
$$\neq(P_{h,1}[X],P_{h,2}[X],P_t[X],P_{h,3}[X],P_\omega[X]).$$

By Schwartz-Zippel, this implies

$$Q_{h,1}[X]+\chi\cdot Q_{h,2}[X]+\chi^2\cdot Q_{h,3}[X]+\chi^3\cdot Q_t[X]+\chi^4\cdot Q_\omega[X]$$
$$\neq P_{h,1}[X]+\chi\cdot P_{h,2}[X]+\chi^2\cdot P_{h,3}[X]+\chi^3\cdot P_t[X]+\chi^4\cdot P_\omega[X] \tag{53}$$

with overwhelming probability $\geq 1-4/p$ over the choice of χ since

$$(\mathbf{x},C_{h,1},C_{h,2},\hat{C}_t,\hat{C}_{h,3},\hat{C}_\omega,\phi,y,t,\boldsymbol{\delta},\omega,\xi,\theta)$$

are included in the inputs of H_χ and the algebraic representation of commitments completely determine the polynomials $(P_{h,1}[X], P_{h,2}[X], P_t[X], P_{h,3}[X], P_\omega[X])$ and $(Q_{h,1}[X], Q_{h,2}[X], Q_t[X], Q_{h,3}[X], Q_\omega[X])$.[18]

From the KZG verification equation (51), we know that

$$\begin{aligned}(Q_{h,1}(\alpha) - p_{h,1}) + \chi \cdot (Q_{h,2}(\alpha) - p_{h,2}) + \chi^2 \cdot (Q_{h,3}(\alpha) - p_{h,3}) \\ + \chi^3 \cdot (Q_t(\alpha) - p_t) + \chi^4 \cdot (Q_\omega(\alpha) - p_\omega) = \pi(\alpha) \cdot (\alpha - z)\end{aligned}$$

where $\pi[X]$ is defined by the algebraic representation of $\pi_{\mathsf{KZG}} = g^{\pi(\alpha)}$. Then, the polynomial identity

$$\begin{aligned}(Q_{h,1}[X] - p_{h,1}) + \chi \cdot (Q_{h,2}[X] - p_{h,2}) + (Q_{h,3}[X] - p_{h,3}) \\ + \chi^3 \cdot (Q_t[X] - p_t) + \chi^4 \cdot (Q_\omega[X] - p_\omega) = \pi[X] \cdot (X - z)\end{aligned} \tag{54}$$

must hold unless the reduction can compute $\alpha \in \mathbb{Z}_p$ by factoring a non-zero polynomial. Since $(p_{h,1}, p_{h,2}, p_t, p_{h,3}, p_\omega) = (P_{h,1}(z), P_{h,2}, P_t(z), P_{h,3}(z), P_\omega(z))$, the identity (54) then implies

$$\begin{aligned}(Q_{h,1}(z) - P_{h,1}(z)) + \chi \cdot (Q_{h,2}(z) - P_{h,2}(z)) + (Q_{h,3}(z) - P_{h,3}(z)) \\ + \chi^3 \cdot (Q_t(z) - P_t(z)) + \chi^4 \cdot (Q_\omega(z) - P_\omega(z)) = 0\end{aligned} \tag{55}$$

Due to the inequality (53), (55) is only possible with negligible probability $\leq n/p$ because the input

$$z = H_z\big(\mathbf{x}, y, t, \phi, \xi, \theta, \omega, \boldsymbol{\delta}, \hat{C}_e, C_e, C_R, \hat{C}_{\mathsf{bin}}, C_{\tilde{r}}, C_y, C_{h,1}, C_{h,2}, \hat{C}_t, \hat{C}_{h,3}, \hat{C}_\omega\big)$$

is defined after $(Q_{h,1}[X], Q_{h,2}[X], Q_t[X], Q_{h,3}[X], Q_\omega[X], P_{h,1}[X], P_{h,2}[X], P_t[X], P_{h,3}[X], P_\omega[X])$.

We conclude that, if an algebraic cheating prover sends incorrect deterministic commitments such that

$$(C_{h,1}, C_{h,2}, \hat{C}_t, \hat{C}_{h,3}, \hat{C}_\omega) \neq (g^{P_{h,1}(\alpha)}, g^{P_{h,2}(\alpha)}, \hat{g}^{P_t(\alpha)}, \hat{g}^{P_{h,3}(\alpha)}, \hat{g}^{P_\omega(\alpha)}),$$

then it can only provide an accepting π_{KZG} with negligible probability under the $(2n, n)$-DLOG assumption. □

[18] The matrix $\mathbf{R}$ is not explicitly included in the inputs of H_χ but it is an input of H_ϕ, which is used to derive ϕ. So, χ is determined after $\mathbf{R}$.